関数型言語
OCamlで作る代数計算機

代数構造の構築から、方程式の厳密解法・微分積分まで



icchon 著

関数型言語OCamlで作る
代数計算機

icchon 編著

2026-10-10　版

発行

前書き

本書は、群・環・体・ベクトル空間・多項式環・拡大体といった代数構造を OCaml で実装しながら、方程式を代数的に解き、微分・積分までを実装していく本です。代数学を網羅する教科書ではなく、その目的地にたどり着くために必要な道具だけを、応用先から逆算して最短経路で揃える一冊です。

想定している読者は、高校までの数学は一通り身についているけれど、それを厳密に使う機会はなかった、という人です。OCaml や関数型プログラミングの経験は前提にしていません（なお、著者自身、工学系の出身で、数学を専門に学んだわけではありません）。数学の世界をコンピュータに落とし込む手法に興味がある方、あるいは関数型プログラミングの直接的な応用例が見たい方であれば、楽しんでいただけると思っています。

読み進める前に、本書が意図的に扱わないことを述べておきます。

群・環・体といった言葉は本文に繰り返し登場しますが、結合律や分配律、可換性が実際に成り立つことの証明はほとんど行わず、成り立っていることを前提に実装へ進みます。厳密な代数学の教科書ではなく、代数の構造をプログラムとして動かしてみる本だからです。

「関数型言語と代数」と聞くと、モナドや圏論、あるいは表示的意味論のような話を連想するかもしれません。しかし本書が扱うのは、高校や大学で出会うような数式、多項式、そして群や体といった、本来の数学としての代数学です。本書が目指すのは、数値計算機ではなく**代数計算機**をつくることです。

読み終える頃には、数学的な構造やその制約が、プログラミング言語の型としてほとんど 1 対 1 で書き下せることを、実装を通して確認できるはずです。数学の面白さと、プログラミングの面白さを、同時に楽しんでもらえたら嬉しいです。

免責事項

本書に記載された内容は、情報の提供のみを目的としています。したがって、本書を用いた開発、製作、運用は、必ずご自身の責任と判断によって行ってください。これらの情報による開発、製作、運用の結果について、著者はいかなる責任も負いません。

目次

前書き **2**
免責事項 3

第 1 章 「代数計算機を作る」とは？ **6**
1.1 数値的に解くか、代数的に解くか 7
1.2 「代数的に解く」の 3 つの段階 8
1.3 代数構造をファンクタとして実装する 9
1.4 前提知識 9
1.5 本書の構成 9

第 2 章 環境構築 **10**
2.1 OCaml をインストールする 11
2.2 コード例を動かしてみる 12
2.3 Overleaf で実行結果を確認する 13

第 3 章 本書で使う OCaml の構文 **15**
3.1 値と関数の束縛 16
3.2 タプルとパターンマッチ 16
3.3 モジュールとシグネチャ 17
3.4 ファンクタ 17
3.5 include 20
3.6 シグネチャ制約 with type := 21

第 4 章 基礎的な代数をつくる **22**
4.1 整数 25
4.2 有理数 34

4.3 複素数 46
4.4 ベクトル 62
4.5 多項式 67
4.6 体の拡大 75

第 5 章 二次方程式 80
5.1 二次方程式を解く 81

第 6 章 微分・積分 99
6.1 微分を導入する 102
6.2 積分を導入する 108
6.3 2 階線形微分方程式を解く 130

あとがき 133

参考文献 134

著者紹介 135

1

第 1 章

「代数計算機を作る」とは？

第 1 章

「代数計算機を作る」とは？

本書は、数学の計算を近似ではなく厳密な式のまま行う計算機を、OCaml の型とファンクタで実装します。応用先は方程式の求解・微分・積分です。この章では、本書が採用するアプローチとその背景を説明します。

1.1 数値的に解くか、代数的に解くか

数学の計算をプログラムにするには、大きく分けて 2 つのアプローチがあります。1 つは、ニュートン法のような数値計算で、近似的な数値を求める方法。もう 1 つは、厳密な答えを式として書き下す、代数的な方法です。

たとえば $x^2 - 2 = 0$ という方程式を考えてみましょう。数値的に解けば、ニュートン法などのアルゴリズムによって $x \approx 1.41421356\ldots$ のような近似値が得られます。一方、代数的に解けば、$x = \pm\sqrt{2}$ という厳密な式がそのまま答えになります。

同じ選択を迫られるのは、方程式を解くときだけではありません。$f(x) = x^2$ の導関数を求める場合、数値的には $(f(x+h) - f(x))/h$ を小さな h で計算した近似値しか得られませんが、代数的には $f'(x) = 2x$ という式がそのまま手に入ります。積分も同様です。$\int x\,dx$ を数値的に区分求積で計算すれば近似的な面積しか得られませんが、代数的には $x^2/2$ という式が直接手に入ります。

本書がつくるのは、どの場面でも後者、厳密な式のまま答えを求める計算機です。この考え方を、本書の後半で方程式の求解から微分・積分へと広げていきます。

1.2 「代数的に解く」の 3 つの段階

「代数的に計算するプログラムをつくる」と一言で言っても、その実現のしかたには深さの違いがあります。2 次方程式を解く例で見てみましょう。

もっとも素朴なのは、解の公式

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{1.1}$$

に数値を当てはめて、その結果を文字列として組み立てて表示するだけのプログラムです。$x^2 - 4x + 3 = 0$ であれば $a = 1, b = -4, c = 3$ なので、

$$x = \frac{4 \pm \sqrt{4}}{2} \tag{1.2}$$

という文字列を出力して終わり、というイメージです。根号の中身 4 が完全平方数であっても、簡約すら行われません。

もう少し頑張ると、$\sqrt{4}$ のような式を見つけたら 2 に簡約する、といった書き換えルールをたくさん用意して、式が変化しなくなるまで機械的に適用し続けるプログラムになります。先ほどの式に「$\sqrt{4} \to 2$」というルールを適用すると

$$x = \frac{4 \pm 2}{2} \tag{1.3}$$

となり、さらに約分するルールまで適用すれば、$x = 3$ または $x = 1$ という最終的な答えにたどり着きます。世の中の数式処理システムの多くは、突き詰めるとこの「式の書き換え」を土台にしています。しかし、この方法は式を単なる「構文木（記号の並び）」として扱うだけで、背後にある体や環といった数学的構造を捉えていないため、例外のたびにルールが際限なく増え続けるという構造的な限界を抱えています。

本書がつくろうとしているのは、これらとは少し違う道です。環・体・拡大体といった代数構造そのものを OCaml の**モジュール**として構築し、四則演算をそのモジュールに対して正しく定義された演算として実装します。式の書き換えによる方法が最終的に得るのは表示用の文字列ですが、本書の方法で得られるのは、ちゃんと**定義された代数構造の値**そのものです。そのため、得られた解に対してそのまま計算を続けたり、検算したりすることが、特別な工夫なしにできます。

1.3 代数構造をファンクタとして実装する

たとえば減法は、「加法と逆元さえあれば自動的につくれる」ものです。本書ではこの考え方を、整数から体の拡大まですべての代数構造・演算に一貫して当てはめ、ある性質を満たす代数から別の演算を自動生成するしくみとして、群・モノイド・環・体を実装します。OCaml ではこのしくみを**ファンクタ**（*Functor*）と呼び、具体的な書き方は後の「本書で使う OCaml の構文」の章で説明します。本書は、こうした代数構造の構築を、OCaml の**モジュール**（*Module*）とファンクタだけでゼロから組み立てる試みです。

1.4 前提知識

高校程度の数学（方程式、複素数、ベクトル、微分・積分の基本）があれば十分です。群・環・体といった抽象代数の知識は前提とせず、必要になった順に定義していきます。

OCaml も、関数型プログラミングの経験は必須ではありません。必要な構文は後の章でまとめて説明します。コードの実行結果は本文中にそのまま示すので、OCaml を実行しなくても最後まで読み進められます。手を動かしたい方向けに環境構築の章でセットアップ方法も説明しますが、任意です。

1.5 本書の構成

本書は、大きく 2 部で構成されています。

基礎的な代数をつくるでは、整数から出発し、有理数・複素数・ベクトル・多項式・体の拡大と代数構造を 1 つずつ実装します。同じファンクタが整数にも有理数にも複素数にも使い回され、「群・環・体」という抽象的な分類がファンクタの入出力を特徴づける具体的なシグネチャとして扱えることを確認します。

続く応用パートでは、ここまでつくったファンクタを、方程式・微分・積分という応用に展開していきます。方程式の章では 2 次方程式を扱います。

これらの章に入る前に、次の「環境構築」で OCaml を動かす準備を整え、続く「本書で使う OCaml の構文」で必要な構文を確認します。整数から始めましょう。

■

2

第 2 章

環境構築

第 2 章

環境構築

本書のコードとその実行結果は、本文中にそのまま示すので、この章を読まなくても最後まで読み進められます。この章は、実際に手元でコードを動かしてみたい方のためのものです。次の 2 つの環境を整えます。

- OCaml のコンパイラ（`ocamlopt`）: 本書のコード例を実際にコンパイル・実行するため
- **Overleaf**: コードの実行結果（LaTeX 形式の文字列）を、数式として組版された形で確認するため

2.1 OCaml をインストールする

OCaml 本体は、**opam**（*OCaml Package Manager*）というパッケージマネージャ経由でインストールするのが一般的です。

2.1.1 macOS の場合

Homebrew が入っていれば、次のコマンドだけで opam がインストールできます。

```
brew install opam
```

2.1.2 Linux の場合

多くのディストリビューションでは、公式のインストールスクリプトを使うのが手軽です。

```
bash -c "sh <(curl -fsSL https://opam.ocaml.org/install.sh)"
```

Debian/Ubuntu 系であれば、apt からインストールすることもできます。

```
sudo apt install opam
```

2.1.3 Windows の場合

WSL（*Windows Subsystem for Linux*）を使い、WSL 上の Linux 環境（Ubuntu など）に対して、上記の Linux 向けの手順を実行してください。

2.1.4 OCaml 本体のセットアップ

opam のインストールができたら、次のコマンドを実行します。初回だけ必要な作業です。

```
opam init
opam switch create default 5.5.1
eval $(opam env)
```

途中でいくつか質問されますが、基本的にはデフォルト（Enter キーを押すだけ）で問題ありません。新しくターミナルを開いたときは、`eval $(opam env)` をもう一度実行してください（面倒であれば、`.zshrc` や `.bashrc` に追記しておくとよいでしょう）。

最後に、正しくインストールできたか確認しましょう。

```
ocaml -version
>>
The OCaml toplevel, version 5.5.1
```

バージョン番号が表示されれば成功です。

2.2 コード例を動かしてみる

次のようなファイル `sample.ml` を用意します。

▲リスト 2.1 "sample.ml"

```
let () = print_endline "hello OCaml"
```

これを次のコマンドでコンパイル・実行します。

```
ocamlopt sample.ml && ./a.out
>>
hello OCaml
```

本書のコード例も、これと同じように `ocamlopt` でコンパイル・実行できます。

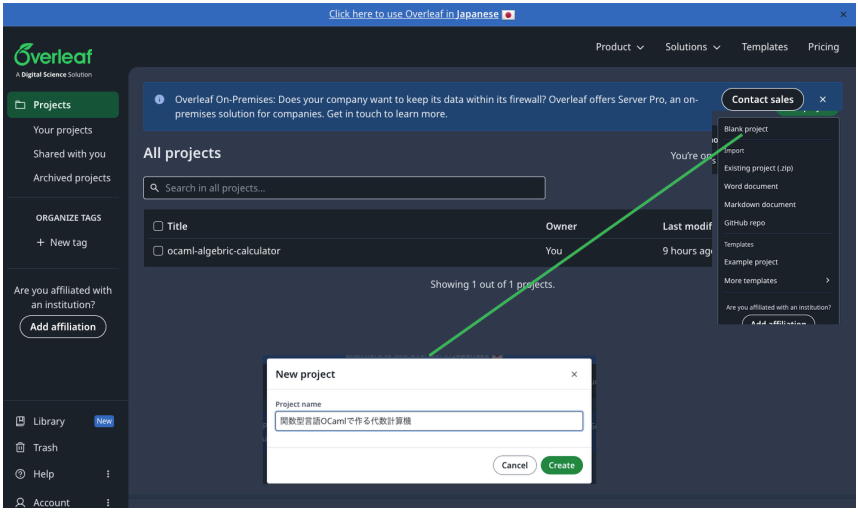
2.3 Overleaf で実行結果を確認する

本書のテストコードは、計算結果を

```
\[2 + 3 = 5\]
```

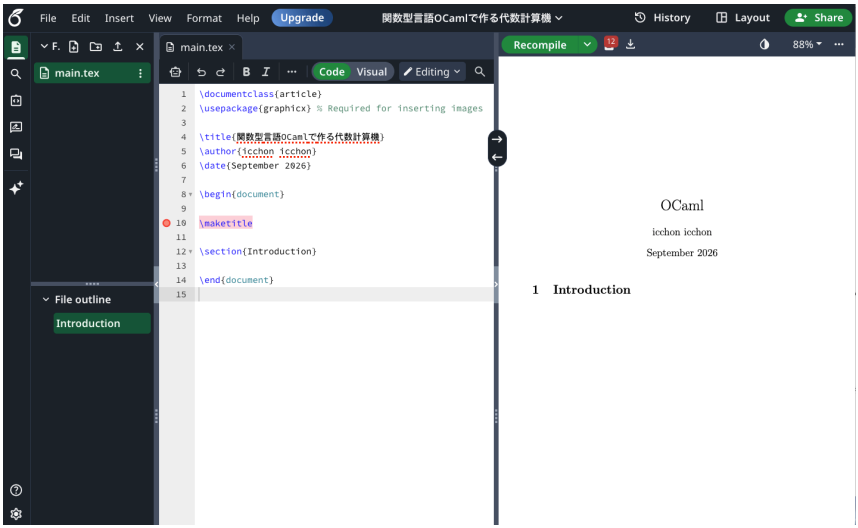
のような、LaTeX 形式の文字列として出力します。このままでは数式として読みづらいので、Overleaf というサービスを使って、実際に組版された形で確認します。Overleaf は、ブラウザ上で LaTeX の文書を編集・コンパイルできる無料のサービスで、手元に LaTeX 環境を用意する必要がありません。

使い方は簡単です。https://www.overleaf.com にアクセスしてアカウントを作成したら、ダッシュボードの「New Project」から「Blank Project」を選び、プロジェクト名を入力して作成します。



▲図 2.1　Overleaf で「Blank Project」から新しいプロジェクトを作成する

作成すると、次のようなエディタ画面が開きます。左がファイル一覧、中央が `main.tex` の編集画面、右がコンパイル結果のプレビューです。



▲図 2.2　Overleaf のエディタ画面

中央のコードを、次のような最小限のテンプレートに置き換えます。

▲リスト 2.2　"main.tex"

```
\documentclass{article}
\begin{document}

(ここにOCamlの実行結果を貼り付けます)

\end{document}
```

あとは、OCaml のコード例を実行して出力された文字列を`\begin{document}`と`\end{document}`の間に貼り付け、画面上部の「Recompile」を押すだけです。数式として組版された結果が確認できます。

これで、本書のコードを実際に手を動かしながら読み進める準備が整いました。

■

3

第 3 章

本書で使うOCamlの構文

第 3 章

本書で使う OCaml の構文

本書のコードは、OCaml の標準的な構文だけで書かれています。`module` や `functor` を多用するので、数学的な内容の説明のたびに構文の説明を挟まずに済むよう、以降の章で繰り返し使う構文をこの章でまとめて説明しておきます。

3.1 値と関数の束縛

OCaml では、`let` で値や関数を名前に束縛します。「定義」ではなく**束縛**（*Binding*）と呼ぶのは、`let` が新しい名前を導入して既存の値と結びつけるだけの構文であり、変数への再代入のように、束縛した値そのものをあとから書き換えることはできないためです。型注釈は `(引数名: 型)` や `: 戻り値の型`のように書きます。関数の中で自分自身を呼び出す場合は、`let` の代わりに `let rec` を使います。

▲リスト 3.1　"let / let rec"

```
let x: int = 3
let add (a: int) (b: int): int = a + b
let rec sum (n: int): int = if n = 0 then 0 else n + sum (n - 1)
```

3.2 タプルとパターンマッチ

複数の値をまとめて扱うときは、`(a, b)` のようなタプルを使います。タプルの中身を取り出すには、`let (a, b) = pair in ...` のように書きます。

値によって処理を分岐させるときは `match ... with` を使います。値が存在するかどうかわからない場合の型として `option` 型があり、値があれば `Some x`、なければ `None` という形で表現します。

▲リスト 3.2　"タプルと match"

```
let swap (pair: int * int): int * int =
  let (a, b) = pair in
  (b, a)

let describe (n: int option): string =
  match n with
  | Some x -> string_of_int x
  | None -> "none"
```

3.3　モジュールとシグネチャ

OCaml の**モジュール**（*module*）は、型や値をひとまとめにした単位です。モジュールが満たすべき型や値の一覧を**シグネチャ**（*signature*）と呼び、`module type ... = sig ... end` で定義します。

▲リスト 3.3　"module と module type"

```
module type COUNTER = sig
  type t
  val zero: t
  val succ: t -> t
end

module Int_counter: COUNTER with type t = int = struct
  type t = int
  let zero = 0
  let succ x = x + 1
end
```

`module Int_counter: COUNTER with type t = int = struct ... end` の `: COUNTER with type t = int` の部分は、`Int_counter` が `COUNTER` というシグネチャを満たすこと、かつ `Int_counter.t` が具体的には `int` であることを表しています。

3.4　ファンクタ

ファンクタ（*functor*）は、モジュールを受け取って別のモジュールを返す、モジュールに対する関数です。`functor (M: シグネチャ) -> struct ... end` という形で定義します。本書では、ある代数構造を受け取って、別の演算を自動生成する道具としてファンクタを使います。

> ※ Haskell の `Functor` 型クラスや、圏論における**関手**（*Functor*）をご存知の方は、同じ「ファンクタ」という名前に混乱するかもしれません。これらは対象と射の対応関係を保つ写像という圏論由来の概念ですが、OCaml のファン

クタはこれとは無関係で、単に「モジュールを引数に取り、モジュールを返すもの」という、モジュール言語の中だけで完結した機能を指す用語です。

まず、ファンクタが受け取るモジュールの条件を、シグネチャとして決めておきます。

▲リスト 3.4　"シグネチャ ADD"

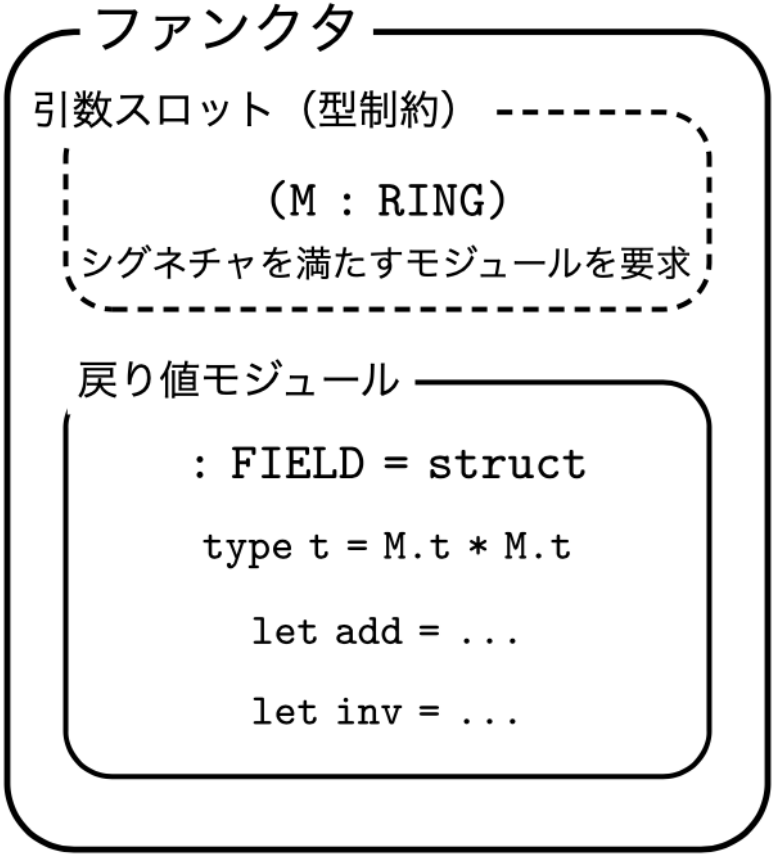
```
module type ADD = sig
  type t
  val zero: t
  val add: t -> t -> t
end
```

このシグネチャを満たすモジュール M を受け取り、M.t を 2 つ組にした型の上で成分ごとの加法を行うモジュールを返すファンクタが、次の Pair_add です。

▲リスト 3.5　"functor Pair_add"

```
module Pair_add (M: ADD): ADD with type t = M.t * M.t = struct
  type t = M.t * M.t
  let zero = (M.zero, M.zero)
  let add (a1, a2) (b1, b2) = (M.add a1 b1, M.add a2 b2)
end
```

シグネチャ ADD さえ満たしていれば、どんなモジュールに対してもこのファンクタを適用できます。



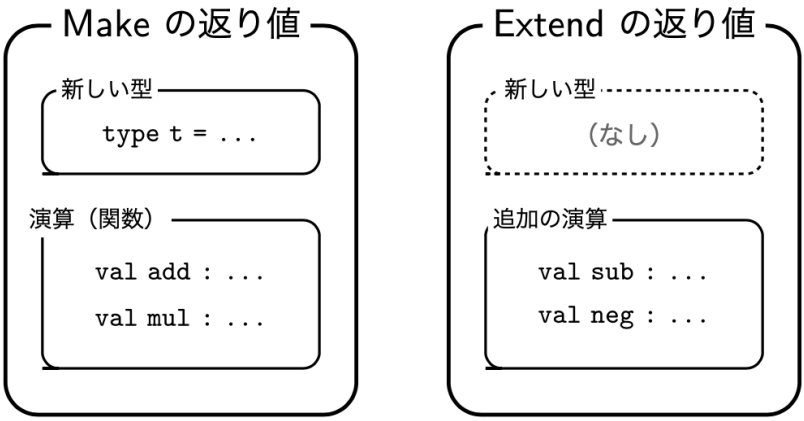
▲図 3.1　ファンクタのイメージ図

本書では、この先登場するファンクタを大きく 2 種類に分けて扱います。

Make ファンクタは、`Pair_add` のように、受け取ったモジュールをもとに**型ごと新しい代数を作って返す**ファンクタです。返ってくる型は、渡した型とは別物（今回なら `M.t` の 2 つ組）になります。返す代数の性質（シグネチャ）が、そのまま別のファンクタの入力として再利用されることも多いので、本書では Make ファンクタについては、実装の前に入力シグネチャと出力シグネチャの**両方**を必ず示すようにし

ます。

Extend ファンクタは、これから何度も登場する `Add_group.Extend` のように、**もとの型はそのまま**で、そこに新しい演算だけを追加して返すファンクタです。返ってくるのは追加された演算（たとえば減法）だけを持つ小さなモジュールで、呼び出し側でもとのモジュールと `include` して組み合わせて初めて意味を持ちます。Extend ファンクタの返り値だけを取り出して、別のファンクタの入力シグネチャとして使うことはないので、本書では Extend ファンクタについては入力シグネチャだけを示し、出力側にわざわざ名前をつけることはしません。



▲図 3.2　Make ファンクタと Extend ファンクタの違い

3.5　include

`include` は、他のモジュールの中身を、そのまま今のモジュールに取り込む構文です。

▲リスト 3.6　"include"

```
module Pair_add_with_double (M: ADD) = struct
  include Pair_add (M)
  let double (x: t): t = add x x
end
```

`Pair_add_with_double` は、`Pair_add (M)` の中身（`type t`・`zero`・`add`）をそ

のまま引き継いだうえで、double を追加しています。本書では、ファンクタが返したモジュールに include で演算を足していく、という形が何度も出てきます。

3.6 シグネチャ制約 with type :=

複数のシグネチャがそれぞれ独自に type t を宣言していると、include で組み合わせたときに type t が二重に宣言されてしまい、コンパイルエラーになります。これを避けるのが with type t := 型という制約で、シグネチャの中の type t という宣言を消し、指定した型に直接置き換えます。

▲リスト 3.7　"with type :=で衝突を避ける"

```
module type MUL = sig
  type t
  val one: t
  val mul: t -> t -> t
end

module type RING = sig
  type t
  include ADD with type t := t
  include MUL with type t := t
end
```

ADD と MUL はどちらも独自の type t を持つので、RING の中でそのまま include すると type t が衝突します。with type t := t によって、ADD・MUL それぞれの type t を RING 自身の type t に置き換えているため、衝突が起きません。本書では、複数の代数的な性質を 1 つの型の上に組み合わせる場面で、この書き方を繰り返し使います。

以上が、本書のコードを読むうえで前提となる OCaml の構文です。次の章から、これらの構文を使って実際に代数構造を実装していきます。

■

4

第 4 章

基礎的な代数をつくる

第 4 章

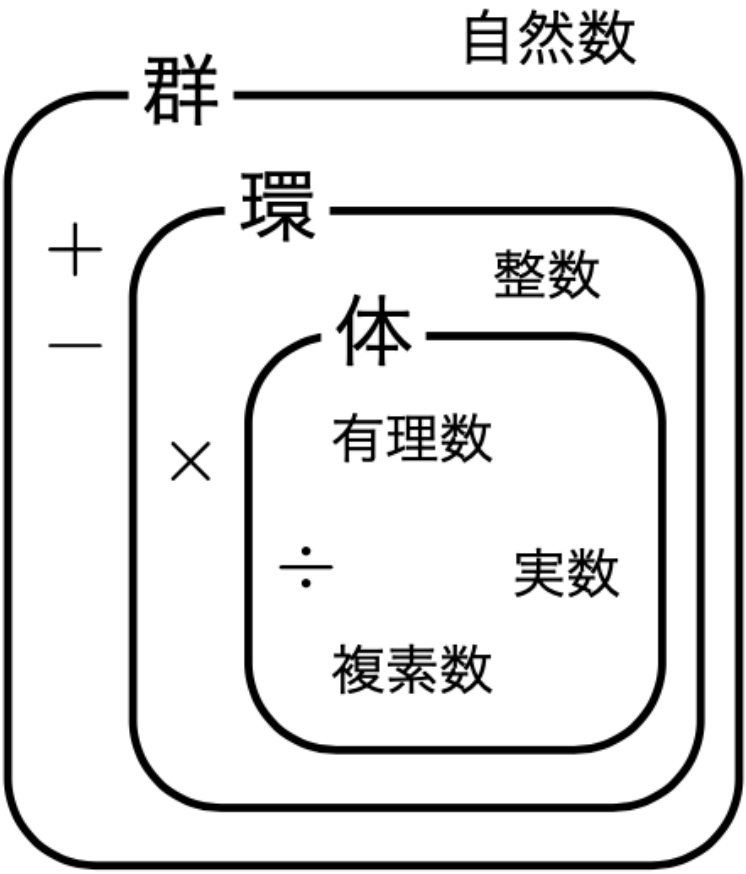
基礎的な代数をつくる

この章では、整数・有理数・複素数・ベクトル・多項式・体の拡大といった、見慣れた数の体系を OCaml で一から実装していきます。加法、乗法、逆元、など演算そのものが満たすべき性質に注目し、その性質をシグネチャとファンクタという形でコードに落とし込んでいきます。

たとえば、整数の減法は、加法と符号反転さえあれば自動的に生成できます。有理数の除法も、乗法と逆数さえあれば自動的に生成できます。このように、「ある性質を満たす代数を受け取ったら、別の演算を自動生成する」ファンクタを積み重ねていくと、整数・有理数・複素数・ベクトル・多項式・体の拡大という一見バラバラに見える代数たちが、実は同じファンクタの組み合わせでできていることが見えてきます。

これから登場する代数的な構造には、次のような大まかな階層があります。

- **モノイド**（*Monoid*）：二項演算と単位元を持つ（逆元は持たない）
- **群**（*Group*）：モノイドに、逆元の存在が加わったもの
- **環**（*Ring*）：加法については群、乗法についてはモノイドであるもの
- **体**（*Field*）：環に、0 以外のすべての元に対する乗法の逆元が加わったもの



▲図 4.1　群環体のイメージ

これらの正式な定義は、整数・有理数を実際につくりながら、この章の中で少しずつ登場します。今はまだ名前だけ頭の片隅に置いておいてください。

4.1 整数

4.1.1 基本的な機能

まずは整数をつくってみましょう。整数に欲しい演算として、最初に

- 加法（+）
- 減法（−）
- 乗法（×）

の 3 つを考えます。除法（÷）は、結果が整数の範囲からはみ出すことがある（$1 \div 2$ は整数になりません）ため、ここでは扱いません。

これらの演算を実現するには、等価判定と単位元（**0** と **1**）も必要です。ひとまずコードに起こしてみましょう。

▲リスト 4.1　integer.ml

```
module BASE = struct
type t = int
(* 等価判定 *)
let equal x1 x2 = x1 = x2
let zero = 0
(* 加法 *)
let add x1 x2 = x1 + x2
let sub x1 x2 = x1 - x2
let one = 1
(* 乗法 *)
let mul x1 x2 = x1 * x2
end
```

※「整数をつくる」と言いながら `type t = int` のように OCaml 組み込みの整数を使うのは反則では、と思うかもしれません。しかし、内部表現に何を使うかは本質ではありません。本書がこれから見ていく**代数**とは、ある型がどんな演算を持ち、それらの演算がどんな性質（結合律や逆元の存在など）を満たすかという、内部表現より一段抽象度の高い話だからです。

加法・減法・乗法が定義できました。ここで、減法に注目してみましょう。

減法は $a - b = a + (-b)$、つまり加法と**逆元**（*Inverse Element*）の組み合わせとして書き直せます。

```
let sub x1 x2 = add x1 (neg x2)
```

つまり減法は、加法と逆元さえあれば自動的に導けるアルゴリズムであり、他の演算と並べて個別に定義する必要はありません。この自動的に導く部分を、ファンクタとして切り出してみましょう。

▲リスト 4.2　add_group.ml

```
module type MIN = sig
  type t
  val add: t -> t -> t
  val neg: t -> t
end

module Extend (M: MIN) = struct
  (* 減法 *)
  let sub x1 x2 = M.add x1 (M.neg x2)
end
```

▲リスト 4.3　integer.ml

```
module BASE = struct
type t = int
(* 等価判定 *)
let equal x1 x2 = x1 = x2
let zero = 0
(* 加法 *)
let add x1 x2 = x1 + x2
(* 符号反転 *)
let neg x = -x
let one = 1
(* 乗法 *)
let mul x1 x2 = x1 * x2
end
include Add_group.Extend(BASE)
```

整数の基本演算をまとめたモジュールを加法群ファンクタ（`Add_group.Extend`）に渡し、そのモジュールを include することで、加法と逆元から減法を自動的に追加しています。

ところで、ここでファンクタを定義したモジュール[*1]に Add_group という名前を付けました。group とは**群**（*Group*）という数学用語のことです。群とは、集合 G と二項演算 $\odot$ の組 $(G, \odot)$ であって、**群の公理**（*Group Axioms*）と呼ばれる次の 4 つの性質を満たすもののことです。

$$
\begin{aligned}
&\text{閉性} && a, b \in G \implies a \odot b \in G \\
&\text{結合律} && (a \odot b) \odot c = a \odot (b \odot c) \\
&\text{単位元の存在} && \exists e \in G,\ a \odot e = e \odot a = a \\
&\text{逆元の存在} && \exists a^{-1} \in G,\ a \odot a^{-1} = a^{-1} \odot a = e
\end{aligned}
\tag{4.1}
$$

[*1] OCaml ではファイル名の先頭を大文字にしたものがモジュール名になる

$(\mathbb{Z}, +)$、つまり整数と加法の組で確認してみましょう。閉性・結合律は、2 つの整数を足せばまた整数になり、足す順番を変えても結果が変わらないことから直感的に成り立ちます。単位元は 0 です。そして、任意の整数 a に対する逆元は $-a$ であり、

$$a + (-a) = (-a) + a = 0 \tag{4.2}$$

が成り立ちます。$(\mathbb{Z}, +)$ は群の公理をすべて満たしているため、**群**（*Group*）です。特に演算が加法であることを強調して、**加法群**（*Additive Group*）と呼びます。

群の定義に「逆元の存在」が含まれていることから、加法群でありさえすれば減法は機械的に導出できます。これが、さきほど実装した加法群ファンクタ（`Add_group.Extend`）、すなわち**加法群を引数にとり、減法を生成するファンクタ**の正体です。

加法群でありさえすれば、整数でなくても構いません。有理数や複素数、あるいは多項式のような代数でも、同じファンクタから自動で減法が生成されます。

ただし、自然数（$0, 1, 2, \ldots$）は、たとえば 1 に対する逆元 -1 が自然数の中に存在しないため、逆元が定義できず、加法群ファンクタ（`Add_group.Extend`）を適用できません。

さて、整数モジュール（`Integer`）をもう少しだけ充実させましょう。次の関数を追加します。

- compare: 比較
- div_rem: 商と余り

▲リスト 4.4　integer.ml

```
module BASE = struct
...
(* 大小比較 *)
let compare x1 x2 = x1 - x2
(* 商と余り *)
let div_rem x1 x2 = (x1 / x2, x1 mod x2)
end
```

これで基本的な関数は揃いました。これらの関数を用いて新しい関数をもう少し追加してみましょう。

4.1.2　最大公約数

最大公約数を定義するには何が必要でしょうか。試しに、一度書いてみましょう。

```
let rec gcd a b =
    if b = 0 then a
    else gcd b (a mod b)
```

このアルゴリズムを**ユークリッドの互除法**（*Euclidean Algorithm*）といいます。なぜこれで最大公約数が求まるのか、簡単に確認しておきましょう。a を b で割った商を q、余りを r とすると、

$$a = qb + r \tag{4.3}$$

と書けます。この式を見ると、a と b を共通に割り切る数は、$r = a - qb$ も割り切ることがわかります。逆に、b と r を共通に割り切る数は、$a = qb + r$ も割り切ります。つまり a, b の公約数の集合と、b, r の公約数の集合はまったく同じであり、

$$\gcd(a, b) = \gcd(b, r) = \gcd(b, a \bmod b) \tag{4.4}$$

が成り立ちます。この関係を使えば、(a, b) の最大公約数を求める代わりに、より小さい $(b, a \bmod b)$ の最大公約数を求めればよいことになります。割った余りは必ずもとの数より小さくなるため、この置き換えを繰り返すといつか b が 0 になり、そこでの a がそのまま最大公約数です（a を 0 で割った余りは考えられないので、これ以上小さくできないところで止まります）。

必要なのは、

- 等価判定
- 加法単位元
- 商と余り

だということがわかります。商と余りは別々の関数にすることもできますが、この先ユークリッドの互除法などで商と余りの両方が同時に必要になる場面が多いため、ここでは 1 つの関数にまとめ、商と余りの組を返すことにします。

実は、この部品がそろうと、最大公約数だけでなく、もう 1 つ便利なアルゴリズムが同時に手に入ります。**拡張ユークリッドの互除法**（*Extended Euclidean Algorithm*）と呼ばれるもので、

$$g = u \cdot a + v \cdot b \tag{4.5}$$

を満たす最大公約数 g と、係数 u, v を同時に求めるアルゴリズムです。今の段階では使い道がわかりにくいと思いますが、これは体の拡大の節で、ある元の逆元を求めるために使います。拡張ユークリッドの互除法まで導出するには、商と余りに加えて、減法・乗法・乗法単位元も必要になるので、ファンクタに渡すべき部品は次のようになります。

▲リスト 4.5　euclidean.ml

```
module type MIN = sig
  type t
  val zero: t
  val one: t
  val sub: t -> t -> t
  val mul: t -> t -> t
  val div_rem: t -> t -> t * t
  val equal: t -> t -> bool
end

module Extend(M: MIN) = struct
  (* 最大公約数 *)
  let rec gcd (a: M.t) (b: M.t): M.t =
    if M.equal b M.zero then a
    else gcd b (M.div_rem a b |> snd)

  (* 拡張ユークリッドの互除法 *)
  let rec ext_gcd (a: M.t) (b: M.t): M.t * M.t * M.t =
    if M.equal b M.zero then
      (a, M.one, M.zero)
    else
      let q, r = M.div_rem a b in
      let (g, u1, v1) = ext_gcd b r in
      (g, v1, M.sub u1 (M.mul q v1))
end
```

これで、最大公約数と、拡張ユークリッドの互除法を生成するファンクタができました。

4.1.3　累乗

累乗を定義するのには何が必要でしょうか。試しに、一度書いてみましょう。

```
let pow n x =
    let rec aux acc n =
      if n = 0 then acc
      else aux (acc * x) (n-1)
    in
    aux 1 n
```

累乗というのは、同じ数を n 回掛け合わせたものでした。これを漸化式で書くと、

$$x^n = \begin{cases} 1 & (n = 0) \\ x \cdot x^{n-1} & (n \geq 1) \end{cases} \tag{4.6}$$

となります。さきほどのコードは、この漸化式をそのまま OCaml の再帰関数にしたものです。

必要なのは、

- 乗法
- 乗法単位元

であることがわかります。この 2 つを引数として受け取るファンクタにすればよいわけです。

▲リスト 4.6　mul_monoid.ml

```
module type MIN = sig
  type t
  val mul: t -> t -> t
  val one: t
end

module Extend(M: MIN) = struct
  (* 累乗 *)
  let pow n x =
    let rec aux acc n =
      if n = 0 then acc
      else aux (M.mul acc x) (n-1)
    in
    aux M.one n
end
```

※ `n = 0` や `n-1` のように、まだ定義していない整数の減法やリテラルが普通に出てきておかしいと思うかもしれません。
`let pow n x` は、一見すると整数同士の二項演算に見えますが、実際には `n` を固定した単項演算で、`val pow: int -> t -> t` という型を持ちます。ここでの **int** と **t** はまったく別の型です（たまたま今回は **t** も整数なので紛らわしく見えます）。`n` 側の計算は、この本で作っている代数 **t** の演算ではなく、OCaml 組み込みの `int` をアルゴリズムの制御用に使っているだけなので、問題ありません。

乗法と単位元から、累乗を生成できました。ここでモジュールに Mul_monoid という名前を付けましたが、monoid とは、**モノイド**（*Monoid*）という数学用語です。モノイドとは、集合 M と二項演算 $\odot$ の組であって、次の性質を満たすもののことです。

$$
\begin{aligned}
&\text{閉性} && a, b \in M \implies a \odot b \in M \\
&\text{結合律} && (a \odot b) \odot c = a \odot (b \odot c) \\
&\text{単位元の存在} && \exists e \in M,\ a \odot e = e \odot a = a
\end{aligned}
\tag{4.7}
$$

先ほどの群の公理と見比べてみると、「逆元の存在」が抜けていることに気づくと思います。つまりモノイドとは、群から逆元の存在という条件だけを取り除いたものです。$(\mathbb{Z}, \times)$、つまり整数と乗法の組を考えてみましょう。閉性・結合律・単位元（1）はすべて満たしていますが、たとえば 2 に対する逆元、つまり $2 \times x = 1$ を満たすような整数 x は存在しません（1/2 は整数ではありません）。そのため $(\mathbb{Z}, \times)$ は群にはならず、モノイドにとどまります。乗法モノイドモジュール（`Mul_monoid`）は、この**乗法モノイド**（*Multiplicative Monoid*）を示します。

整数モジュール（`Integer`）全体としては次のようになります。

▲リスト 4.7　integer.ml

```
type t = int
(* 等価判定 *)
let equal (x1: t) (x2: t): bool =
   x1 = x2
let zero: t = 0
let one: t = 1
(* 加法 *)
let add (x1: t) (x2: t): t =
  x1 + x2
(* 符号反転 *)
let neg (x: t): t =
  -x
(* 乗法 *)
let mul (x1: t) (x2: t): t =
  x1 * x2
(* 商と余り *)
let div_rem (x1: t) (x2: t): t * t =
  (x1 / x2, x1 mod x2)

(* 大小比較 *)
let compare (x1: t) (x2: t): int =
  x1 - x2

include Mul_monoid.Extend(struct
  type nonrec t = t
  let one, mul = one, mul
end)
include Add_group.Extend(struct
  type nonrec t = t
  let add, neg = add, neg
end)
include Euclidean.Extend(struct
  type nonrec t = t
  let zero, one, sub, mul, div_rem, equal = zero, one, sub, mul, div_rem, equal
end)
```

各ファンクタには、必要な演算だけを集めた無名モジュール（struct ... end）をその場で渡しています。最後の拡張ユークリッドの互除法ファンクタ（Euclidean.Extend）だけは、減法（sub）も必要とします。これは加法群ファンクタ（Add_group.Extend）が自動生成したものなので、直前の include によってすでにスコープに入っている sub を、他の演算とまとめてあらためて渡しています。

4.1.4 テスト

作成した整数モジュール（Integer）の動作を確認してみましょう。計算結果の表示には、本書専用の文字列化ライブラリ（Expr、本書の GitHub リポジトリ https://github.com/icchon/ocaml-algebraic-calculator-book で公開）を使います。Integer のような計算モジュール自体には文字列化のコードを書きません。リポジトリの README にしたがってビルドすると to_tex.cmxa ができます。このライブラリには、代数モジュールの実装そのものは一切含まれておらず、TeX の文字列に変換するためのコードだけが入っています。そのため、コンパイルするときは、ここまでに書いたモジュールを先に、to_tex.cmxa を後に並べる必要があります。

▲リスト 4.8　integer_test.ml

```
let () =
  let add = Expr.infix_biop "+" Integer.add in
  Expr.put (Expr.biop_to_string (module Expr.Integer) add 2 3);
  Expr.put (Expr.biop_to_string (module Expr.Integer) add 2 (-3));
  let mul = Expr.cdot_biop Integer.mul in
  Expr.put (Expr.biop_to_string (module Expr.Integer) mul 2 3);
  Expr.put (Expr.biop_to_string (module Expr.Integer) mul 2 (-3));
  let gcd = Expr.call_biop "gcd" Integer.gcd in
  Expr.put (Expr.biop_to_string (module Expr.Integer) gcd 6 2);
  Expr.put (Expr.biop_to_string (module Expr.Integer) gcd 11 4);
  let pow4 = Expr.pow_unop 4 (Integer.pow 4) in
  Expr.put (Expr.unop_to_string (module Expr.Integer) pow4 4);
  Expr.put (Expr.unop_to_string (module Expr.Integer) pow4 0);
```

加減乗除や累乗のように標準的な記法を持つ演算は、$x + y$ や x^n のような見慣れた形で表示できます。一方、最大公約数のように標準的な中置記法を持たない演算は、名前と実体を組にして、関数呼び出しの形で表示します。これらの組を受け取って実際に計算し、表示形式まで組み立てる仕組みも用意してあります。テストコード側には、TeX の記法が一切登場していない点に注目してください。コンパイルして、実行用バイナリを作成しましょう。

```
ocamlopt mul_monoid.ml add_group.ml euclidean.ml integer.ml to_tex.cmxa integer_test.ml
```

出力バイナリに名前を指定しない場合、a.out が作成されるので、実行してみましょう。

```
./a.out
>>
\[2 + 3 = 5\]
\[2 + \left(-3\right) = -1\]
\[2 \cdot 3 = 6\]
\[2 \cdot \left(-3\right) = -6\]
\[\mathrm{gcd}\left(6, 2\right) = 2\]
\[\mathrm{gcd}\left(11, 4\right) = 1\]
\[4^{4} = 256\]
\[0^{4} = 0\]
```

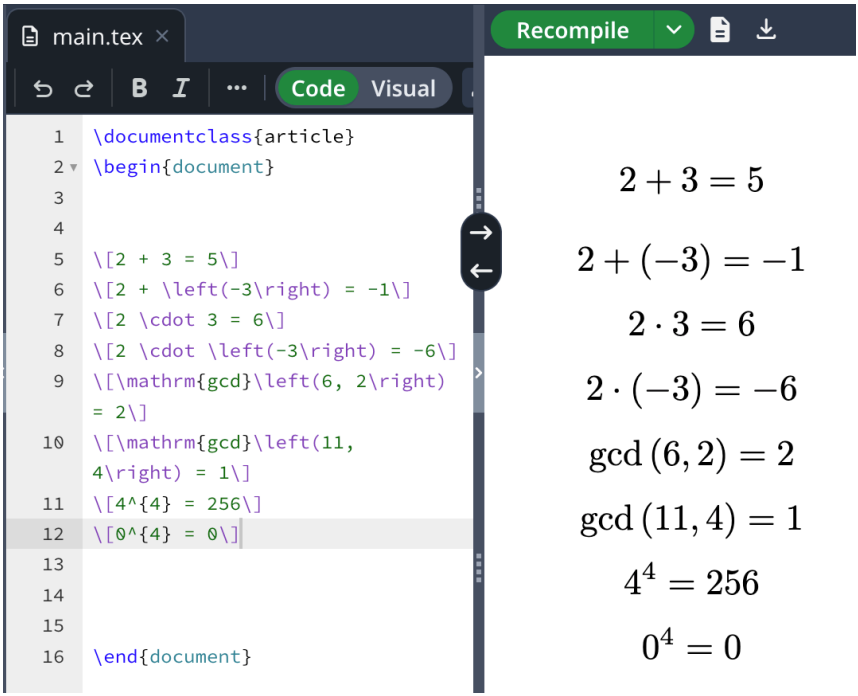
結果が LaTeX 形式で出力されました。これを TeX のビューアで確認してみましょう。

▲リスト 4.9　main.tex

```
\documentclass{article}
\begin{document}

\[2 + 3 = 5\]
\[2 + \left(-3\right) = -1\]
\[2 \cdot 3 = 6\]
\[2 \cdot \left(-3\right) = -6\]
\[\mathrm{gcd}\left(6, 2\right) = 2\]
\[\mathrm{gcd}\left(11, 4\right) = 1\]
\[4^{4} = 256\]
\[0^{4} = 0\]

\end{document}
```



▲図 4.2　ブラウザ上でチェック

4.2　有理数

4.2.1　基本的な機能

有理数モジュール（Rational）をつくっていきましょう。型 t は (Integer.t * Integer.t) のように、分子と分母をそれぞれ対応させた 2 つの整数組で表すことにします。

さて、整数モジュール（Integer）においては内部で int 型を用い、データと一対一に対応していたので、等価判定はシンプルでした。しかしながら、有理数には、分子と分母を同じ数 k 倍しても値が変わらないという自由度があります。

$$\frac{n}{d} = \frac{k \cdot n}{k \cdot d} \tag{4.8}$$

そのため、分子と分母のペアをそのまま比較しても、正しく等価判定できません。等価判定には、次の 2 つのアプローチが考えられます。

1. 同値変形して等価判定する
2. 正規化をしてデータを比較する

というものです。

1. 同値変形して等価判定する

有理数において次の同値変形をすることです。

$$\begin{aligned} \frac{n_1}{d_1} &\iff \frac{n_2}{d_2} \\ n_1 \cdot d_2 &\iff n_2 \cdot d_1 \end{aligned} \tag{4.9}$$

2. 正規化をしてデータを比較する

定数倍だけの自由度がある有理数に対し、一意に表される形に式変形をするということです。

$$\frac{k \cdot n'}{k \cdot d'} = \frac{n'}{d'} \quad (n' \text{と } d' \text{は互いに素}) \tag{4.10}$$

本書では、この **2. 正規化をしてデータを比較する**を採用します。等価判定を行うときに、内部のデータそのものを比較するだけでよくなり、シンプルだからです。また、出力を見たいとき、どのみち正規化をしたいというモチベーションがあるためです。

正規化を実装する際、何が必要か考えてみましょう。分子と分母を互いに素にするには、それぞれを最大公約数で割ればよいだけです。さらに、負の数のときは、必ず分子にマイナスをつけるようにしましょう。

```
let normalize (n, d) =
  let make_positive x =
    if Integer.compare x Integer.zero >= 0 then x else Integer.neg x
  in
  let g = Integer.gcd (make_positive n) (make_positive d) in
  let n' = Integer.div_rem n g |> fst in
  let d' = Integer.div_rem d g |> fst in
  if Integer.compare d' Integer.zero < 0 then
    (Integer.neg n', Integer.neg d')
  else
    (n', d')
```

これで正規化が行われるようになります。続けて、他の演算も考えてみましょう。加法・乗法は次のように表されます。

加法

$$\frac{n_1}{d_1} + \frac{n_2}{d_2} = \frac{n_1 d_2 + n_2 d_1}{d_1 d_2} \tag{4.11}$$

```
let add (n1, d1) (n2, d2) =
  let n = Integer.add (Integer.mul n1 d2) (Integer.mul n2 d1) in
  let d = Integer.mul d1 d2 in
  (n, d)
```

乗法

$$\frac{n_1}{d_1} \cdot \frac{n_2}{d_2} = \frac{n_1 \cdot n_2}{d_1 \cdot d_2} \tag{4.12}$$

```
let mul (n1, d1) (n2, d2) =
  let n = Integer.mul n1 n2 in
  let d = Integer.mul d1 d2 in
  (n, d)
```

さて、有理数は整数とは違い、乗法に関する逆元を考えることができます。

乗法の逆元

$$\left(\frac{n}{d}\right)^{-1} = \frac{d}{n} \tag{4.13}$$

```
let inv (n, d) =
  if Integer.equal n Integer.zero then
    failwith "Division by zero"
  else
    (d, n)
```

$n = 0$ のとき、そのまま (d, n) を組み立てると分母が 0 の値ができてしまいます。0 の逆元は数学的に定義されないので、ここでエラーとして扱うことにします。こうすることで、逆元を求める演算の型は `t -> t option` のように複雑にはならず、最後まで `t -> t` のまま保たれます。

先ほど作成した加法群ファンクタ（`Add_group.Extend`）と同様に、乗法の逆元をもとに除法を生成するファンクタ、乗法群ファンクタ（`Mul_group.Extend`）を作成してみましょう。

▲リスト 4.10　mul_group.ml

```
module type MIN = sig
  type t
  val mul: t -> t -> t
  val inv: t -> t
end

module Extend(M: MIN) = struct
  (* 除法 *)
  let div x1 x2 = M.mul x1 (M.inv x2)
end
```

加法群ファンクタ（Add_group.Extend）とほとんど同じ形です（リスト 4.2）。

ここで、乗法の逆元が考えられることにより累乗を拡張することができます。整数モジュール（Integer）においては、

$$\mathrm{pow}\,(n, x) := x^n \quad (n \geq 0) \tag{4.14}$$

このように、指数部分が自然数でしたが、有理数においては負の数まで拡張ができます。

$$\mathrm{pow}\,(n, x) := \begin{cases} x^n & (n \geq 0) \\ (x^{-1})^{|n|} & (n < 0) \end{cases} \tag{4.15}$$

マイナスの指数は、逆数にしてから累乗することで定義できます。負の数の累乗まで拡張できたのは、乗法の逆元があったからです。

乗法の逆元がある代数を受け取るファンクタを定義したモジュールが、乗法群ファンクタ（Mul_group.Extend）でした。この乗法群ファンクタ（Mul_group.Extend）に、累乗の関数を追加してみましょう。

▲リスト 4.11　mul_group.ml

```
module type MIN = sig
  type t
  val one: t
  val mul: t -> t -> t
  val inv: t -> t
end

module Extend(M: MIN) = struct
  (* 除法 *)
  let div x1 x2 = M.mul x1 (M.inv x2)
  (* 累乗 *)
  let pow n x =
    let base = if n >= 0 then x else M.inv x in
    let rec aux acc n =
      if n = 0 then acc
      else aux (M.mul acc base) (n - 1)
```

```
    in
    aux M.one (abs n)
end
```

n が負のときは、x そのものではなく、あらかじめ逆数にした $1/x$ を使って、その $|n|$ 乗を計算しています。これは先ほどの数式

$$(x^{-1})^{|n|} \tag{4.16}$$

をそのままコードにしたものです。これで、乗法の逆元があれば、負の数までの累乗が自動で生成されるようになりました。

乗法モノイドファンクタ（`Mul_monoid.Extend`）と乗法群ファンクタ（`Mul_group.Extend`）のどちらにも累乗を求める演算（型は `int -> t -> t`）が存在するため、両方 include したい際は、乗法モノイドファンクタ→乗法群ファンクタの順に include するようにしましょう。そのようにすれば、乗法群ファンクタ（`Mul_group.Extend`）の累乗により、乗法モノイドファンクタ（`Mul_monoid.Extend`）の累乗が上書きされます。

有理数全体を書いてみましょう。

▲リスト 4.12　rational.ml

```
module BASE = struct
type t = Integer.t * Integer.t
let one = (Integer.one, Integer.one)
let zero = (Integer.zero, Integer.one)
(* 約分と符号の正規化 *)
let normalize (n, d) =
  let make_positive x =
    if Integer.compare x Integer.zero >= 0 then x else Integer.neg x
  in
  let g = Integer.gcd (make_positive n) (make_positive d) in
  let n' = Integer.div_rem n g |> fst in
  let d' = Integer.div_rem d g |> fst in
  if Integer.compare d' Integer.zero < 0 then
    (Integer.neg n', Integer.neg d')
  else
    (n', d')
(* 符号反転 *)
let neg (n, d) = (Integer.neg n, d)
(* 加法 *)
let add (n1, d1) (n2, d2) =
  let n = Integer.add (Integer.mul n1 d2) (Integer.mul n2 d1) in
  let d = Integer.mul d1 d2 in
  (n, d)
(* 乗法 *)
let mul (n1, d1) (n2, d2) =
  let n = Integer.mul n1 n2 in
  let d = Integer.mul d1 d2 in
  (n, d)
(* 逆数 *)
let inv (n, d) =
  if Integer.equal n Integer.zero then
```

```
    failwith "Division by zero"
  else
    (d, n)
(* 大小比較 *)
let compare (n1, d1) (n2, d2) =
  Integer.compare (Integer.mul n1 d2) (Integer.mul n2 d1)
(* 等価判定 *)
let equal x1 x2 = (x1 |> normalize) = (x2 |> normalize)
end
include Mul_monoid.Extend(BASE)
include Add_group.Extend(BASE)
include Mul_group.Extend(BASE)
```

整数モジュール（Integer）を使って、有理数が完成しました。

このままでも十分なのですが、有理数をつくる際に受け取るモジュールは、Integer に限定する必要がないことに気づくと思います。

- 加法
- 加法の逆元
- 乗法

があればなんでもいいわけです。このような性質を満たすものを**環**（*Ring*）と呼びます。また、これに加え、乗法の逆元が加わったものを**体**（*Field*）と呼びます。今回作成した有理数モジュール（Rational）も**体**（*Field*）です。

改めて、環と体の定義を整理しておきましょう。**環**（*Ring*）とは、集合 R と 2 つの二項演算（加法 $+$、乗法 $\times$）の組 $(R, +, \times)$ であって、次の性質を満たすもののことです。

$$\begin{aligned} &\text{加法について可換群} && (R,+) \text{ が群であり、} a+b=b+a \\ &\text{乗法についてモノイド} && (R,\times) \text{ がモノイドである} \\ &\text{分配律} && a\times(b+c)=a\times b+a\times c \end{aligned} \tag{4.17}$$

整数 $\mathbb{Z}$ はまさにこの環の例でした。加法については逆元（減法）が存在しますが、乗法については逆元（除法）が存在しません。

体（*Field*）は、環にさらに「0 以外のすべての元に乗法の逆元が存在する」という条件を加えたものです。

$$a \in R,\ a \neq 0 \implies \exists a^{-1} \in R,\ a \times a^{-1} = a^{-1} \times a = 1 \tag{4.18}$$

有理数 $\mathbb{Q}$ は、整数を分数にすることで、この乗法の逆元を人工的に手に入れた体だと言えます。実際、n/d の逆元は d/n です（$n \neq 0$ であれば）。このように、環か

ら分数をつくることで体を得る構成のことを、数学では**分数体**（*Field of Fractions*）と呼びます。

そこで、整数モジュール（Integer）を受け取り、有理数をつくるのではなく、**環**（*Ring*）を受け取り分数体を返すファンクタをつくることにしましょう。こうしておけば、この先新しい環をつくるたびに、ファンクタを適用するだけで分数体が手に入ります。数学用語がいくつか登場しましたが、やることは先ほどのコードをファンクタに変えるだけなので難しくありません。

環が満たすべき核となる性質（加法群・乗法モノイド）は、この先も繰り返し必要になります。そこで、小さな Ring というモジュール型にまとめておきましょう。

ここで、これまで Extend ファンクタの引数として使ってきた MIN と、これから使う S という 2 つのシグネチャの役割の違いに触れておきます。MIN は、減法のような新しい演算を導出するために最低限必要な性能だけを要求するシグネチャでした。一方 S は、「加法群として一通りの演算がすでに揃っている」という完成形を表すシグネチャです。Ring が必要としているのはこの完成形の方なので、Add_group・Mul_monoid にそれぞれ S を追加しておきます。

▲リスト 4.13　add_group.ml

```
module type S = sig
  include MIN
  include Add_monoid.S with type t := t
  val sub: t -> t -> t
end
```

Add_monoid.S は、加法の単位元 zero を持つことだけを要求する小さなシグネチャで、Add_monoid という別モジュールに切り出してあります。このファイルにはファンクタが存在しないので、MIN は用意せず S だけを定義します。

▲リスト 4.14　add_monoid.ml

```
module type S = sig
  type t
  val zero: t
  val add: t -> t -> t
end
```

Mul_monoid の方は、MIN の時点ですでに単位元を要求しているので、追加のファイル分割は必要なく、累乗が使えることを加えるだけです。

▲リスト 4.15　mul_monoid.ml

```
module type S = sig
  include MIN
  val pow: int -> t -> t
end
```

これで、Ring が要求する部品がそろいました。

▲リスト 4.16　ring.ml

```
module type S = sig
  type t
  include Add_group.S with type t := t
  include Mul_monoid.S with type t := t
  val equal : t -> t -> bool
end
```

乗法について、逆元まで要求する乗法群（Mul_group）ではなく、逆元を要求しない乗法モノイド（Mul_monoid）を使っている点に注目してください。逆元まで要求してしまうと、あとで体（Field）に使い回せなくなってしまいます。

実はこのあと作る分数体ファンクタ（Fraction.Make）は、まさにその「体」を返すファンクタです。そこで環（Ring）のときと同じ要領で、体が満たすべき性質もあわせて Field というモジュール型にまとめておきましょう。環（Ring）との違いは、乗法についても逆元まで要求する乗法群（Mul_group）を使っている点です。そのためにはまず、Mul_group にも加法群（Add_group）・乗法モノイド（Mul_monoid）と同じ要領で完成形のシグネチャ S を追加しておく必要があります。

▲リスト 4.17　"mul_group.ml"

```
module type S = sig
  include MIN
  include Mul_monoid.S with type t := t
  val div: t -> t -> t
  val pow: int -> t -> t
end
```

乗法モノイド（Mul_monoid）を再利用しているのは、累乗をすでに要求するシグネチャとして持っているからです。これで Field の部品がそろいました。

▲リスト 4.18　"field.ml"

```
module type S = sig
  type t
  include Add_group.S with type t := t
  include Mul_group.S with type t := t
  val equal : t -> t -> bool
end
```

分数体ファンクタ（Fraction.Make）が実際に必要とするのは、この環（Ring）に加えて、分子・分母を約分するための大小比較・最大公約数・商と余りです。

▲リスト 4.19　fraction.ml

```
module type Ring = sig
  include Ring.S
  val compare: t -> t -> int
  val gcd: t -> t -> t
  val div_rem: t -> t -> t * t
end
```

```
module type FUNCTOR_TYPE = functor (R: Ring) ->
  (sig
     include Field.S
     val compare: t -> t -> int
   end with type t = R.t * R.t)
```

分数体ファンクタ（Fraction.Make）は、受け取った環 R をもとに、分子・分母の組 R.t * R.t という新しい型を丸ごと作って返します。返ってくるものは、逆元まで含めて体（Field.S）の性質を満たすので、出力シグネチャの中心は Field.S です。あわせて大小比較も出力に含めています。この先、拡大体の並列拡大などで「便宜的な大小関係」として何度も使うことになるためです。with type t = R.t * R.t を付けているのは、t を抽象型のままにせず、R.t * R.t という具体的な形を外から見えるようにしておくためです。

▲リスト 4.20　"fraction.ml"

```
module Make: FUNCTOR_TYPE = functor (R: Ring) -> struct
  type t = R.t * R.t
  let one = (R.one, R.one)
  let zero = (R.zero, R.one)
  (* 約分と符号の正規化 *)
  let normalize (n, d) =
    let make_positive x =
      if R.compare x R.zero >= 0 then x else R.neg x
    in
    let g = R.gcd (make_positive n) (make_positive d) in
    let n' = R.div_rem n g |> fst in
    let d' = R.div_rem d g |> fst in
    if R.compare d' R.zero < 0 then
      (R.neg n', R.neg d')
    else
      (n', d')
  (* 符号反転 *)
  let neg (n, d) =
    (R.neg n, d) |> normalize
  (* 加法 *)
  let add (n1, d1) (n2, d2) =
    let d = R.mul d1 d2 in
    let n = R.add (R.mul n1 d2) (R.mul n2 d1) in
    (n, d) |> normalize
  (* 乗法 *)
  let mul (n1, d1) (n2, d2) =
    let n = R.mul n1 n2 in
    let d = R.mul d1 d2 in
    (n, d) |> normalize
  (* 逆数 *)
  let inv (n, d) =
    if R.equal n R.zero then
      failwith "Division by zero"
    else
      (d, n) |> normalize
  (* 大小比較 *)
  let compare (n1, d1) (n2, d2) =
    R.compare (R.mul n1 d2) (R.mul n2 d1)
  (* 等価判定 *)
  let equal x1 x2 = compare x1 x2 = 0

  include Mul_monoid.Extend(struct
    type nonrec t = t
```

```
    let one, mul, equal = one, mul, equal
  end)
  include Add_group.Extend(struct
    type nonrec t = t
    let add, neg = add, neg
  end)
  include Mul_group.Extend(struct
    type nonrec t = t
    let one, mul, inv = one, mul, inv
  end)
end
```

ここで、有理数の節で採用した「2. 正規化をしてデータを比較する」ではなく、あえて「1. 同値変形して等価判定する」（分子・分母を交差させて掛ける方法）に戻っている点に注意してください。整数のような単数（掛けても大きさが変わらない特別な元）が ± 1 の 2 つしかない環では、「分母の符号を必ず正にする」という約束だけで、約分後の表現を一意に定められました。しかしこの先、分数体ファンクタ（`Fraction.Make`）は多項式環のような環にも適用します。体を係数とする多項式環では、0 でない定数はすべて単数になるため、同じ符号の約束だけでは表現を一意に定めることができません。そこで、どんな環 `R` を受け取っても常に正しく動く、同値変形による判定に戻しています。

逆元を求める演算に 0 除算のチェックを入れる理由は先ほど述べたとおりですが、こうしておけば、この分数体ファンクタ（`Fraction.Make`）を土台にするすべての体（有理数はもちろん、この先登場する有理関数体なども）で、0 除算が一貫してエラーになります。

もし逆元を求める演算が失敗時に `t -> t option` を返す設計にしていたら、そこから自動生成される除法や、さらにその上に乗る演算にまで、失敗の可能性が次々と伝搬してしまいます。本書のファンクタが演算を組み合わせるだけでシンプルに新しい演算を生成できているのは、`t -> t` という型を最後まで崩さず、0 除算のような異常な入力は素直に例外として扱っているからです。

分数体ファンクタができました。これを用いて有理数を作成しましょう。

▲リスト 4.21　rational.ml

```
include Fraction.Make(Integer)
```

たった一行のコードで、有理数モジュール（`Rational`）が出来上がりました。有理数が整数を**環**（*Ring*）とする分数体だという定義そのものです。

これだけだと整数から有理数への変換手段がないので、1 つだけ関数を足しておきます。

▲リスト 4.22　"rational.ml"

```
(* 整数の埋め込み *)
let of_integer (x: Integer.t): t =
  (x, Integer.one)
```

整数 x を、分母が 1 の分数 $x/1$ として埋め込むだけです。この先、整数リテラルから有理数を作りたい場面で繰り返し使います。

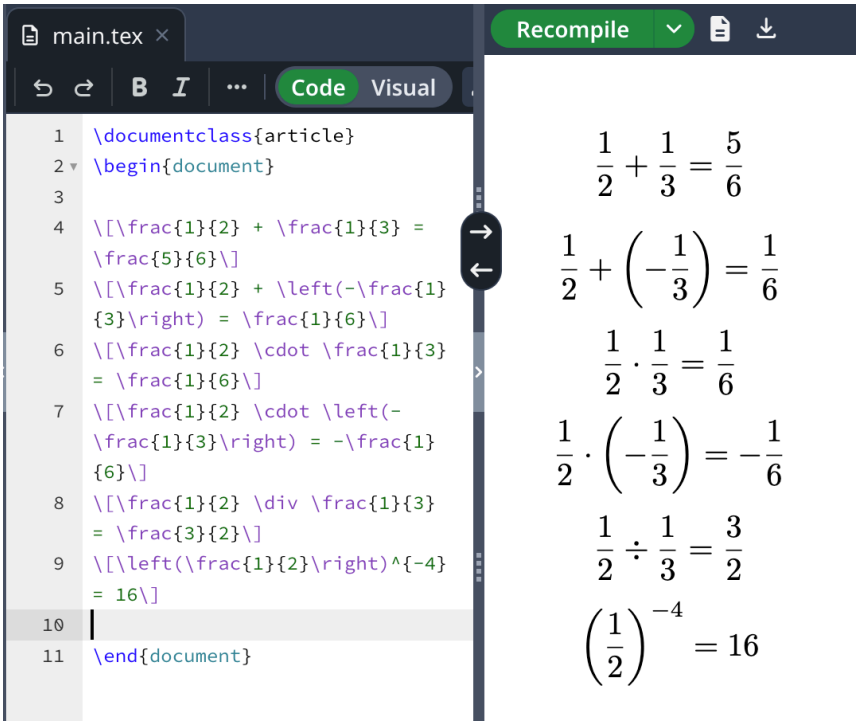
4.2.2 テスト

作成した有理数モジュール（Rational）の動作を確認してみましょう。

▲リスト 4.23　rational_test.ml

```
let () =
  let add = Expr.infix_biop "+" Rational.add in
  Expr.put (Expr.biop_to_string (module Expr.Rational) add (1, 2) (1, 3));
  Expr.put (Expr.biop_to_string (module Expr.Rational) add (1, 2) (-1, 3));
  let mul = Expr.cdot_biop Rational.mul in
  Expr.put (Expr.biop_to_string (module Expr.Rational) mul (1, 2) (1, 3));
  Expr.put (Expr.biop_to_string (module Expr.Rational) mul (1, 2) (-1, 3));
  let div = Expr.infix_biop "\\div" Rational.div in
  Expr.put (Expr.biop_to_string (module Expr.Rational) div (1, 2) (1, 3));
  let pow4 = Expr.pow_unop (-4) (Rational.pow (-4)) in
  Expr.put (Expr.unop_to_string (module Expr.Rational) pow4 (1, 2))
```

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml to_tex.cmxa rational_test.ml && ./a.out
>>
\[\frac{1}{2} + \frac{1}{3} = \frac{5}{6}\]
\[\frac{1}{2} + \left(-\frac{1}{3}\right) = \frac{1}{6}\]
\[\frac{1}{2} \cdot \frac{1}{3} = \frac{1}{6}\]
\[\frac{1}{2} \cdot \left(-\frac{1}{3}\right) = -\frac{1}{6}\]
\[\frac{1}{2} \div \frac{1}{3} = \frac{3}{2}\]
\[\left(\frac{1}{2}\right)^{-4} = 16\]
```



▲図 4.3　ブラウザ上でチェック

コラム: なぜ有理数の次は実数ではなく複素数なのか

有理数を実装したあとの自然な流れとしては、実数へ進むことを考えるかもしれません。しかし本書の代数計算機では、実数を飛び越えて複素数の実装へ進みます。単に浮動小数点数の丸め誤差を嫌ったからではなく、厳密な実数を代数計算機の型として実装すること自体が、原理的にできないからです。理由は大きく 2 つあります。

1 つ目は、非可算性の問題です。プログラムのソースコードは有限の記号列なので、書けるプログラムの総数は高々可算無限（自然数と同じ個数）しかありません。一方、カントールの対角線論法が示すとおり、実数全体は非

可算無限（可算無限よりも真に大きい濃度）です。可算個の入れ物に非可算個の実数を詰め込むことはできないため、どのようなデータ構造を設計しても、大半の実数はコードとして表現することすらできません。

では、π や e のように、テイラー展開や級数によって「第 n 項の有理数を返す関数（有理数のコーシー列）」としてコードに落とし込める数だけを集めれば、型にできるでしょうか。これも破綻します。計算可能解析学の枠組み[*2]において、実数を有理数のコーシー列として表現したとき、等値判定 $x = y$（差をとったゼロ判定 $x - y = 0$）を有限時間で必ず停止して判定できるアルゴリズムは存在しないことが知られています。等値判定が停止しない型では、代数計算の基本である値の正規化や同値関係が維持できず、モジュールとして成立しません。

一方、複素数は有理数体 $\mathbb{Q}$ に、多項式の関係式 $i^2 = -1$ を満たす元 i を添加する代数拡大によって構成されます。複素数 $a + bi$ は、有理数のペア (a, b) だけで完全に表現でき、等値判定も 2 つの有理数の等値判定に帰着するので有限ステップで確実に停止します。本書が実数を飛び越えて複素数へ進むのは、四則演算と等値判定が有限ステップで確実に停止する型を保つための、必然の選択なのです。

4.3 複素数

4.3.1 基本的な機能

有理数を作ることができたので、次は複素数を作っていきましょう。複素数は、$i^2 = -1$ を満たす新しい数 i（虚数単位）を有理数に付け加え、$a + bi$（a, b は有理数）という形の数の全体として定義されます。

複素数になったことにより、新しく共役という単項演算が登場します。

共役

$$\overline{a + bi} = a - bi \tag{4.19}$$

他の基本的な演算に関しては、次のとおりになります。

[*2] V. Brattka, G. Gherardi, A. Pauly, "Weihrauch Complexity in Computable Analysis," 2018 (arXiv:1707.03202) などを参照。

加法

$$(a_1 + b_1 i) + (a_2 + b_2 i) = (a_1 + a_2) + (b_1 + b_2)i \tag{4.20}$$

乗法

$$(a_1 + b_1 i) \cdot (a_2 + b_2 i) = (a_1 a_2 - b_1 b_2) + (a_1 b_2 + b_1 a_2)i \tag{4.21}$$

共役同士の積

$$\begin{aligned} (a + bi) \cdot (a - bi) &= a^2 + b^2 \\ z \cdot \overline{z} &= |z|^2 \end{aligned} \tag{4.22}$$

乗法の逆元

$$z^{-1} = \frac{\overline{z}}{|z|^2} \tag{4.23}$$

なぜこれが逆元なのかは、直前の「共役同士の積」の式からわかります。$z \cdot \overline{z} = |z|^2$ なので、両辺を $|z|^2$（0 でない有理数）で割ると

$$z \cdot \frac{\overline{z}}{|z|^2} = 1 \tag{4.24}$$

となり、たしかに共役を $|z|^2$ で割った値が z の乗法の逆元になっていることがわかります。実装するときも、この式をそのままなぞればよく、z に共役をかけて $|z|^2$ を求め、それぞれの成分を割るだけです。

これらを踏まえて、複素数をつくっていきましょう。実部と虚部からなる 2 つの有理数のペア、コード上では `type t = Rational.t * Rational.t` として表現します。

▲リスト 4.24　complex.ml

```
module BASE = struct
  type t = Rational.t * Rational.t
  (* 等価判定 *)
  let equal (a1, b1) (a2, b2) =
    (Rational.equal a1 a2) && (Rational.equal b1 b2)
  let zero = (Rational.zero, Rational.zero)
  let one = (Rational.one, Rational.zero)
```

```
  (* 加法 *)
  let add (a1, b1) (a2, b2) = (Rational.add a1 a2, Rational.add b1 b2)
  (* 符号反転 *)
  let neg (a, b) = (Rational.neg a, Rational.neg b)
  (* 共役 *)
  let conj (a, b) = (a, Rational.neg b)
  (* 乗法 *)
  let mul (a1, b1) (a2, b2) =
    (Rational.sub (Rational.mul a1 a2) (Rational.mul b1 b2),
    Rational.add (Rational.mul a1 b2) (Rational.mul b1 a2))
  (* ノルムの2乗 *)
  let norm2 x = mul x (conj x) |> fst
  (* 逆元 *)
  let inv x =
    let norm = norm2 x in
    let (a, b) = conj x in
    (Rational.div a norm, Rational.div b norm)
end

include BASE
include Add_group.Extend(BASE)
include Mul_monoid.Extend(BASE)
include Mul_group.Extend(BASE)
```

複素数モジュール（Complex）が完成しました。内部で有理数モジュール（Rational）を使い、演算の大部分をファンクタで自動生成しているため、新しく書いたコードはごくわずかです。

さて、複素数モジュール（Complex）は、内部で有理数モジュール（Rational）のペアを使っているだけでした。これをそのままファンクタの引数として受け取り、**複素的**な構造を返すファンクタにできないでしょうか。分数体ファンクタをつくったときと同じ発想です。

つまり、体 F を受け取り、F の元のペア (a_1, a_2)（$a_1, a_2 \in F$）からなる新しい構造を返すファンクタです。F が有理数であれば、これはまさに複素数 $a_1 + a_2 i$ になります。

複素数をつくるためだけに使うのであれば、ここまで抽象化する意味はありません。

しかし、複素構造ファンクタ（Complexical.Make）の本当のうれしさは、複素数以外の場面にあります。F がどんな体であっても、Complexical.Make(F) によって「F に虚数単位を添加した体」を機械的に作れる、という点です。有理数体に限らず、この先本書に登場する別の体に対しても、同じファンクタがそのまま使えます。

さらに、複素構造ファンクタ（Complexical.Make）は何度でも重ねて適用できます。Complexical.Make(Complexical.Make(F)) のように 2 回適用すれば、ペアがさらにペアになった、より高次の構造が手に入ります。**四元数**（*Quaternion*）・八元数・十六元数といった**超複素数**（*Hypercomplex Number*）は、まさにこの「ファンクタを繰り返し適用するタワー構造」の恩恵を受けている例です。まずは、有理数体を渡して複素数をつくるところから実装してみましょう。

体が満たすべき核となる性質（加法群・乗法群）は、有理数の節で分数体ファンクタ（Fraction.Make）の出力シグネチャとしてすでに Field を用意してありました。

そのままこのファンクタの入力に使います。実装に入る前に、つくろうとしているファンクタの入出力の型を確認しておきましょう。

▲リスト 4.25　"complexical.ml"

```
module type Field = Field.S

module type FUNCTOR_TYPE =
  functor (F: Field) -> (Field with type t = F.t * F.t)
```

入力も出力も、（ここではまだ `Field.S` そのものである）`Field` シグネチャで、F の元のペア (a_1, a_2) を型 `t` としています。この型に沿って実装していきます。

▲リスト 4.26　complexical.ml

```
module Make(F: Field) = struct
  type t = F.t * F.t
  (* 等価判定 *)
  let equal (a1, b1) (a2, b2) =
    (F.equal a1 a2) && (F.equal b1 b2)
  let zero = (F.zero, F.zero)
  let one = (F.one, F.zero)
  (* 加法 *)
  let add (a1, b1) (a2, b2) = (F.add a1 a2, F.add b1 b2)
  (* 符号反転 *)
  let neg (a, b) = (F.neg a, F.neg b)
  (* 共役 *)
  let conj (a, b) = (a, F.neg b)
  (* 乗法(暫定) *)
  let mul (a1, b1) (a2, b2) =
    (F.sub (F.mul a1 a2) (F.mul b1 b2),
    F.add (F.mul a1 b2) (F.mul b1 a2))
  (* ノルムの2乗 *)
  let norm2 x = mul x (conj x) |> fst
  (* 逆元 *)
  let inv x =
    let norm = norm2 x in
    let (a, b) = conj x in
    (F.div a norm, F.div b norm)

  include Mul_monoid.Extend(struct
    type nonrec t = t
    let one, mul, equal = one, mul, equal
  end)
  include Add_group.Extend(struct
    type nonrec t = t
    let add, neg = add, neg
  end)
  include Mul_group.Extend(struct
    type nonrec t = t
    let one, mul, inv = one, mul, inv
  end)
end
```

▲リスト 4.27　complex.ml

```
include Complexical.Make(Rational)
```

複素数モジュール（Complex）の中にあった実装を複素構造ファンクタ（Complexical.Make）に移動させ、複素数モジュール（Complex）では、複素構造ファンクタ（Complexical.Make）に有理数を渡し、複素数を生成するようにしました。

4.3.2　テスト

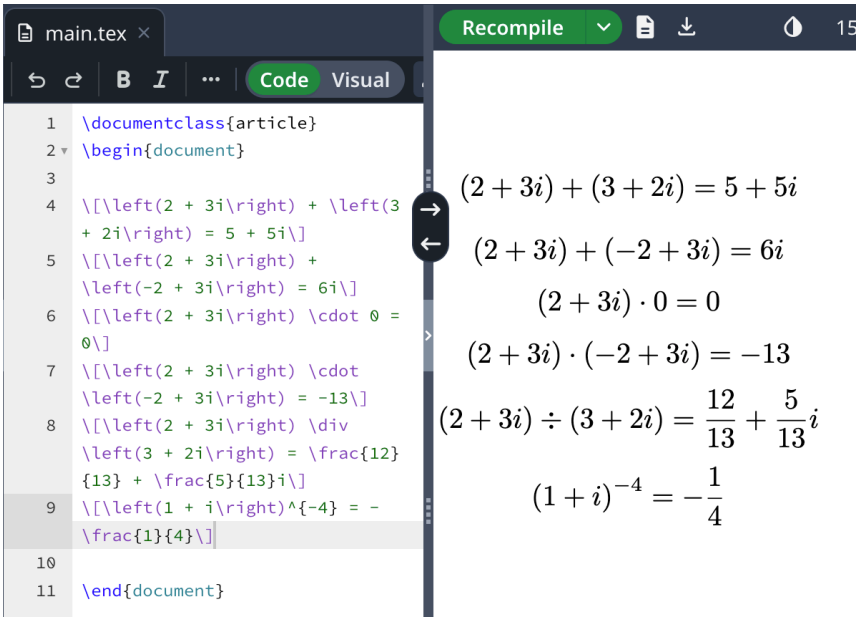
作成した複素数モジュール（Complex）の動作を確認してみましょう。

▲リスト 4.28　complex_test.ml

```
let () =
  let x_2 = (2, 1) in
  let x_3 = (3, 1) in
  let x_neg2 = (-2, 1) in

  let add = Expr.infix_biop "+" Complex.add in
  Expr.put (Expr.biop_to_string (module Expr.Complex) add
    (x_2, x_3) (x_3, x_2));
  Expr.put (Expr.biop_to_string (module Expr.Complex) add
    (x_2, x_3) (x_neg2, x_3));
  let mul = Expr.cdot_biop Complex.mul in
  Expr.put (Expr.biop_to_string (module Expr.Complex) mul
    (x_2, x_3) Complex.zero);
  Expr.put (Expr.biop_to_string (module Expr.Complex) mul
    (x_2, x_3) (x_neg2, x_3));
  let div = Expr.infix_biop "\\div" Complex.div in
  Expr.put (Expr.biop_to_string (module Expr.Complex) div
    (x_2, x_3) (x_3, x_2));
  let pow4 = Expr.pow_unop (-4) (Complex.pow (-4)) in
  Expr.put (Expr.unop_to_string (module Expr.Complex) pow4
    (Rational.one, Rational.one))
```

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml complexical.ml complex.ml to_tex.cmxa complex_test.ml && ./a
.out
>>
\[\left(2 + 3i\right) + \left(3 + 2i\right) = 5 + 5i\]
\[\left(2 + 3i\right) + \left(-2 + 3i\right) = 6i\]
\[\left(2 + 3i\right) \cdot 0 = 0\]
\[\left(2 + 3i\right) \cdot \left(-2 + 3i\right) = -13\]
\[\left(2 + 3i\right) \div \left(3 + 2i\right) = \frac{12}{13} + \frac{5}{13}i\]
\[\left(1 + i\right)^{-4} = -\frac{1}{4}\]
```



▲図 4.4　ブラウザ上でチェック

先ほど説明したとおり、複素構造ファンクタ（`Complexical.Make`）は何度でも重ねて適用できるのでした。そのタワー構造の最初の 1 段である**四元数**（*Quaternion*）から見ていきましょう。

四元数・八元数・十六元数は、本書が主題とする「方程式を解く」「微分・積分する」といった話題からは外れる寄り道です。ここで確認したいのは、`Complexical.Make` という 1 つのファンクタが複素数以外にもそのまま適用でき、同じ入出力の型を持つかぎり何度でも重ねられるという、本書のファンクタ設計の汎用性だけなので、最短で応用まで到達したい場合はここから先の四元数・八元数・十六元数の節を読み飛ばしてもかまいません。

四元数

四元数は、複素数を包含する集合として定義されます。複素数の虚数部が 1 つだったのに対し、四元数には虚数部が 3 つあります。実は空想上の代数ではなく、3 次元空間での回転を表現するのに適した性質を持つため、コンピュータグラフィックスや人工衛星・ロボットの姿勢制御など、現実のさまざまな場面で使われています。

$$
\begin{aligned}
& a + bi + cj + dk \\
& i^2 = j^2 = k^2 = -1 \\
& ij = k \\
& jk = i \\
& ki = j
\end{aligned}
\tag{4.25}
$$

基本的な演算は、次のようなルールになります。

和

$$
\begin{aligned}
(a_1 + b_1 i + c_1 j + d_1 k) + (a_2 + b_2 i + c_2 j + d_2 k) &= (a_1 + a_2) \\
&+ (b_1 + b_2) i \\
&+ (c_1 + c_2) j \\
&+ (d_1 + d_2) k
\end{aligned}
\tag{4.26}
$$

積

$$
\begin{aligned}
(a_1 + b_1 i + c_1 j + d_1 k) \cdot (a_2 + b_2 i + c_2 j + d_2 k) &= (a_1 a_2 - b_1 b_2 - c_1 c_2 - d_1 d_2) \\
&+ (a_1 b_2 + b_1 a_2 + c_1 d_2 - d_1 c_2) i \\
&+ (a_1 c_2 - b_1 d_2 + c_1 a_2 + d_1 b_2) j \\
&+ (a_1 d_2 + b_1 c_2 - c_1 b_2 + d_1 a_2) k
\end{aligned}
\tag{4.27}
$$

四元数の積の結果はとても複雑になりました。四元数の積を普通に実装しようとすると、これらの計算ルールを愚直に記述する必要があります。

しかし、先ほど作成した複素構造ファンクタ（`Complexical.Make`）の引数に複素数モジュール（`Complex`）を渡せば、この複雑な計算ルールを自分で書き下さなくても、四元数を生成できそうです。ただし、そのままでは正しく動かず、これから説明する修正が必要になります。

▲リスト 4.29　quaternion.ml

```
include Complexical.Make(Complex)
```

このように、複素的な構造に着目した代数学の分野として、**ケーリーディクソン代数**（*Cayley-Dickson Algebras*）というものがあります。複素数や四元数はそれ単体で定義されるイメージが強いですが、**ケーリーディクソン代数**（*Cayley-Dickson Algebra*）においては、低次の複素的代数から高次の複素的代数を生成するルールによって定められます。

$$\begin{aligned}\overline{(p,q)} &= (\overline{p}, -q) \\ (p,q)\cdot(r,s) &= (pr - \overline{s}q, sp + q\overline{r})\end{aligned} \tag{4.28}$$

それぞれ、共役と積についてのルールが定められています。実数を入れれば複素数が、複素数を入れれば四元数が、四元数を入れれば八元数が作られる、といった具合に**ケーリーディクソン代数**（*Cayley-Dickson Algebras*）をいくらでも生成できます。一般化すると、

$$2^n\text{元数} \longrightarrow 2^{n+1}\text{元数} \tag{4.29}$$

を生成する、ということです。

先ほどの複素構造ファンクタ（`Complexical.Make`）の乗法を、上のケーリーディクソンの式と見比べてみましょう。

$$(a_1,b_1)\cdot(a_2,b_2) = (a_1a_2 - b_1b_2,\ a_1b_2 + b_1a_2) \tag{4.30}$$

これは、ケーリーディクソンの式で共役を全部外してしまったものになっています。有理数は共役をとっても自分自身と等しい（$\overline{x} = x$）ので、F が有理数である複素数まではこれで正しく動きます。

しかし、これは F が有理数だから、たまたま共役を省略しても計算が合っていただけです。四元数をつくるために F に複素数モジュール（`Complex`）を渡すと、共役が恒等写像ではなくなるため、この省略が破綻します。乗法を、ケーリーディクソンの式のとおり、共役を省略せずに修正しましょう。

▲リスト 4.30　complexical.ml

```
(* 乗法 *)
let mul (a1, b1) (a2, b2) =
    (F.sub (F.mul a1 a2) (F.mul (F.conj b2) b1),
    F.add (F.mul b2 a1) (F.mul b1 (F.conj a2)))
```

共役になっていなかった部分を共役にしました。ただし、これだけでは足りません。共役を求める演算を呼べるようにするには、ファンクタの引数となる Field シグネチャ自体に共役を追加する必要があります。

▲リスト 4.31　complexical.ml

```
module type Field = sig
  include Field.S
  val conj: t -> t
end
```

そして、これまで Field シグネチャを満たすものとして渡してきた有理数モジュール（Rational）にも、共役を実装しなければなりません。幸い、有理数は実数なので、共役をとっても自分自身のままです。

▲リスト 4.32　rational.ml

```
(* 共役 *)
let conj (x: t): t = x
```

これで、Complexical.Make(Rational) で複素数を、Complexical.Make(Complex) で四元数を、Complexical.Make(Complexical.Make(Complex)) で八元数を、と再帰的に高次の複素的代数を生成できる、正しいケーリーディクソン構成のファンクタが完成しました。試しに $i \cdot j$ を計算してみると、修正前は正しく k になりませんでしたが、修正後はきちんと $i \cdot j = k$、$j \cdot i = -k$ となることが確認できます。

最後に、この Complexical.Make が実際に満たしている型を確認しておきましょう。明示的な型注釈をつけていなくても、Complexical.Make はちゃんと次の型を満たしています。

▲リスト 4.33　"complexical.ml"

```
module type FUNCTOR_TYPE =
  functor (F: Field) -> (Field with type t = F.t * F.t)
```

入力も出力も同じ Field シグネチャになっている点に注目してください。だからこそ、Complexical.Make(Complexical.Make(F)) のように、このファンクタは何度でも重ねて適用できるのです。

ケーリーディクソン代数

ここで、ケーリーディクソン代数の重要な補足を行います。ケーリーディクソンの構成法は、**環**（*Ring*）を受け取り、**環**（*Ring*）を返すというものになっています。受

け取る側と返す側がどちらも同じ**環**（*Ring*）という構造なので、その結果をふたたび入力として渡すことができ、原理上いくらでも繰り返し適用できます。

しかしながら、今回は**環**（*Ring*）に乗法の逆元が追加された**体**（*Field*）を受け取っています。つまり、本来は積の逆元は考えられないということです。

- 一元数（実数）
- 二元数（複素数）
- 四元数
- 八元数

までは乗法の逆元が存在しますが、十六元数以降は乗法の逆元が定義できません。十六元数になると**非自明な零因子**、つまり 0 でない数同士をかけても 0 になってしまうケースが現れるからです。これにより乗法の逆元が一意に定まらなくなり、除法が可能なのは八元数までとなります。

これらの性質をまとめた表が次のとおりです（表中の性質はすべて乗法についてのものです）。表を見る前に、各列が何を指しているかを確認しておきましょう。

- 次元： 実数がいくつ分の情報を持つか（実数は 1 個分、複素数は 2 個分というように、ケーリーディクソン構成で 1 回添加するたびに 2 倍になります）
- 順序： 値同士の大小比較ができること
- 交換法則：$xy = yx$
- 結合法則：$(xy)z = x(yz)$
- 交代代数：$(xx)y = x(xy)$ と $(yx)x = y(xx)$
- 冪結合性：$(xx)x = x(xx)$
- 非自明な零因子：$x \neq 0$ かつ $y \neq 0$ なのに $xy = 0$ となることがある

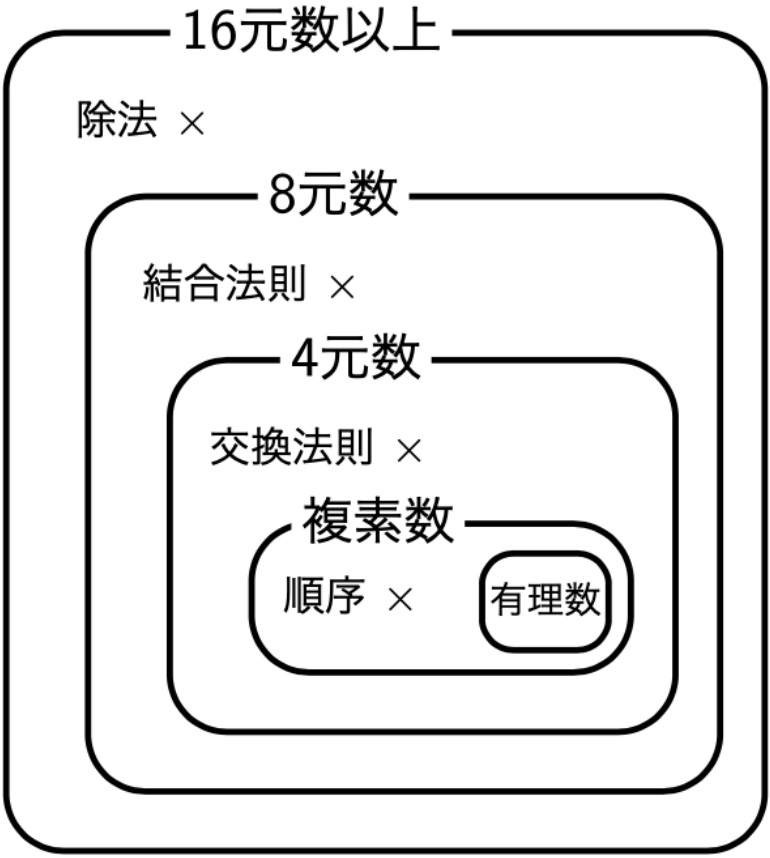
▼表 4.1　ケーリーディクソン代数の性質

代数	次元	順序	交換法則	結合法則	交代代数	冪結合性	非自明な零因子
実数	1	○	○	○	○	○	×
複素数	2	×	○	○	○	○	×
四元数	4	×	×	○	○	○	×
八元数	8	×	×	×	○	○	×
十六元数	16	×	×	×	×	○	○
（16 >）	>16	×	×	×	×	○	○

- 四元数では交換法則
- 八元数では結合法則

- 十六元数では非自明な零因子の存在

といった具合で、少しずつ代数的性質が失われていきます。



▲図 4.5　超複素数の代数的性質の包含関係

この表の内容を、実際にコードを動かして確認してみましょう。

四元数のテスト

まずは、四元数で交換法則が失われることを確認します。四元数モジュール（Quaternion）は先ほど修正した Complexical.Make(Complex) をそのまま使います。

▲リスト 4.34　quaternion.ml

```
include Complexical.Make(Complex)
```

四元数は Complexical.Make(Complex) をそのまま使うだけで完成します。表示は $a + bi + cj + dk$ という伝統的な 4 項の形にしています。

これで、i と j を実際に作って、積を計算してみます。

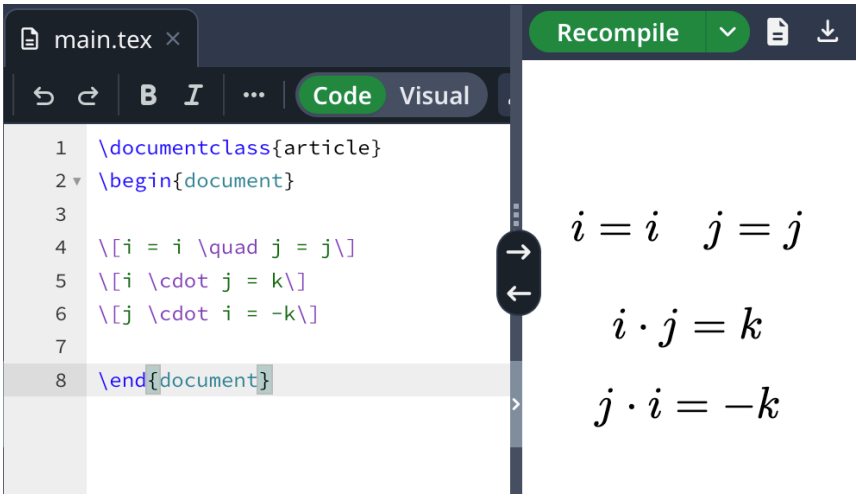
▲リスト 4.35　quaternion_test.ml

```
let complex_i : Complex.t = (Rational.zero, Rational.one)
let i : Quaternion.t = (complex_i, Complex.zero)
let j : Quaternion.t = (Complex.zero, Complex.one)

let () =
  Expr.put (Expr.join [
    Printf.sprintf "i = %s" (Expr.to_string_of (module Expr.Quaternion) i);
    Printf.sprintf "j = %s" (Expr.to_string_of (module Expr.Quaternion) j);
  ]);
  let mul = Expr.cdot_biop Quaternion.mul in
  Expr.put (Expr.biop_to_string (module Expr.Quaternion) mul i j);
  Expr.put (Expr.biop_to_string (module Expr.Quaternion) mul j i)
```

i は複素数の虚数単位をそのまま四元数に埋め込んだもの、j はケーリーディクソン構成で新しく添加された単位です。コンパイルして実行してみましょう。

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
  field.ml fraction.ml rational.ml complexical.ml complex.ml quaternion.ml to_tex.cmxa
quaternion_test.ml && ./a.out
>>
\[i = i \quad j = j\]
\[i \cdot j = k\]
\[j \cdot i = -k\]
```



▲図 4.6　ブラウザ上でチェック

$i \cdot j = k$ である一方、$j \cdot i = -k$ となっており、$i \cdot j \neq j \cdot i$、つまり交換法則が成り立たないことが実際に確認できました。

八元数のテスト

次に、八元数で結合法則が失われることを確認します。八元数モジュール（`Octonion`）は `Complexical.Make(Quaternion)` を使います。

▲リスト 4.36　octonion.ml

```
include Complexical.Make(Quaternion)
```

e_1, e_2, e_3 は四元数の i, j, k をそのまま埋め込んだもの、e_4 はケーリーディクソン構成で新しく添加された単位です。$(e_1 \cdot e_2) \cdot e_4$ と $e_1 \cdot (e_2 \cdot e_4)$ を計算し、比べてみます。

▲リスト 4.37　octonion_test.ml

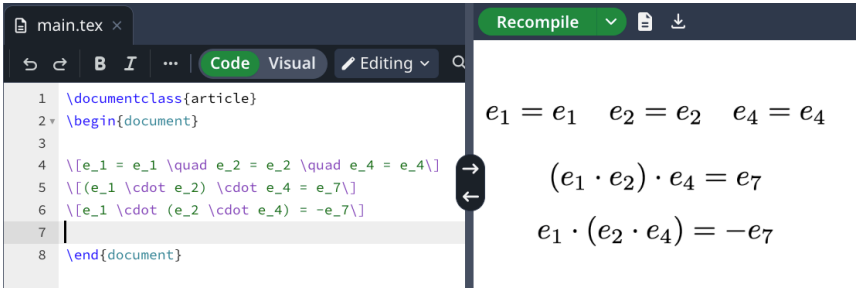
```
let complex_i : Complex.t = (Rational.zero, Rational.one)
let qi : Quaternion.t = (complex_i, Complex.zero)
let qj : Quaternion.t = (Complex.zero, Complex.one)

let e1 : Octonion.t = (qi, Quaternion.zero)
let e2 : Octonion.t = (qj, Quaternion.zero)
let e4 : Octonion.t = (Quaternion.zero, Quaternion.one)

let () =
```

```
  Expr.put (Expr.join [
    Printf.sprintf "e_1 = %s" (Expr.to_string_of (module Expr.Octonion) e1);
    Printf.sprintf "e_2 = %s" (Expr.to_string_of (module Expr.Octonion) e2);
    Printf.sprintf "e_4 = %s" (Expr.to_string_of (module Expr.Octonion) e4);
  ]);
  let lhs1 =
    let inner = Expr.cdot_join ["e_1"; "e_2"] in
    Expr.cdot_join [Printf.sprintf "(%s)" inner; "e_4"]
  in
  Expr.put (Printf.sprintf "%s = %s" lhs1
    (Octonion.mul (Octonion.mul e1 e2) e4
     |> Expr.to_string_of (module Expr.Octonion)));
  let lhs2 =
    let inner = Expr.cdot_join ["e_2"; "e_4"] in
    Expr.cdot_join ["e_1"; Printf.sprintf "(%s)" inner]
  in
  Expr.put (Printf.sprintf "%s = %s" lhs2
    (Octonion.mul e1 (Octonion.mul e2 e4)
     |> Expr.to_string_of (module Expr.Octonion)))
```

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml complexical.ml complex.ml quaternion.ml octonion.ml to_tex.
cmxa octonion_test.ml && ./a.out
>>
\[e_1 = e_1 \quad e_2 = e_2 \quad e_4 = e_4\]
\[(e_1 \cdot e_2) \cdot e_4 = e_7\]
\[e_1 \cdot (e_2 \cdot e_4) = -e_7\]
```



▲図 4.7　ブラウザ上でチェック

$(e_1 \cdot e_2) \cdot e_4 = e_7$ である一方、$e_1 \cdot (e_2 \cdot e_4) = -e_7$ となっており、$(e_1 \cdot e_2) \cdot e_4 \neq e_1 \cdot (e_2 \cdot e_4)$、つまり結合法則が成り立たないことが実際に確認できました。

十六元数のテスト

最後に、十六元数で非自明な零因子が現れることを確認します。十六元数モジュール（`Hexadecanion`）は `Complexical.Make(Octonion)` を使います。

$$1,\ e_1,\ \ldots,\ e_{15} \tag{4.31}$$

▲リスト 4.38　hexadecanion.ml

```
include Complexical.Make(Octonion)
```

非自明な零因子とは、$a \neq 0$ かつ $b \neq 0$ であるにもかかわらず $a \cdot b = 0$ となってしまうような a, b のことです。十六元数の基底 e_1（八元数部分に埋め込まれた虚数単位）と e_4（新しく添加された単位）を使って、$a = e_1 + e_4$、$b = e_1 - e_4$ という 2 つの元をつくり、その積を計算してみます。

▲リスト 4.39　hexadecanion_test.ml

```
let complex_i : Complex.t = (Rational.zero, Rational.one)
let qi : Quaternion.t = (complex_i, Complex.zero)

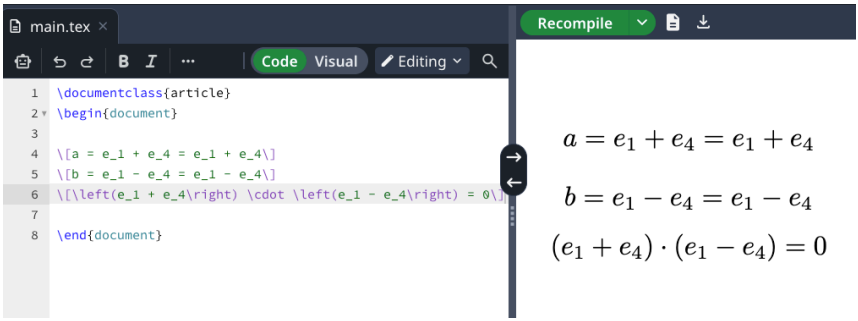
let oe1 : Octonion.t = (qi, Quaternion.zero)
let oe4 : Octonion.t = (Quaternion.zero, Quaternion.one)

let e1 : Hexadecanion.t = (oe1, Octonion.zero)
let e4 : Hexadecanion.t = (oe4, Octonion.zero)

let a : Hexadecanion.t = Hexadecanion.add e1 e4
let b : Hexadecanion.t = Hexadecanion.sub e1 e4

let () =
  let a_str = Expr.to_string_of (module Expr.Hexadecanion) a in
  let b_str = Expr.to_string_of (module Expr.Hexadecanion) b in
  Expr.put (Printf.sprintf "a = e_1 + e_4 = %s" a_str);
  Expr.put (Printf.sprintf "b = e_1 - e_4 = %s" b_str);
  let mul = Expr.cdot_biop Hexadecanion.mul in
  Expr.put (Expr.biop_to_string (module Expr.Hexadecanion) mul a b)
```

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml complexical.ml complex.ml quaternion.ml octonion.ml
hexadecanion.ml to_tex.cmxa hexadecanion_test.ml && ./a.out
>>
\[a = e_1 + e_4 = e_1 + e_4\]
\[b = e_1 - e_4 = e_1 - e_4\]
\[\left(e_1 + e_4\right) \cdot \left(e_1 - e_4\right) = 0\]
```



▲図 4.8　ブラウザ上でチェック

a も b もゼロではないのに、$a \cdot b = 0$ となってしまいました。十六元数では、0 でない数同士をかけても 0 になってしまうことがある、つまり非自明な零因子が存在することが実際に確認できました。これが、十六元数で除法（乗法の逆元）が定義できない理由です。もし b に逆元が存在し、なおかつ結合律が成り立つなら、$a \cdot b = 0$ の両辺に右から b の逆元をかけることで

$$(a \cdot b) \cdot b^{-1} = a \cdot (b \cdot b^{-1}) = 0 \cdot b^{-1} \implies a = 0 \tag{4.32}$$

となり、$a \neq 0$ と矛盾します。しかし、この書き換えは

$$(a \cdot b) \cdot b^{-1} = a \cdot (b \cdot b^{-1}) \tag{4.33}$$

という結合律を前提にしており、十六元数はその結合律を満たしません。実際、

$$b \cdot b^{-1} = b^{-1} \cdot b = 1 \tag{4.34}$$

を満たす b の逆元（共役をノルムで割ったもの）自体は、結合律がなくても形式的に存在してしまいます。それにもかかわらず $a \cdot b = 0$ となってしまうのは、まさに

$$(a \cdot b) \cdot b^{-1} \neq a \cdot (b \cdot b^{-1}) \tag{4.35}$$

となる、つまり結合律が崩れているからです。逆元の候補が形式上存在することと、それを使って矛盾なく除法を行えることが乖離してしまう、これが十六元数で除法が定義できない本当の理由です。

4.4 ベクトル

この節では、ベクトルを実装していきます。

さて、ベクトルは馴染みのある代数構造でしょう。物理量や座標を表すのに、すでに何度も目にしてきたはずです。ベクトルには、次の 2 つの重要な演算があります。

- スカラー倍
- ベクトル同士の和

$$3 \cdot \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 3 \\ 6 \end{pmatrix} \tag{4.36}$$

ベクトルの要素である**スカラー**（*Scalar*）に対して、スカラー倍を行っています。一方、

$$\begin{pmatrix} 1 \\ 3 \end{pmatrix} + \begin{pmatrix} 2 \\ 4 \end{pmatrix} = \begin{pmatrix} 3 \\ 7 \end{pmatrix} \tag{4.37}$$

これは、ベクトルの各成分（スカラー）同士で演算を行うことで、全体としてのベクトル同士の和が実現されている、ということです。

ここで、**ベクトル空間**（*Vector Space*）を定義しましょう。ベクトル空間とは、集合 V と、体 F（スカラーの集合）の組 $(V, +, \cdot)$ であって、次の性質を満たすもののことです。

- $(V, +)$ が加法群である（V の元同士は加法・減法ができる）
- スカラー倍 $F \times V \to V$ が定義されていて、次の法則を満たす

$$\begin{aligned}
&\text{スカラーに関する分配律} && (a+b)\cdot v = a\cdot v + b\cdot v \\
&\text{ベクトルに関する分配律} && a\cdot(v_1+v_2) = a\cdot v_1 + a\cdot v_2 \\
&\text{結合律} && (a\cdot b)\cdot v = a\cdot(b\cdot v) \\
&\text{単位元} && 1\cdot v = v
\end{aligned} \tag{4.38}$$

このとき、$(V, +, \cdot)$ は、体 F 上のベクトル空間である、というように表現され、V の元を**ベクトル**（*Vector*）と呼びます。V 自身は、加法ができることから加法群ですが、V 同士の乗法は定義に含まれていないため、**環**（*Ring*）ではありません。

ベクトルを使う時、よく使う概念として、**単位ベクトル**（*Unit Vector*）というものがありました。これは、大きさ（ノルム）が 1 であるベクトルのことです。もとのベクトル v から単位ベクトルをつくるには、

$$\hat{v} = \frac{v}{|v|} \tag{4.39}$$

のように、v をその大きさ $|v|$（スカラー）で割る必要があります。そのため、ベクトル空間では、スカラーが**環**（*Ring*）ではなく、**体**（*Field*）である必要があるのです。

ここまでベクトル空間の性質を見てきたので、具体的な例を見ていきましょう。もっともシンプルなベクトル空間は、**数ベクトル空間**（*Numeric Vector Space*）です。数ベクトル空間は、

$$(a_1, a_2, \ldots, a_n) \quad a_i \in F \tag{4.40}$$

のように、n 個のタプルで表されます。我々が、座標を表したり、物理量を表したりするとき、実数体上の数ベクトル空間を考えていたということです。

また、複素数や四元数なども、実はそれ自体がベクトル空間におけるベクトルになっています。先ほどつくった複素数を例にすると、複素数は有理数体上の 2 次元数ベクトル空間です。

加法

$$(a+bi)+(c+di) = a+c+(b+d)i \iff (a,b)+(c,d) = (a+c, b+d) \tag{4.41}$$

スカラー倍

$$s \cdot (a+bi) = sa + sbi \iff s \cdot (a,b) = (sa, sb) \tag{4.42}$$

このように、複素数の加法やスカラー倍は、ベクトルの加法やスカラー倍と同じように振る舞っていることがわかります。

なお、数ベクトル空間でないベクトル空間の例として、行列があります。

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} \quad a_i \in F \tag{4.43}$$

行列は、数ベクトル空間とは違い、二次元的なコンテナ構造を持っています。

4.4.1 ベクトル空間ファンクタ

ベクトル空間ファンクタをつくっていきましょう。この役割は、スカラー体 F からベクトル空間 V を生成することですが、スカラー体だけでは生成できません。なぜなら、同じ体でも、数ベクトル空間や行列が考えられるように、ベクトル空間は一意に定まらないためです。

そのため、スカラー体 F に加えて、ベクトル空間のコンテナ構造 C の情報も受け取り、ベクトル空間 V を返すファンクタにします。

ファンクタの引数となる、体 F のシグネチャには、複素数の節でつくった `Field.S` をそのまま使います。

コンテナのシグネチャを作成します。コンテナは、ベクトル空間の要素を格納するための構造です。

▲リスト 4.40　vector_space.ml

```
module type Container = sig
  type 'a t
  val make: 'a -> 'a t
  val map: ('a -> 'b) -> 'a t -> 'b t
  val map2: ('a -> 'b -> 'c) -> 'a t -> 'b t -> 'c t
  val for_all2: ('a -> 'b -> bool) -> 'a t -> 'b t -> bool
end
```

- make: コンテナのコンストラクタ
- map: コンテナの要素に対して関数を適用する
- map2: 2 つのコンテナの要素に対して関数を適用する
- for_all2: 2 つのコンテナの要素に対して判定関数を適用し、すべての要素が真であるかを判定する

今回はこの 4 つの関数をコンテナのシグネチャとしますが、これは唯一の正解ではなく、ベクトル空間をつくる上で必要な関数を定義しただけなので、他の実装方法も考えられます。

では、ベクトル空間ファンクタのシグネチャを作成します。

▲リスト 4.41　vector_space.ml

```
module type S = sig
    type t
    type scalar

    val equal: t -> t -> bool
    include Add_group.S with type t := t

    val mul_scalar: scalar -> t -> t
    val div_scalar: scalar -> t -> t
end

module type FUNCTOR_TYPE =
  functor (F: Field.S) (C: Container) ->
    (S with type scalar = F.t and type t = F.t C.t)
```

ファンクタの返り値のシグネチャは、

- 加法群
- スカラー倍

を満たすようになっています。それでは、ファンクタのシグネチャをもとに、ベクトル空間ファンクタ（Vector_space.Make）の実装を行います。

▲リスト 4.42　vector_space.ml

```
module Make: FUNCTOR_TYPE = functor (F: Field.S) (C: Container) -> struct
    type t = F.t C.t
    type scalar = F.t
    (* 等価判定 *)
    let equal (x1: t) (x2: t): bool = C.for_all2 F.equal x1 x2
    (* 零ベクトル *)
    let zero: t = C.make F.zero
    (* 加法 *)
    let add (x1: t) (x2: t): t = C.map2 F.add x1 x2
    (* 符号反転 *)
    let neg (x: t): t = C.map F.neg x

    include Add_group.Extend(struct
      type nonrec t = t
      let zero, add, neg = zero, add, neg
    end)
    (* スカラー倍 *)
    let mul_scalar (s: scalar) (v: t): t = C.map (fun x -> F.mul s x) v
    (* スカラーでの除法 *)
    let div_scalar (s: scalar) (v: t): t =
      C.map (fun x -> F.mul (F.inv s) x) v
end
```

これで、ベクトル空間ファンクタ（Vector_space.Make）が完成しました。

4.4.2　数ベクトル空間

作成したベクトル空間ファンクタ（Vector_space.Make）を使って、数ベクトル空間ファンクタ（Numeric_vector_space.Make）を作成してみましょう。スカ

ラー体として、有理数モジュール（Rational）を使用し、コンテナとして、リストを使用します。次元を指定できるようにするため、数ベクトル空間ファンクタ（Numeric_vector_space.Make）自体は、次元の情報を受け取るようにします。

▲リスト 4.43　numeric_vector_space.ml

```
module type Dim = sig
  val dim: int
end

module type FUNCTOR_TYPE =
  functor (D: Dim) (F: Field.S) ->
    (Vector_space.S with type scalar = F.t and type t = F.t list)

module Make: FUNCTOR_TYPE = functor (D: Dim) (F: Field.S) -> struct
  module Container = struct
    include List
    (* 定数ベクトルの生成 *)
    let make (x: 'a): 'a t =
      List.init D.dim (fun _ -> x)
  end
  include Vector_space.Make(F)(Container)
end
```

コンテナとしてリストを使っているので、出力の型 t も F.t list に確定しています。

数ベクトル空間の他にも、ベクトル空間の例として行列をつくることができますが、本書では方程式の求解や微分・積分の応用に一度も登場しないため割愛します。次はまた別の例として、多項式をつくっていきましょう。多項式もまた、あとで見るようにベクトル空間の一例になっています。

4.4.3　テスト

作成した数ベクトル空間ファンクタ（Numeric_vector_space.Make）の動作を確認してみましょう。有理数体 $\mathbb{Q}$ 上の 3 次元数ベクトル空間として、Rational_vector モジュールをつくります。

▲リスト 4.44　"rational_vector.ml"

```
module Dim = struct
  let dim = 3
end
include Numeric_vector_space.Make(Dim)(Rational)
```

ベクトルは有理数のリストとして表現されるので、表示には各成分を文字列化してから組として並べる小さな補助関数を用意します。

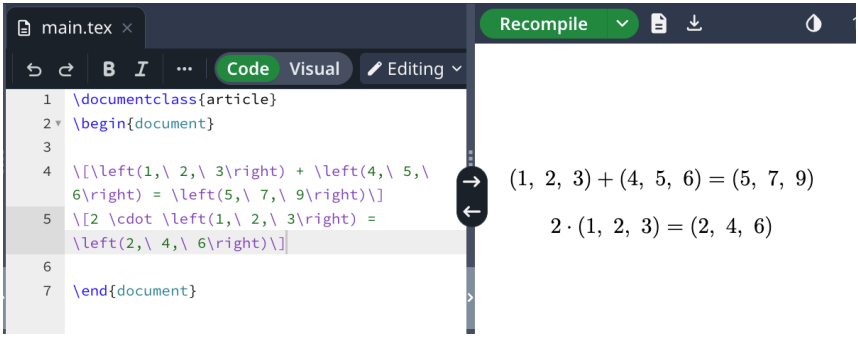
▲リスト 4.45　rational_vector_test.ml

```
let vec_to_string (v: Rational_vector.t): string =
  v |> List.map (Expr.to_string_of (module Expr.Rational)) |> Expr.tuple

let () =
  let a : Rational_vector.t = List.map Rational.of_integer [1; 2; 3] in
  let b : Rational_vector.t = List.map Rational.of_integer [4; 5; 6] in
  let s = Rational.of_integer 2 in

  Expr.put (Printf.sprintf "%s + %s = %s"
    (vec_to_string a) (vec_to_string b) (vec_to_string (Rational_vector.add a b)));
  Expr.put (Printf.sprintf "%s \\cdot %s = %s"
    (Expr.to_string_of (module Expr.Rational) s) (vec_to_string a)
    (vec_to_string (Rational_vector.mul_scalar s a)))
```

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
  field.ml fraction.ml rational.ml vector_space.ml numeric_vector_space.ml rational_vector.ml
to_tex.cmxa rational_vector_test.ml && ./a.out
>>
\[\left(1,\ 2,\ 3\right) + \left(4,\ 5,\ 6\right) = \left(5,\ 7,\ 9\right)\]
\[2 \cdot \left(1,\ 2,\ 3\right) = \left(2,\ 4,\ 6\right)\]
```



▲図 4.9　ブラウザ上でチェック

ベクトルの加法が成分ごとの和として、スカラー倍が各成分に同じ数を掛ける操作として、それぞれ定義どおりに計算できていることが確認できました。

4.5　多項式

この節では多項式をつくっていきます。多項式は、$2x^2 + x - 1$ や $x^4 + 1$ のように、変数 x の累乗の和で表されるものでした。多項式には加法や乗法を考えることができます。

$$(x+3)+(x-2)=2x+1 \tag{4.44}$$

$$(x+3)\cdot(x-2)=x^2+x-6 \tag{4.45}$$

このことから、多項式は**環**（*Ring*）であることがわかります。さらに、これらの演算をよく見ると、変数 x という記号自体は本質的に必要ではないことに気づきます。x であろうが y であろうが、計算結果の意味は変わらないからです。

そのため、多項式を数のペアとして考えてみましょう。たとえば 1 次多項式 $ax+b$ は、0 次の係数と 1 次の係数を順に並べたペア (b,a) に対応します。

$$\begin{aligned}
(a_1x+b_1)+(a_2x+b_2) &= (b_1,a_1)+(b_2,a_2)\\
&= (b_1+b_2,\ a_1+a_2)\\
\\
(a_1x+b_1)\cdot(a_2x+b_2) &= (b_1,a_1)\cdot(b_2,a_2)\\
&= (b_1b_2,\ a_1b_2+b_1a_2,\ a_1a_2)\\
\\
s\cdot(ax+b) &= s\cdot(b,a)\\
&= (sb,sa)
\end{aligned} \tag{4.46}$$

加法の結果は 2 つの成分のペアのままですが、乗法の結果は 3 つの成分になっていることに注目してください。1 次多項式同士を掛けると 2 次多項式になるので、これは自然なことです。多項式の次数は演算によって変わりうる、ということです。このように、数のペア同士の演算とした方が、多項式が環であることがわかりやすくなります。

ペア同士の演算に加えて、スカラー倍も行えることがわかります。

ここで、係数のスカラーについて考えてみましょう。右辺のペアの中身を見ると、係数のスカラーには $a_1b_2+b_1a_2$ のように乗法と加法ができる必要があることがわかります。

さらに、多項式の商についても考えてみましょう。たとえば、x^2 を $2x$ で割ってみます。

$$x^2 \div 2x = \frac{1}{2}x \tag{4.47}$$

商の最高次の項を求めるには、係数同士を $1 \div 2$ のように割る必要があり、整数の範囲には収まらない値が登場します。つまり、係数のスカラーには乗法だけでなく除法（逆元）も必要だということです。

ここまでをまとめると、加法・減法・乗法に加えて除法まで必要になるので、係数のスカラーは**環**（*Ring*）ではなく**体**（*Field*）である必要があります。

- スカラー: **体**（*Field*）
- 多項式自体: **環**（*Ring*）

となります。

多項式を考えるときに、もう 1 つ重要なことがあります。値の評価、つまり何かを代入して値を求める操作です。x^2+4x+1 に $x=3$ を代入すると、$3^2+4\cdot3+1=22$ のように計算できます。

値を評価するときは、係数スカラーとの乗法が必ず発生します。そのため、代入できる値は、係数スカラーと同じ代数に限られます。係数スカラーが整数なら、代入する値も整数である必要がある、ということです。

それでは、これまでの議論をもとに実装していきましょう。まず、係数として使える代数が満たすべき性質を、先ほど確認したとおり**体**（*Field*）としてシグネチャにします。多項式を係数体上のベクトル空間として扱いたい、という点でも都合がよいことは、ベクトルの節で見たとおりです。係数が満たすべき性質は、複素数やベクトル空間のときと同じく `Field.S` そのものなので、新たにシグネチャを定義し直す必要はありません。ファンクタの引数として、そのまま `Field.S` を受け取ります。

多項式そのものが満たすべきシグネチャ `S` も定義しておきます。

▲リスト 4.46　"polynomial.ml"

```
module type S = sig
  type coeff
  type t = coeff list
  val zero: t
  val one: t
  val normalize: t -> t
  val equal: t -> t -> bool
  val degree: t -> int
  val mul: t -> t -> t
  val add: t -> t -> t
  val neg: t -> t
  val eval: t -> coeff -> coeff
  val div_rem: t -> t -> t * t
  val pow: int -> t -> t
  val sub: t -> t -> t
  val gcd: t -> t -> t
```

```
  val ext_gcd: t -> t -> t * t * t
  val leading_coeff: t -> coeff
  val div_scalar: coeff -> t -> t
  val mul_scalar: coeff -> t -> t
end
```

最大公約数・拡張ユークリッドの互除法・スカラーによる除法・スカラー倍など、まだ登場していない演算も混ざっていますが、これらは実装しながら順番に説明していきます。

これで入出力のシグネチャがそろったので、ファンクタの型を決めておきましょう。

▲リスト 4.47　"polynomial.ml"

```
module type FUNCTOR_TYPE = functor (C: Field.S) -> (S with type coeff = C.t)
```

S の中で `type t = coeff list` とすでに定義してあるので、`coeff` を `C.t` に固定するだけで、`t` も自動的に `C.t list` に決まります。

多項式は、係数を低次から並べたリスト `C.t list` として表現します。たとえば $[c_0; c_1; c_2]$ は $c_0 + c_1 x + c_2 x^2$ を表す、という対応です。

▲リスト 4.48　"polynomial.ml"

```
module Make: FUNCTOR_TYPE = functor (C: Field.S) -> struct
  type t = C.t list
  type coeff = C.t
  let zero: t = []
  let one: t = [C.one]
  (* 正規化 *)
  let normalize (x: t): t =
    let rec remove_leading_zeros = function
      | [] -> []
      | c :: cs ->
        if C.equal c C.zero then remove_leading_zeros cs
        else c :: cs
    in
    List.rev x |> remove_leading_zeros |> List.rev

  (* 等価判定 *)
  let equal (x1: t) (x2: t): bool =
    let p1 = normalize x1 in
    let p2 = normalize x2 in
    let rec aux l1 l2 =
      match l1, l2 with
      | [], [] -> true
      | [], _ | _, [] -> false
      | c1 :: cs1, c2 :: cs2 -> C.equal c1 c2 && aux cs1 cs2
    in
    aux p1 p2

  (* 次数 *)
  let degree (p: t) : int =
    match normalize p with
    | [] -> -1
    | p' -> List.length p' - 1

  (* 最高次係数 *)
  let leading_coeff (p: t): C.t =
```

```
  match List.rev p with
  | [] -> C.zero
  | x::xs -> x
```

正規化は、リストの末尾（最高次の係数）に並ぶ 0 を取り除く処理です。同じ多項式でも、末尾に 0 が並ぶかどうかでリストとしての表現が変わってしまうため、正規化によってこの表現の自由度をなくし、等価判定などで正しく比較できるようにしています。次数が -1 のときは、多項式全体が 0 であることを表します。

次に乗法を実装します。

▲リスト 4.49　"polynomial.ml"

```
(* 乗法 *)
let mul (x1: t) (x2: t): t =
  let p1 = normalize x1 in
  let p2 = normalize x2 in
  if p1 = [] || p2 = [] then []
  else
    let res_len = List.length p1 + List.length p2 - 1 in
    let res = Array.make res_len C.zero in
    List.iteri (fun i c1 ->
      List.iteri (fun j c2 ->
        res.(i + j) <- C.add res.(i + j) (C.mul c1 c2)
      ) p2
    ) p1;
    Array.to_list res |> normalize
```

たとえば、$(x+1)(x+2)$ を展開すると x^2+3x+2 になりますが、係数ごとに見ると次のように対応しています。

$$\begin{aligned} 0\text{ 次の係数} &= c_0d_0 = 1\cdot 2 = 2 \\ 1\text{ 次の係数} &= c_0d_1 + c_1d_0 = 1\cdot 1 + 1\cdot 2 = 3 \\ 2\text{ 次の係数} &= c_1d_1 = 1\cdot 1 = 1 \end{aligned} \tag{4.48}$$

k 次の係数を求めるのに、足し合わせて k になる次数の組み合わせすべてについて係数の積をとっていることがわかります。これを一般化すると、係数リスト $[c_0; c_1; \ldots]$ が $c_0 + c_1x + c_2x^2 + \cdots$ を表すので、2 つの多項式の積の k 次の係数は、

$$\sum_{i+j=k} c_id_j \tag{4.49}$$

という**畳み込み**（*Convolution*）になります。コードは、この畳み込みの式をそのまま実装したものです。

加法・否定・評価はいたってシンプルです。

▲リスト 4.50　"polynomial.ml"

```
(* 加法 *)
let rec add (x1: t) (x2: t): t =
  match x1, x2 with
  | [], l | l, [] -> l
  | c1 :: cs1, c2 :: cs2 -> C.add c1 c2 :: add cs1 cs2

(* 符号反転 *)
let neg (x: t): t =
  List.map C.neg x

(* 評価 *)
let eval (xs: t) (x: C.t): C.t =
  List.mapi (fun i c -> C.mul c (C.pow i x)) xs
  |> List.fold_left (fun acc s -> C.add acc s) C.zero
```

多項式の値は、係数 c_i に x^i を掛けて足し合わせる、$\sum_i c_i x^i$ という式そのものです。代入する x の型が係数の型と同じでなければならないのは、この計算に係数同士の乗法が必ず発生するためです。

ここまでで加法・乗法がそろったので、加法群ファンクタ（`Add_group.Extend`）・乗法モノイドファンクタ（`Mul_monoid.Extend`）を適用して、減法などを自動生成しておきましょう。

▲リスト 4.51　"polynomial.ml"

```
include Mul_monoid.Extend(struct
  type nonrec t = t
  let one, mul = one, mul
end)

include Add_group.Extend(struct
  type nonrec t = t
  let add, neg = add, neg
end)
```

続いて、多項式の商と余りを求める演算を実装します。

▲リスト 4.52　"polynomial.ml"

```
(* 商と余り *)
let div_rem (num: t) (den: t) : t * t =
  let den_norm = normalize den in
  if den_norm = [] then failwith "Division by zero polynomial"
  else
    let lc_den = List.nth den_norm (List.length den_norm - 1) in
    let deg_den = degree den_norm in
    let rec aux rem quo_acc =
      let rem_norm = normalize rem in
      let deg_rem = degree rem_norm in
      if deg_rem < deg_den then
        (normalize quo_acc, rem_norm)
      else
        let lc_rem = List.nth rem_norm deg_rem in
        let factor = C.div lc_rem lc_den in
        let shift = deg_rem - deg_den in
        let term =
          List.init (shift + 1)
```

```
            (fun i -> if i = shift then factor else C.zero)
        in
        let new_rem = sub rem_norm (mul term den_norm) in
        let new_quo = add quo_acc term in
        aux new_rem new_quo
    in
    aux num []
```

やっていることは、筆算で多項式の商と余りを求める手順そのものです。

係数の代数に除法まで要求していた理由はここにあります。整数のような環では商が割り切れるとは限りません（たとえば 3 を 2 で割り切ることはできません）が、係数が体であれば、0 でない数では必ず割り切れます。

商と余りさえ手に入れば、整数のときとまったく同じ手順で最大公約数が計算できます。

▲リスト 4.53　"polynomial.ml"

```
include Euclidean.Extend(struct
  type nonrec t = t
  let zero, one, sub, mul, div_rem, equal =
    zero, one, sub, mul, div_rem, equal
end)
```

整数の節で紹介したユークリッドの互除法モジュール（Euclidean）は、商と余りをまとめて返す演算さえ受け取れば、最大公約数に加えて拡張ユークリッドの互除法まで導出してくれるのでした。多項式の商と余りもちょうど同時に返す形になっていたので、このモジュールがそのまま使えます。最大公約数・拡張ユークリッドの互除法が具体的にどう役立つのかは、体の拡大の節で詳しく見ていきます。

最後に、多項式は係数体 C 上のベクトル空間でもあります。先ほどのベクトル空間ファンクタでつくったスカラー倍・スカラーによる除法を、そのまま多項式にも使いたいところです。

▲リスト 4.54　"polynomial.ml"

```
  include (Vector_space.Make(C)(struct
    include List
    let make x = []
  end) : sig
    type t
    val mul_scalar: C.t -> t -> t
    val div_scalar: C.t -> t -> t
  end with type t := t)
end
```

ベクトル空間ファンクタ（Vector_space.Make）を丸ごと include してしまうと、そちらの等価判定や加法が後勝ちで上書きされてしまいます。ベクトル空間側の等価判定は固定長のコンテナを前提にしているため、次数の異なる多項式同士を比較すると例外になってしまうのです。欲しいのはあくまで「多項式は係数体上のベクトル空

間でもある」という性質から得られるスカラー倍・スカラーによる除法だけなので、シグネチャで絞り込んで必要な演算だけを取り込んでいます。

これで、任意の係数体 C から多項式環をつくるファンクタが完成しました。

4.5.1 テスト

係数体に有理数を使って、実際に動かしてみましょう。

▲リスト 4.55　polynomial_test.ml

```
module P = Polynomial.Make(Rational)

let () =
  let p1 : P.t =
    [Rational.of_integer 1; Rational.of_integer 2; Rational.of_integer 3]
  in
  let p2 : P.t = [Rational.of_integer 1; Rational.of_integer 1] in
  let p1_str = Expr.to_string_of (module Expr.Q_poly) p1 in
  let p2_str = Expr.to_string_of (module Expr.Q_poly) p2 in

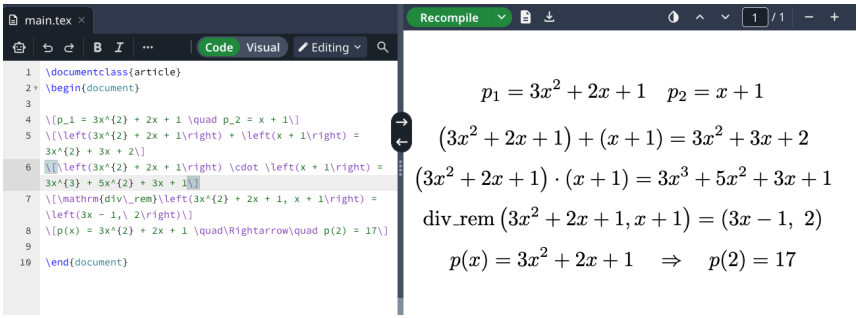
  Expr.put (Expr.join [
    Printf.sprintf "p_1 = %s" p1_str;
    Printf.sprintf "p_2 = %s" p2_str;
  ]);
  Expr.put (Expr.biop_to_string (module Expr.Q_poly)
    (Expr.infix_biop "+" P.add) p1 p2);
  Expr.put (Expr.biop_to_string (module Expr.Q_poly)
    (Expr.cdot_biop P.mul) p1 p2);

  let (q, r) = P.div_rem p1 p2 in
  let q_str = Expr.to_string_of (module Expr.Q_poly) q in
  let r_str = Expr.to_string_of (module Expr.Q_poly) r in
  Expr.put (Expr.call "div_rem" [p1_str; p2_str] (Expr.tuple [q_str; r_str]));

  Expr.put (Expr.poly_eval_to_string (module Expr.Q_poly) (module Expr.Rational)
    P.eval p1 (Rational.of_integer 2))
```

$p_1 = 1 + 2x + 3x^2$、$p_2 = 1 + x$ として、和・積・除法・評価を確認します。コンパイルして実行してみましょう。

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml vector_space.ml polynomial.ml to_tex.cmxa polynomial_test.ml
 && ./a.out
>>
\[p_1 = 3x^{2} + 2x + 1 \quad p_2 = x + 1\]
\[\left(3x^{2} + 2x + 1\right) + \left(x + 1\right) = 3x^{2} + 3x + 2\]
\[\left(3x^{2} + 2x + 1\right) \cdot \left(x + 1\right) = 3x^{3} + 5x^{2} + 3x + 1\]
\[\mathrm{div\_rem}\left(3x^{2} + 2x + 1, x + 1\right) = \left(3x - 1,\ 2\right)\]
\[p(x) = 3x^{2} + 2x + 1 \quad\Rightarrow\quad p(2) = 17\]
```



▲図 4.10　ブラウザ上でチェック

積は $(1 + 2x + 3x^2)(1 + x) = 1 + 3x + 5x^2 + 3x^3$ となり、手計算と一致します。商と余りも $(1 + 2x + 3x^2) = (x + 1)(3x - 1) + 2$ という関係を満たしています。$1 + 2x + 3x^2$ に $x = 2$ を代入した値も、手計算どおり $1 + 4 + 12 = 17$ になっています。

4.6　体の拡大

次に、体の拡大をつくっていきます。

4.6.1　拡大体とは

一般に、体 K を部分体として含む、より大きな体 K' のことを、K の**拡大体**（*Field Extension*）と呼びます。K' は K にはなかった新しい元を持ちながら、K の元同士の計算結果はそのまま引き継ぐ、というイメージです。

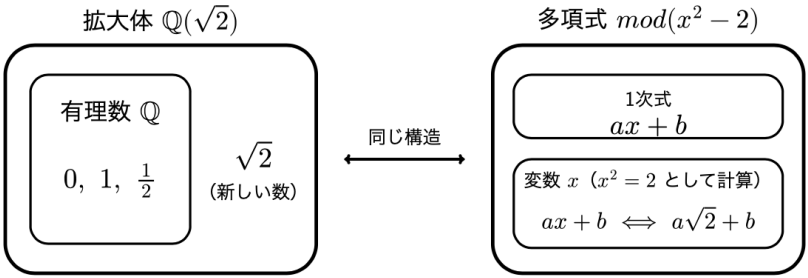
たとえば、複素数の節でつくった $\mathbb{C}$ は $\mathbb{R}$ の拡大体です。$\mathbb{R}$ にはなかった i という新しい元を持ちますが、実数同士の加法・乗法の結果は $\mathbb{R}$ のときのままです。同様に、$\sqrt{2}$ を含む最小の体も、$\mathbb{Q}$ の拡大体です。

体 K に新しい元 α を 1 つ添加してできる拡大体のことを、$K(\alpha)$ と表記します。今の例で言えば、$\sqrt{2}$ を添加した拡大体は $\mathbb{Q}(\sqrt{2})$ と書けます。この節では、こうして既存の体へ平方根や立方根といった新しい元を 1 つ添加していく方法を実装していきます。

4.6.2 多項式による拡大体の構成

まずは直感的なイメージから始めましょう。たとえば、「x^2 が出てきたら、いつでも 2 として扱ってよい」という約束をつくりたいとします。これは $x^2 - 2 = 0$ が成り立つとみなすことと同じであり、計算の途中でどれだけ次数の高い多項式が出てきても、$x^2 - 2$ で割った**余りだけ**が意味を持つ、ということでもあります（$x^2 - 2$ の倍数の部分は、この約束のもとでは 0 になって消えます）。これを一般の体 K と多項式 $f(x)$ に対して行うのが、これから実装する拡大体の構成です。

こうした拡大体を、既存の体 K に新しい元を 1 つ添加することで機械的に構成する方法を考えます。K の中に根を持たない多項式 $f(x)$（**既約多項式**（*Irreducible Polynomial*）と呼びます）に対して、多項式環 $K[x]$ を $f(x)$ で割った余りの集合 $K[x] \bmod f(x)$ を考えると、これが K に $f(x)$ の根を添加した拡大体になります。K の元はすべて 0 次の多項式として $K[x] \bmod f(x)$ にそのまま含まれるので、K は部分体として埋め込まれています。x を $f(x)$ で割った余り、つまり変数 x 自身に対応する元に注目すると、余りの世界では $f(x) \equiv 0$ なので、この元は方程式 $f(x) = 0$ の根になっています。$f(x)$ は K の中には根を持たない多項式だったので、この x こそが $K[x] \bmod f(x)$ で新しく手に入った根であり、$K[x] \bmod f(x)$ は x を添加した拡大体 $K(x)$ にほかなりません。



▲図 4.11　有理数に $\sqrt{2}$ を拡大する例

さっそく実装していきましょう。既約多項式による拡大体ファンクタ（`Field_extension.Make`）に必要なものは、係数体 C と、その係数体上の多項式環 P、そして割る対象となる既約多項式 $poly$ です。

▲リスト 4.56　field_extension.ml

```
module type IrreduciblePoly = sig
  module C : Field.S
  module P : Polynomial.S with type coeff = C.t
  val poly : P.t
end
```

この拡大体ファンクタ（Field_extension.Make）が返すのは、逆元まで含めて体の性質を満たす代数、つまり Field.S です。

▲リスト 4.57　"field_extension.ml"

```
module type FUNCTOR_TYPE =
  functor (I: IrreduciblePoly) -> (Field.S with type t = I.P.t)
```

with type t = I.P.t を付けているのは、この先 $K(\sqrt{2})$ の元を多項式の係数リストとして直接組み立てたい場面があるためです。

ファンクタの実装は次のようになります。

▲リスト 4.58　field_extension.ml

```
module Make: FUNCTOR_TYPE = functor (I: IrreduciblePoly) -> struct
  include I.P
  (* 正規化 *)
  let normalize (p: t): t =
    div_rem p I.poly |> snd
  (* 等価判定 *)
  let equal (x: t) (y: t): bool =
    equal (normalize x) (normalize y)
  (* 加法 *)
  let add (x: t) (y: t): t =
    add x y |> normalize
  (* 符号反転 *)
  let neg (x: t): t =
    neg x |> normalize
  (* 乗法 *)
  let mul (x: t) (y: t): t   =
    mul x y |> normalize
  (* 逆元 *)
  let inv (x: t): t =
    let x_norm = normalize x in
    if equal x_norm I.P.zero then
      failwith "Division by zero in extension field"
    else
      let (g, u, _) = ext_gcd x_norm I.poly in
      let lc = I.P.leading_coeff g in
      let u_monic = div_scalar lc u in
      normalize u_monic

  include Add_group.Extend(struct
      type nonrec t = t
      let add, neg = add, neg
    end)
  include Mul_monoid.Extend(struct
    type nonrec t = t
    let one, mul = one, mul
  end)
  include Mul_group.Extend(struct
    type nonrec t = t
```

```
    let one, mul, inv = one, mul, inv
  end)
end
```

加法・乗法は、多項式としての演算をしたあとに正規化（既約多項式で割った余りをとる）をかけるだけです。

注目したいのは逆元を求める演算です。今度はフェルマーの小定理のような近道は使えません。かわりに、整数の節で最大公約数を求めるのに使った、互除法が再登場します。ここで逆元を求めたいのは、生成元 x そのものとは限りません。コード中の引数名 x は、拡大体の任意の非零元（次数が f 未満の多項式 $a(x)$ として表される元全般）を指す仮の名前だと考えてください。**拡張ユークリッドの互除法**（*Extended Euclidean Algorithm*）は、二つの元 a, b に対して

$$g = u \cdot a + v \cdot b \tag{4.50}$$

を満たす最大公約数 g と係数 u, v を同時に求めます。$f(x)$ が既約で $a(x)$ が 0 でなければ、$a(x)$ と $f(x)$ の最大公約数は（定数倍を除いて）1 になります。つまり

$$g = u(x) \cdot a(x) + v(x) \cdot f(x) \tag{4.51}$$

の両辺を $f(x)$ で割った余りをとると、右辺の $v(x) \cdot f(x)$ の項は消えて

$$g \equiv u(x) \cdot a(x) \pmod{f(x)} \tag{4.52}$$

となります。g は定数（0 次多項式）なので、$u(x)$ を g で割ってスケールを直してやれば、それがそのまま $a(x)$ の逆元です。

これで、既約多項式さえ用意すれば、どんな係数体からでも拡大体をつくれるファンクタ（`Field_extension.Make`）が手に入りました。

4.6.3 デモ: $K(\sqrt{2})$ をつくる

このファンクタの一番シンプルな使い道を試してみましょう。有理数体 $\mathbb{Q}$ 上の多項式 $x^2 - 2$ は、有理数の範囲では分解できない既約多項式です（分解できるとしたら $\sqrt{2}$ が有理数ということになってしまいます）。この既約多項式で拡大体をつくってみます。

▲リスト 4.59　root2_field.ml

```
module Root2Poly = struct
  module C = Rational
  module P = Polynomial.Make(C)
  let poly = [C.of_integer (-2); C.zero; C.one] (* x^2 - 2 *)
end

module K = Field_extension.Make(Root2Poly)
```

K.t の中身は Rational.t list、つまり有理数体 $\mathbb{Q}$ 上のベクトル（2 つの成分の組）です。x に対応する元を取り出して、2 乗した値と、逆元をかけた値を実際に計算させてみましょう。

▲リスト 4.60　root2_field_test.ml

```
let () =
  let root2 : Root2_field.K.t = [Rational.zero; Rational.one] in
  Expr.put (Expr.to_string_of (module Expr.Root2_field) root2);
  let square = Expr.pow_unop 2 (fun x -> Root2_field.K.mul x x) in
  Expr.put (Expr.unop_to_string (module Expr.Root2_field) square root2);
  let mul = Expr.cdot_biop Root2_field.K.mul in
  let root2_inv = Root2_field.K.inv root2 in
  Expr.put (Expr.biop_to_string (module Expr.Root2_field) mul root2 root2_inv)
```

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml vector_space.ml polynomial.ml field_extension.ml root2_field
.ml to_tex.cmxa root2_field_test.ml && ./a.out
>>
\[\sqrt{2}\]
\[\sqrt{2}^{2} = 2\]
\[\sqrt{2} \cdot \frac{1}{2}\sqrt{2} = 1\]
```

$\sqrt{2}$ の 2 乗がきちんと 2 になり、逆元をかけると 1 になることが確認できました。多項式の根を 1 つ添加するだけで、その根を含む新しい体が手に入ります。

■

5

第 5 章

二次方程式

第 5 章

二次方程式

方程式を解くとは、$f(x) = 0$ を満たす x を求めることです。整数、有理数、複素数、そして拡大体まで、これまで作り上げてきた代数を使って、いよいよ多項式方程式を解いていきましょう。ここから先の章は、代数構造を組み立てるのではなく道具として使い、具体的な問題を解決していく応用編です。

第 1 章で確認したとおり、方程式を解くアプローチには数値的な方法と代数的な方法があり、本書が一貫して選ぶのは後者です。環・体・拡大体といった代数構造そのものを OCaml のモジュールとして構築し、そのモジュールが定義する演算として解を求めるので、得られる解はそのまま計算を続けたり検算したりできます。

本書はこのあと、2 次方程式を実装します。

5.1 二次方程式を解く

5.1.1 二次方程式の解全体がつくる体

まず 2 次方程式を代数的に解くとはどういうことかを確認しておきます。$ax^2 + bx + c = 0$ の解は、おなじみの解の公式

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{5.1}$$

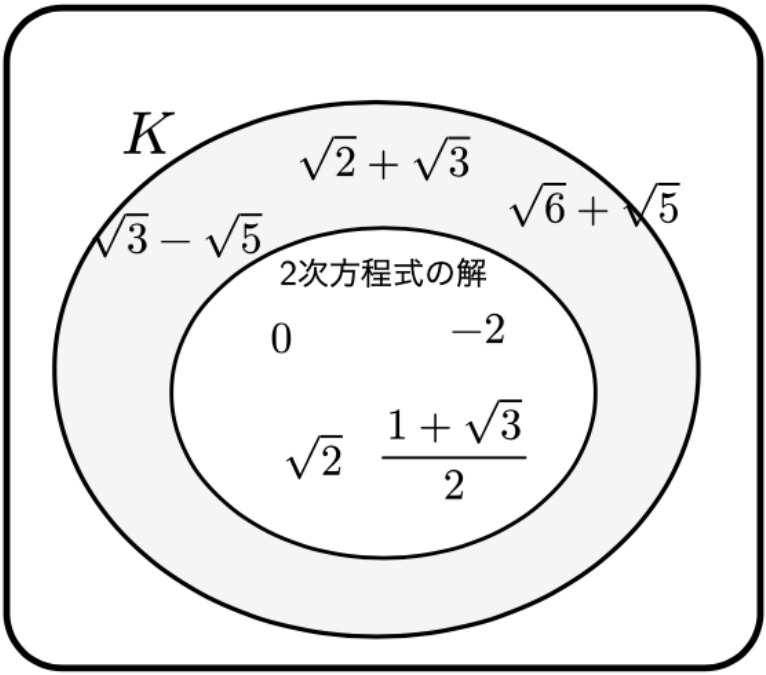
で求まります。四則演算はすでに有理数体 $\mathbb{Q}$ の中で閉じているので、この公式を実行するのに足りないのは、判別式 $b^2 - 4ac$ の平方根だけです。

厄介なのは、判別式は正にも負にもなりうることです。正であれば実数の範囲で平方根が求まりますが、負であれば虚数単位 i が必要になります。つまり、有理数係数

のあらゆる 2 次方程式の解を表現できる体 K には

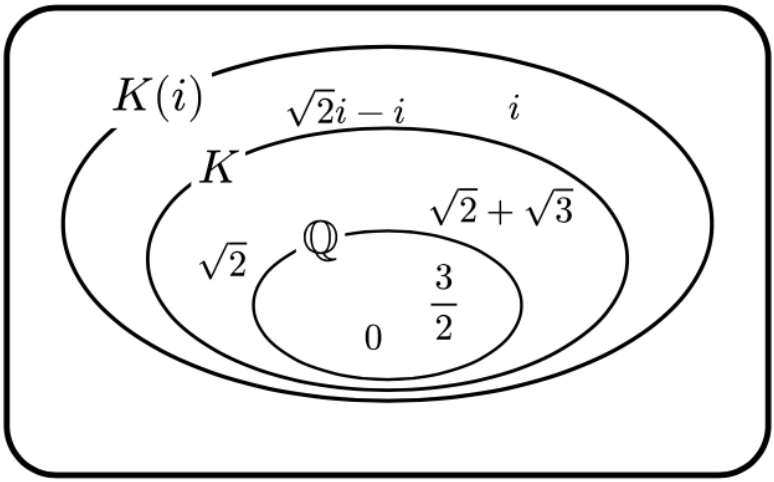
- 判別式が正の場合に備えて、あらゆる平方根
- 判別式が負の場合に備えて、虚数単位 i（$i^2 = -1$ を満たす数）

の両方が必要です。



▲図 5.1　今回作る体 K と二次方程式の解の集合

この 2 つは別々の道具で用意します。まず平方根の方だけを集めた、実数の範囲にとどまる体をつくります。i の方は、複素数の節でつくった複素構造ファンクタ（`Complexical.Make`）をあとから 1 回適用するだけで手に入ります。



▲図 5.2　今回の拡大の流れ

判別式の平方根をどう表すか

「あらゆる平方根」が必要だと言っても、実際に基底として用意すべき数はもっと絞り込めます。判別式 $b^2 - 4ac$ を出発点に、その中身を段階的に整理していきましょう。

まず、a, b, c はすべて有理数なので、判別式 $b^2 - 4ac$ もまた有理数です。つまり最初に考えるべきなのは、**有理数の平方根**をどう扱うかです。

有理数 p/q の平方根は、分母・分子に q を掛けて分母を 2 乗の形にそろえることで、

$$\sqrt{\frac{p}{q}} = \frac{\sqrt{pq}}{q} \tag{5.2}$$

のように、**有理数** $\times$ **整数の平方根**の形に整理できます。たとえば

$$\sqrt{\frac{4}{3}} = \frac{\sqrt{12}}{3} \tag{5.3}$$

です。つまり、有理数の平方根はどんな形をしていても、結局は**整数の平方根**さえ用意しておけば表現できるのです。

残る整数の平方根も、まだ絞り込めます。たとえば $\sqrt{6}$ は、

$$\sqrt{6} = \sqrt{2} \cdot \sqrt{3} \tag{5.4}$$

のように、$\sqrt{2}$ と $\sqrt{3}$ の積として表せます。$\sqrt{6}$ 専用の基底をわざわざ持たなくても、$\sqrt{2}$ と $\sqrt{3}$ さえあれば自動的に手に入るということです。この考え方を一般化すると、任意の正の整数 n は、素因数分解によって

$$n = p_1^{e_1} p_2^{e_2} \cdots \tag{5.5}$$

と書けます。指数が偶数の素因数はそのまま根号の外に出せるので、指数が奇数だった素因数（相異なる素数の積、無平方な部分）だけを根号の中に残せば、

$$\sqrt{n} = (\text{有理数}) \times \sqrt{q_1 q_2 \cdots} \tag{5.6}$$

という形に整理できます。つまり、すべての素数 p の平方根さえ用意しておけば、その積と有理数倍の組み合わせだけで、**どんな整数の平方根も表現できてしまう**わけです。

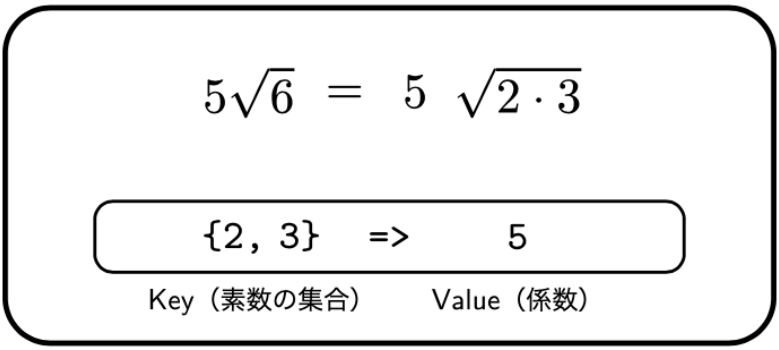
ただし、素数の平方根だけで**表現できる**ことは、基底の選び方として**十分**だと言っているにすぎません。もし欲張って $\sqrt{2}$、$\sqrt{3}$ に加えて $\sqrt{6}$ まで独立な基底にしてしまうと、$\sqrt{6}$ という同じ値が「基底 $\sqrt{6}$ そのもの」と「基底 $\sqrt{2}$ と $\sqrt{3}$ の積」という 2 通りの方法で作れてしまい、等価判定や正規化が破綻します。基底には、あらゆる平方根を過不足なく作り出せること（表現力）に加えて、同じ数の作り方が 2 通り以上存在しないこと（一意性）も求められ、素数の平方根はこの両方を満たします。

まとめると、

$$\begin{array}{c} \text{有理数の平方根} \\ \downarrow \\ \text{有理数} \times \text{整数の平方根} \\ \downarrow \\ \text{有理数} \times \text{素数の平方根の積} \end{array} \tag{5.7}$$

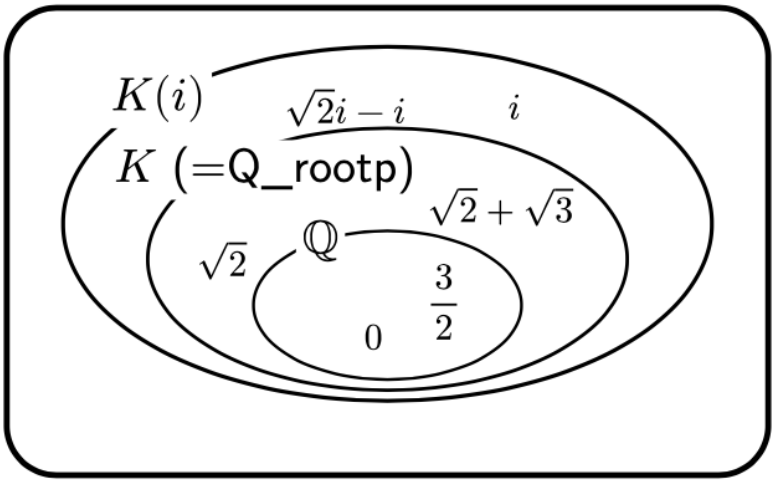
という 2 段階の言い換えによって、判別式の平方根はどんな有理数であっても最終的に素数の平方根だけに帰着するので、本当に拡大すべきなのは素数の平方根だけであり、これ以上細かく分解できない、過不足のない生成元の集合になっています。

これをデータとして表すなら、素数の集合をキー、その積の平方根に掛かる係数（有理数）を値とする辞書型のデータ構造が使えそうです。



▲図 5.3　辞書型データのイメージ

実数の範囲で並列拡大をつくる



▲図 5.4　今回の拡大の流れ

先ほどの発想を、実際にコードにしてみましょう。ここでつくるのは、二次方程式の解の拡大体（Q_rootp）というモジュールです。

▲リスト 5.1　"q_rootp.ml"

```
module PrimeSet = Set.Make(Integer)
module Coeff = Rational

module BasisMap = struct
  include Map.Make(PrimeSet)
  (* 単項コンテナの生成 *)
  let make (v : 'a) : 'a t = singleton PrimeSet.empty v
  let map = map
  (* Zip合成 *)
  let map2 (f : 'a -> 'b -> 'c) (m1 : 'a t) (m2 : 'b t) : 'c t =
    merge (fun _ o1 o2 ->
      match o1, o2 with
      | Some v1, Some v2 -> Some (f v1 v2)
      | _ -> invalid_arg "BasisMap.map2: maps have different domain/basis"
    ) m1 m2
  (* 全称判定 *)
  let for_all2 (p : 'a -> 'b -> bool) (m1 : 'a t) (m2 : 'b t) : bool =
    if cardinal m1 <> cardinal m2 then false
    else
```

```
      for_all (fun k v1 ->
        match find_opt k m2 with
        | Some v2 -> p v1 v2
        | None -> false
      ) m1
end

type t = Coeff.t BasisMap.t
let zero: t = BasisMap.empty
let one: t = BasisMap.singleton PrimeSet.empty Coeff.one
```

マップのキーは、先ほど述べたとおり 1 つの素数ではなく**素数の集合**です。空集合は、何も掛かっていない（$\sqrt{1} = 1$ の）基底なので、有理数そのものを表します。単位元は、この空集合の基底に係数 1 を乗せたものです。

加法はいたって素直です。同じ基底同士の係数を足すだけです。

▲リスト 5.2　"q_rootp.ml"

```
(* 加法 *)
let add (v1: t) (v2: t): t =
  BasisMap.merge (fun _ c1_opt c2_opt ->
    match c1_opt, c2_opt with
    | None, None -> None
    | Some c, None | None, Some c -> Some c
    | Some c1, Some c2 -> Some (Coeff.add c1 c2)
  ) v1 v2

(* 符号反転 *)
let neg (v : t) : t =
  BasisMap.map Coeff.neg v
```

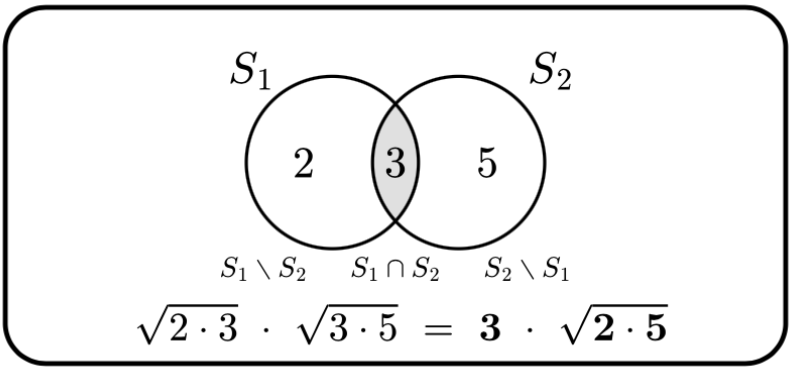
問題は乗法です。2 つの基底となる集合 S_1 と S_2 をもとに、それぞれの平方根同士を掛けたとき、その結果はどの基底に乗るのでしょうか。ここで使うのが、平方根の性質

$$\sqrt{a} \cdot \sqrt{a} = a, \qquad \sqrt{a} \cdot \sqrt{b} = \sqrt{ab} \ (a \neq b) \tag{5.8}$$

です。**両方に共通して含まれる数**は、掛け合わさることで根号の外に出て、ただの整数（有理数）になります。**片方にしか含まれない数**は、根号の中に残ります。つまり、

$$\sqrt{\prod_{S_1}} \cdot \sqrt{\prod_{S_2}} = \left(\prod (S_1 \cap S_2)\right) \cdot \sqrt{\prod (S_1 \triangle S_2)} \tag{5.9}$$

というわけです（$S_1 \triangle S_2$ は**対称差**（*Symmetric Difference*）、つまり「どちらか片方にだけ含まれる要素の集合」を表します）。



▲図 5.5　対象差のイメージと乗法の流れ

コードにすると、次のようになります。

▲リスト 5.3　"q_rootp.ml"

```
(* 基底同士の積 *)
let mul_basis
    (s1 : PrimeSet.t) (s2 : PrimeSet.t) (c1 : Coeff.t) (c2 : Coeff.t)
  : Coeff.t * PrimeSet.t =
  let common = PrimeSet.inter s1 s2 in
  let diff = PrimeSet.union (PrimeSet.diff s1 s2) (PrimeSet.diff s2 s1) in
  let factor_int =
    PrimeSet.fold (fun p acc -> Integer.mul acc p) common Integer.one
  in
  let new_coeff = Coeff.mul (Coeff.mul c1 c2) ((factor_int, Integer.one)) in
  (new_coeff, diff)

(* 基底への係数加算 *)
let add_term (c : Coeff.t) (s : PrimeSet.t) (acc : t) : t =
  let current_c =
    BasisMap.find_opt s acc |> Option.value ~default:Coeff.zero
  in
  BasisMap.add s (Coeff.add current_c c) acc

(* 乗法 *)
let mul (v1 : t) (v2 : t) : t =
  BasisMap.fold (fun s1 c1 acc1 ->
    BasisMap.fold (fun s2 c2 acc2 ->
      let (new_c, new_s) = mul_basis s1 s2 c1 c2 in
      add_term new_c new_s acc2
    ) v2 acc1
  ) v1 zero
```

これで加法群・乗法モノイドとしての演算がそろったので、いつものように加法群ファンクタ（`Add_group.Extend`）・乗法モノイドファンクタ（`Mul_monoid.Extend`）

で減法などを自動生成しておきます。あわせて、有理数を並列拡大の中に埋め込む関数も定義しておきましょう。

▲リスト 5.4　"q_rootp.ml"

```
include Add_group.Extend(struct
  type nonrec t = t
  let add, neg = add, neg
end)
include Mul_monoid.Extend(struct
  type nonrec t = t
  let one, mul = one, mul
end)

(* 有理数の埋め込み *)
let of_rational (q : Coeff.t) : t =
  if Coeff.equal q Coeff.zero then BasisMap.empty
  else BasisMap.singleton PrimeSet.empty q
```

有理数 q は、根号が何も掛かっていない空集合の基底に、係数 q を乗せるだけでこの体に埋め込めます。

実際に違いが現れるのは、平方根を計算する部分だけです。この関数は、「与えられた有理数が、そもそも有理数の範囲で平方根を持つかどうか」をまず判定する必要があります。そのための道具を、整数・有理数それぞれに用意しておきましょう。

まず整数の n 乗根です。n 乗して x になる整数を、二分探索で探すだけです。

▲リスト 5.5　"integer.ml"

```
(* n乗根 *)
let rec root (n : int) (x : t) : t option =
  if n < 0 then None
  else if equal x zero then Some zero
  else if compare x zero < 0 then
    if n mod 2 = 0 then None
    else
      match root n (neg x) with
      | Some k -> Some (neg k)
      | None -> None
  else
    let rec search low high =
      if compare low high > 0 then None
      else
        let mid = div_rem (add low high) 2 |> fst in
        let mid_pow = pow n mid in
        let cmp = compare mid_pow x in
        if cmp = 0 then Some mid
        else if cmp < 0 then search (add mid one) high
        else search low (sub mid one)
    in
    search one x
```

これを使って、有理数の k 乗根も定義できます。分子・分母それぞれの k 乗根が整数として求まれば、それが答えです。

▲リスト 5.6　"rational.ml"

```
(* k乗根 *)
let root (k : int) ((n, d) : t) : t option =
  if Integer.equal n Integer.zero then Some zero
  else
    let d_pow = Integer.pow (k - 1) d in
    let target = Integer.mul n d_pow in
    match Integer.root k target with
    | Some num -> Some (mul (num, d) one)
    | None -> None
```

分母 d をそのまま k 乗根に持ち込めないので、いったん次のように変形します。

$$\frac{n}{d} = \frac{n \cdot d^{k-1}}{d^k} \tag{5.10}$$

この右辺の分子が整数の k 乗根を持つかどうかに帰着させているわけです。ただし、こうして求まる分子・分母の組は、必ずしも既約分数になっているとは限りません（たとえば $1/4$ の平方根は $2/4$ という約分前の形で出てきます）。そこで、分数体ファンクタ（`Fraction.Make`）がもともと備えている約分の仕組みにそのまま乗せることで、既約な形に整えています。

これで、有理数が平方根を持つかどうかを判定する関数が書けます。ただし、開けない場合に残る整数 nd をそのまま根号の中に残してしまうと、たとえば $\sqrt{8}$ が $2\sqrt{2}$ まで簡約されずに残ってしまいます。そこで、開けない場合は nd をあらかじめ平方因子を持たない無平方数 r と、$r \cdot s^2 = nd$ を満たす s に分解しておきます。この分解には試し割りを使うので、整数を 1 つずつ増やしていくための後者関数（$n \mapsto n+1$）を整数モジュール（`Integer`）に追加しておきましょう。

▲リスト 5.7　"integer.ml"

```
(* 後者関数 *)
let succ (x: t): t =
  add x one
```

▲リスト 5.8　"rational.ml"

```
(* 整数mを、平方因子を持たない無平方数rと、r * s^2 = mを満たすsの組に分解する *)
let extract_square_factor (m : Integer.t) : Integer.t * Integer.t =
  let rec aux d remaining square_part =
    let d2 = Integer.mul d d in
    if Integer.compare d2 remaining > 0 then
      (remaining, square_part)
    else
      let (q, r) = Integer.div_rem remaining d2 in
      if Integer.equal r Integer.zero then
        aux d q (Integer.mul square_part d)
      else
        aux (Integer.add d Integer.one) remaining square_part
  in
  aux (Integer.succ Integer.one) m Integer.one
```

求めたいのは、

$$\mathrm{try_sqrt}(q) = \begin{cases} \mathrm{Ok}(r) & (r \in \mathbb{Q},\ r^2 = q \text{ を満たす } r \text{ が存在するとき}) \\ \mathrm{Error}(r,\ s/d) & (\text{存在しないとき。} q = n/d \text{ は既約分数、} nd = r \cdot s^2 \text{、} r \text{ は無平方}) \end{cases} \tag{5.11}$$

という関数です。開けない側の $(r, s/d)$ は、

$$\sqrt{q} = \frac{\sqrt{nd}}{d} = \frac{s\sqrt{r}}{d} \tag{5.12}$$

という関係を満たす組になっています。

▲リスト 5.9　"rational.ml"

```
(* 平方根が有理数として開けるか判定する。開ければその値を、開けなければ
   無平方数(下の体で添加する基底そのもの)と、その平方根の係数を返す *)
let try_sqrt (q : t) : (t, Integer.t * t) result =
  match root 2 q with
  | Some r -> Ok r
  | None ->
      let (num, den) = q in
      let (square_free, s) = extract_square_factor (Integer.mul num den) in
      Error (square_free, div (of_integer s) (of_integer den))
```

たとえば $q = 1/4$ のように有理数の範囲で開ける場合は開ける側に、$q = 1/3$ のように開けない場合は開けない側になります。後者では $1/3 = 3/9$ なので、根号の中に残る無平方数は 3 です。$q = 1/8$ のように $nd = 8$ が平方因子 4 を持つ場合は、無平方数 2 と係数 $1/4$ の組になり、

$$\sqrt{\frac{1}{8}} = \frac{1}{4}\sqrt{2} \tag{5.13}$$

という簡約された形が得られます。この開けない場合の値が、次に見る平方根を求める関数で使われます。求めたいのは、

$$\mathrm{of_rational_sqrt}(q) = \sqrt{q} \in K \qquad (q \geq 0) \tag{5.14}$$

という関数です。

ただし、`try_sqrt` が返す無平方数は、必ずしも素数そのものとは限りません。たとえば $15 = 3 \times 5$ のように、異なる素数の積であることもあります。この体の基底

は**素数の集合**であることを前提にしているので、無平方数をそのまま 1 つの元として基底に入れてしまうと、同じ実数の平方根が、計算の道筋によって「15」という基底と「3 と 5」という基底の両方で表現されてしまい、基底の一意性が崩れます。そこで、まず無平方数を素因数に分解しておきます。

▲リスト 5.10　"q_rootp.ml"

```
(* 無平方数を、素因数の集合に分解する *)
let prime_factors (n : Integer.t) : PrimeSet.t =
  let rec aux d remaining acc =
    let d2 = Integer.mul d d in
    if Integer.compare d2 remaining > 0 then
      PrimeSet.add remaining acc
    else
      let (q, r) = Integer.div_rem remaining d in
      if Integer.equal r Integer.zero then
        aux (Integer.succ d) q (PrimeSet.add d acc)
      else
        aux (Integer.succ d) remaining acc
  in
  aux (Integer.succ Integer.one) n PrimeSet.empty
```

▲リスト 5.11　"q_rootp.ml"

```
(** 平方根 *)
let of_rational_sqrt (q : Coeff.t) : t =
  if Coeff.equal q Coeff.zero then zero
  else
    match Rational.try_sqrt q with
    | Ok r -> of_rational r
    | Error (square_free, coeff) ->
        BasisMap.singleton (prime_factors square_free) coeff
```

try_sqrt が開ける場合はその有理数をそのまま返し、開けない場合は先ほどの無平方数と係数の組から平方根の項を組み立てます。この関数は q が負でないことを前提にしています。判別式が負になる場合の扱いは、この節の最後で改めて説明します。

残る等価判定・比較も一気に片付けておきます。

▲リスト 5.12　"q_rootp.ml"

```
(* 正規化：係数が0になった基底を取り除く *)
let normalize (v: t): t =
  BasisMap.filter (fun _ c -> not (Coeff.equal c Coeff.zero)) v

(* 等価判定 *)
let equal (x1: t) (x2: t): bool =
  BasisMap.equal Coeff.equal (normalize x1) (normalize x2)

(* 大小比較(自然な順序ではなく、符号の正規化のための便宜的な全順序) *)
let compare (x1: t) (x2: t): int =
  BasisMap.compare Coeff.compare (normalize x1) (normalize x2)
```

この比較は自然な大小関係ではなく、この先の章で並列拡大の体を係数とする有理関数をつくる際に分母の符号を正規化するために使う、便宜的な全順序です。今の時点では使い道がありません。

最後に残るのが、乗法の逆元です。

▲リスト 5.13　"q_rootp.ml"

```
(* 登場する素数全体 *)
let collect_primes (v : t) : PrimeSet.t =
  BasisMap.fold (fun s _ acc -> PrimeSet.union s acc) v PrimeSet.empty

(* 素数pの有無による分解 *)
let split_by_prime (p : Integer.t) (v : t) : t * t =
  BasisMap.fold (fun s c (a, b) ->
    if PrimeSet.mem p s then
      let s' = PrimeSet.remove p s in
      (a, add_term c s' b)
    else
      (add_term c s a, b)
  ) v (zero, zero)

(* 逆元 *)
let rec inv (v : t) : t =
  let v = normalize v in
  if BasisMap.is_empty v then
```

```
    failwith "Division by zero"
  else
    let primes = collect_primes v in
    if PrimeSet.is_empty primes then
      let c = BasisMap.find PrimeSet.empty v in
      BasisMap.singleton PrimeSet.empty (Coeff.inv c)
    else
      let p = PrimeSet.min_elt primes in
      let a, b = split_by_prime p v in
      let sqrt_p = BasisMap.singleton (PrimeSet.singleton p) Coeff.one in
      let conj = sub a (mul b sqrt_p) in
      let norm = mul v conj in
      mul conj (inv norm)

include Mul_group.Extend(struct
  type nonrec t = t
  let one, mul, inv = one, mul, inv
end)
```

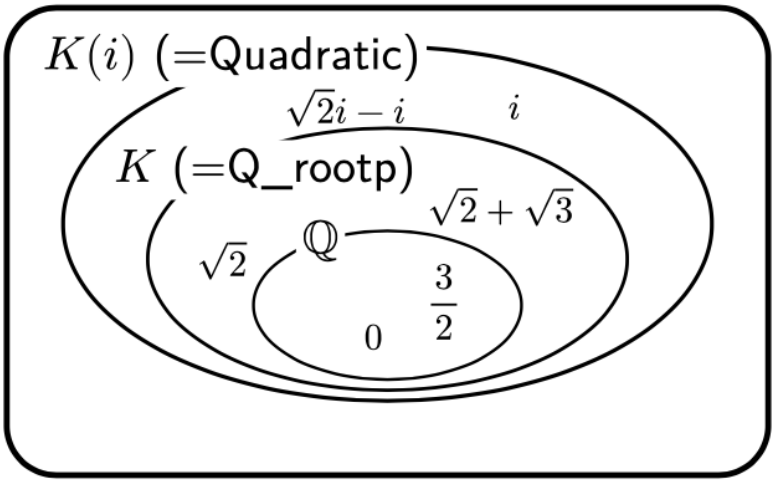
v に含まれるある素数 p に注目し、$v = a + b\sqrt{p}$（a は p を含まない部分、b は p の係数部分）の形に分解します。この共役 $a - b\sqrt{p}$ を掛けてノルム $a^2 - b^2 p$ を作ることで、素数を 1 つずつ消していく、というアルゴリズムです。ノルムを取ったあとにまだ他の素数が残っていれば、同じ操作を再帰的に繰り返します。

※実は、この基底の集合に -1 も紛れ込ませれば、虚数単位 i を「素数 -1 の平方根」として同じ枠組みで扱うこともできます。

$$\sqrt{-1} \cdot \sqrt{-1} = -1 \tag{5.15}$$

基底同士の積の規則は基底が素数であることを本質的には使っていないため、そのまま矛盾なく拡張できます。

複素構造ファンクタで虚数単位を添加する



▲図 5.6　今回の拡大の流れ

先ほどつくった二次方程式の解の拡大体（`Q_rootp`）は実数の範囲にとどまる体でした。判別式が負になる場合に備えて、虚数単位 i を添加した体も必要です。ここで、複素数の節でつくった複素構造ファンクタ（`Complexical.Make`）が再登場します。F がどんな体であっても、この複素構造ファンクタを適用するだけで「F に虚数単位を添加した体」を機械的に作れるのでした。F にこの拡大体を渡すだけで、2 次方程式の解全体を表現できる体が手に入ります。

複素構造ファンクタ（`Complexical.Make`）に渡すには、共役を持つ `Field` シグネチャを満たす必要があるのでした。二次方程式の解の拡大体（`Q_rootp`）はあくまで実数の範囲の体なので、共役をとっても自分自身のままです。

▲リスト 5.14　"q_rootp.ml"

```
(* 共役 *)
let conj (x: t): t = x
```

▲リスト 5.15　"quadratic.ml"

```
include Complexical.Make(Q_rootp)
```

驚くほどシンプルです。有理数から複素数をつくったときとまったく同じ形です。残るは、有理数の埋め込み・平方根・大小比較です。

▲リスト 5.16　"quadratic.ml"

```
(* 大小比較(Fraction.Makeに渡すための便宜的な全順序) *)
let compare ((a1, b1): t) ((a2, b2): t): int =
  let c = Q_rootp.compare a1 a2 in
  if c <> 0 then c else Q_rootp.compare b1 b2

(* 有理数の埋め込み *)
let of_rational (q : Rational.t) : t =
  (Q_rootp.of_rational q, Q_rootp.zero)

(* 平方根(q < 0なら虚部に、q >= 0なら実部にQ_rootpの平方根を埋め込む) *)
let of_rational_sqrt (q : Rational.t) : t =
  if Rational.compare q Rational.zero >= 0 then
    (Q_rootp.of_rational_sqrt q, Q_rootp.zero)
  else
    (Q_rootp.zero, Q_rootp.of_rational_sqrt (Rational.neg q))
```

複素数は実部・虚部の組 (a, b) として $a + bi$ を表すのでした。ここで、先送りにしていた符号による場合分けをようやく行います。q が 0 以上ならその平方根を実部に、負なら絶対値の平方根を虚部に埋め込みます。実数の範囲にとどまる並列拡大の平方根自体は符号を判定していなかったので、判別式の符号による場合分けは、結局この 1 箇所に集約されました。

solve 関数で 2 次方程式を解く

これで、判別式の符号にかかわらず 2 次方程式を解く準備が整いました。

▲リスト 5.17　"quadratic.ml"

```
(* 2次方程式の求解 *)
let solve (a : Rational.t) (b : Rational.t) (c : Rational.t) : t * t =
  let disc =
    Rational.sub (Rational.mul b b)
      (Rational.mul (Rational.of_integer 4) (Rational.mul a c))
  in
  let sqrt_disc = of_rational_sqrt disc in
  let neg_b = of_rational (Rational.neg b) in
  let two_a = of_rational (Rational.mul (Rational.of_integer 2) a) in
  (div (add neg_b sqrt_disc) two_a, div (sub neg_b sqrt_disc) two_a)
```

実際に解いてみましょう。もとの方程式に解を代入して 0 になるかどうかの検算もあわせて行うため、得られた二次方程式の解拡大モジュール（`Quadratic`）の上に多項式モジュールもつくっておきます。多項式ファンクタ（`Polynomial.Make`）に `Quadratic` を渡すだけです。

▲リスト 5.18　"quadratic_poly.ml"

```
include Polynomial.Make(Quadratic)
```

▲リスト 5.19　"quadratic_test.ml"

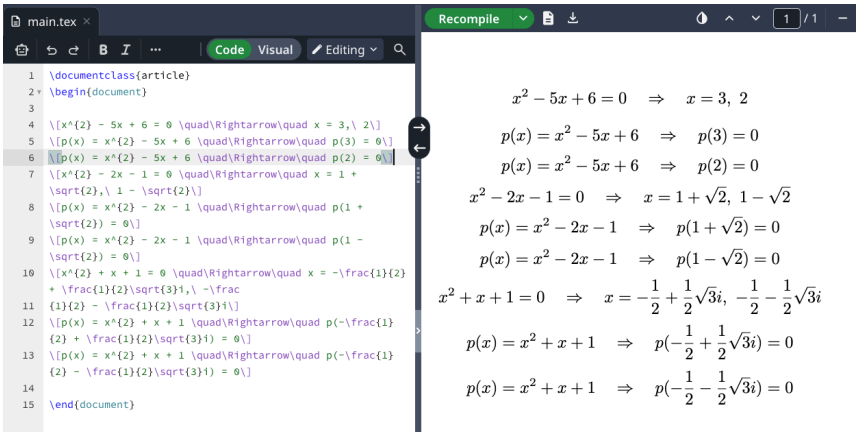
```
module P = Polynomial.Make(Quadratic)
module Quadratic_expr = Expr.Quadratic.Make (Q_rootp)
module Quadratic_poly_expr = Expr.Quadratic_poly.Make (Q_rootp)

let run_example (a, b, c) =
  let poly : Rational.t list = [c; b; a] in
  let solve (_ : Rational.t list) =
    let (x1, x2) = Quadratic.solve a b c in
    [x1; x2]
  in
  Expr.put (Expr.solve_to_string
    (module Expr.Q_poly) (module Quadratic_expr) solve poly);
  let (x1, x2) = Quadratic.solve a b c in
  let check_poly : Quadratic_poly.t =
    [Quadratic.of_rational c;
     Quadratic.of_rational b;
     Quadratic.of_rational a]
  in
  Expr.put (Expr.poly_eval_to_string
    (module Quadratic_poly_expr) (module Quadratic_expr)
    P.eval check_poly x1);
  Expr.put (Expr.poly_eval_to_string
    (module Quadratic_poly_expr) (module Quadratic_expr)
    P.eval check_poly x2)

let () =
  run_example ((1, 1), (-5, 1), (6, 1));
  run_example ((1, 1), (-2, 1), (-1, 1));
  run_example ((1, 1), (1, 1), (1, 1))
```

$x^2 - 5x + 6 = 0$、$x^2 - 2x - 1 = 0$、$x^2 + x + 1 = 0$ の 3 つを求解関数に与えて、実行してみましょう。

```
ocamlopt add_monoid.ml add_group.ml mul_monoid.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml vector_space.ml polynomial.ml complexical.ml q_rootp.ml
quadratic.ml quadratic_poly.ml to_tex.cmxa quadratic_test.ml && ./a.out
>>
\[x^{2} - 5x + 6 = 0 \quad\Rightarrow\quad x = 3,\ 2\]
\[p(x) = x^{2} - 5x + 6 \quad\Rightarrow\quad p(3) = 0\]
\[p(x) = x^{2} - 5x + 6 \quad\Rightarrow\quad p(2) = 0\]
\[x^{2} - 2x - 1 = 0 \quad\Rightarrow\quad x = 1 + \sqrt{2},\ 1 - \sqrt{2}\]
\[p(x) = x^{2} - 2x - 1 \quad\Rightarrow\quad p(1 + \sqrt{2}) = 0\]
\[p(x) = x^{2} - 2x - 1 \quad\Rightarrow\quad p(1 - \sqrt{2}) = 0\]
\[x^{2} + x + 1 = 0 \quad\Rightarrow\quad x = -\frac{1}{2} + \frac{1}{2}\sqrt{3}i,\ -\frac
{1}{2} - \frac{1}{2}\sqrt{3}i\]
\[p(x) = x^{2} + x + 1 \quad\Rightarrow\quad p(-\frac{1}{2} + \frac{1}{2}\sqrt{3}i) = 0\]
\[p(x) = x^{2} + x + 1 \quad\Rightarrow\quad p(-\frac{1}{2} - \frac{1}{2}\sqrt{3}i) = 0\]
```



▲図 5.7　ブラウザ上でチェック

1 つ目は判別式が完全平方数になる有理数解、2 つ目は実の無理数解、3 つ目は複素数解です。3 つ目のように判別式が負になる場合でも、特別な場合分けをすることなく、同じ求解関数が i を含む解をそのまま返してくれます。そして 3 つの方程式すべてで、2 つの解を代入した結果はきちんと 0 になっています。2 次方程式の解空間は、これで完全に閉じました。動的にモジュールを組み立てるような仕組みは、ここでは一切必要ありません。

次の章では、微分・積分に取り組みます。

■

6

第 6 章

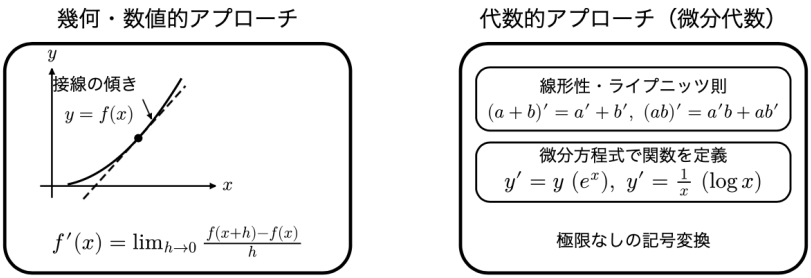
微分・積分

第 6 章

微分・積分

ここまでの章では、方程式を解くという 1 つの問題に取り組んできました。この章からは主題を変え、微分・積分に取り組みます。

微分・積分と聞くと、極限を使った定義（高校や大学で習う、あの一連の計算）を思い浮かべるかもしれません。実際のコンピュータでの計算では、数値微分・数値積分（有限の差分や区分求積によって極限を近似する方法）や、機械学習で広く使われる自動微分のように、むしろ数値的・近似的な手法のほうが一般的です。本書がこれから扱うのは、そのどれとも違う、式を式のまま変換する方法です。x^2 を微分すると機械的に $2x$ になる、$\sin x$ を微分すると機械的に $\cos x$ になる、という**式から式への変換規則**を、そのまま実行するのです。これを**シンボリック微分**（*Symbolic Differentiation*）と呼びます。同様に、不定積分を式のまま求める処理を**シンボリック積分**（*Symbolic Integration*）と呼びます。WolframAlpha や Mathematica、SymPy のような数式処理システムが行っているのは、まさにこれです。



▲図 6.1　数値微分とシンボリック微分の違い

一般的なシンボリック微分の実装は、式を**抽象構文木**（*Abstract Syntax Tree*）として持ち、木を辿りながらノードの種類ごとに定められた規則を当てはめて、新しい木（＝新しい式）を組み立てていく、という形をとります。本書のやり方はこれとは違い、式を木として持ち回るのではなく、整数・有理数・多項式・拡大体といった、ここまで作ってきた代数そのものに微分という演算を追加し、その演算を実行させることで答えを得ます。方程式を解いてきたときと同じ、本書が最初から選んできたやり方を、微積分でも続けているだけです。

実は、微分は「接線の傾き」として定義しなくても、代数的に定義できます。加法についての線形性

$$(a+b)' = a' + b' \tag{6.1}$$

と、積の微分法そのものであるライプニッツ則

$$(a \cdot b)' = a' \cdot b + a \cdot b' \tag{6.2}$$

の 2 つさえ満たせば、その演算は微分と呼んでよい、という立場です。環や体にこの演算を 1 つ備えたものを**微分代数**（*Differential Algebra*）と呼びます。

ただし、この 2 つの公理だけでは、個々の関数の微分がいくつになるかまでは決まりません。線形性とライプニッツ則は、和や積の微分を、その部品の微分から組み立てる規則にすぎないからです。たとえば x という元そのものの微分は、この 2 つの公理からは導けず、$x' = 1$ であることを別途決めておく必要があります。ただし、この 1 つさえ決めてしまえば、あとはライプニッツ則を繰り返し適用するだけで、

$$(x^2)' = x' \cdot x + x \cdot x' = x + x = 2x, \qquad (x^n)' = nx^{n-1} \tag{6.3}$$

のように、見慣れたべき乗則まで自動的に導けます。

一方、e^x や $\log x$ のように、x の四則演算だけでは組み立てられない新しい関数の微分は、ライプニッツ則からは自動的に出てきません。そこで、これらの関数自体を、微分方程式によって定義してしまいます。たとえば e^x は「自分自身が自分の微分になっている元」、つまり

$$y' = y \tag{6.4}$$

という微分方程式によって定義されます。同様に $\log x$ は

$$y' = \frac{1}{x} \tag{6.5}$$

によって定義されます。微積分の教科書でおなじみの初期値（$y(0) = 1$ や $y(1) = 0$）は、この微分方程式を満たす関数を実数値関数としてただ 1 つに選び出すための条件で、ここでは必要ありません。代数的にどう微分されるかだけが問題であり、$\sqrt{2}$ を添加するときに「どちらの根か」を気にしなかったのと同じ立場です。e^x や $\log x$ を、指数や対数という元々の意味からではなく、こうした微分方程式を満たす元として後から追加する、というのが微分代数の考え方です。

これは、本書がここまで一貫して選んできた道（数値的な近似ではなく、代数的に厳密な値として答えを求める道）と相性のよい話題です。群・環・体・拡大体を、公理を満たす演算の組として定義してきたのと同じやり方で、微分もまた、線形性とライプニッツ則という公理を満たす演算として、既存の代数に追加できるわけです。

ただし、微分と積分は同じ難しさではありません。微分はどんな初等関数に対しても必ず機械的に計算できますが、積分は必ずしもそうではありません。たとえば e^{-x^2} の不定積分は、初等関数の範囲には存在しないことが知られています。この非対称性は、この章の後半で積分を実装する際に、微分よりずっと立ち入った設計が必要になる理由でもあります。

6.1 微分を導入する

ある形式的な変数 x についての微分だと考えてください。整数や有理数のような、x に依存しない「ただの数」の微分は常に 0 になりますが、多項式のように x そのものを表現できる代数では、微分は非自明な計算になります。

すでにつくったモジュールを 1 つずつ見ながら、微分を追加していきます。

6.1.1 Integer: ただの定数

整数は、x に依存しない定数です。定数の微分は常に 0 なので、実装もそのままです。

▲リスト 6.1 "integer.ml"

```
(* 微分 *)
let differentiate (_: t): t = zero
```

6.1.2 Fraction.Make: 商の微分法

有理数は、整数を環とする分数体でした。分数 n/d の微分には、**商の微分法**（*Quotient Rule*）が必要です。

$$\left(\frac{n}{d}\right)' = \frac{n'd - nd'}{d^2} \tag{6.6}$$

これを実装するには、もとになる環にも微分が定義されていることを要求する必要があります。あわせて、出力の側にも微分を追加しておきましょう。

▲リスト 6.2 "fraction.ml"

```
module type Ring = sig
  include Ring.S
  val compare: t -> t -> int
  val gcd: t -> t -> t
  val div_rem: t -> t -> t * t
  val differentiate: t -> t
end

module type FUNCTOR_TYPE = functor (R: Ring) ->
  (sig
     include Field.S
     val differentiate: t -> t
     val compare: t -> t -> int
   end with type t = R.t * R.t)
```

▲リスト 6.3 "fraction.ml"

```
(* 微分 *)
let differentiate ((n, d): t): t =
  let n' = R.differentiate n in
  let d' = R.differentiate d in
  (R.sub (R.mul n' d) (R.mul n d'), R.mul d d) |> normalize
```

ここがこの節でいちばん重要な設計です。分子 n・分母 d をどう微分するかは、もとになる環に任せてしまい、分数体を組み立てる仕組み自体は、商の微分法という**組み立て方**だけを知っていればよいのです。

もとになる環が整数（つまり有理数をつくる場合）のときは、整数の微分が常に 0 なので、公式に代入すると $n' = 0, d' = 0$ となり、結果は自動的に 0 に潰れます。つまり、有理数のために新しいコードを **1 行も書くことなく**、整数の微分規則だけから有理数の微分が自動的に正しく求まるのです。

6.1.3 Complexical.Make: 実部と虚部それぞれを微分する

$a + bi$ の微分は、積の微分法を必要としません。a と b は互いに独立な成分の組で、i 自体は x に依存しない定数だからです。

$$(a + bi)' = a' + b'i \tag{6.7}$$

ここでも、入力・出力どちらの `Field` シグネチャにも微分を追加しておきます。

▲リスト 6.4　"complexical.ml"

```
module type Field = sig
  include Field.S
  val conj: t -> t
  val differentiate: t -> t
end

module type FUNCTOR_TYPE =
  functor (F: Field) -> (Field with type t = F.t * F.t)
```

▲リスト 6.5　"complexical.ml"

```
(* 微分 *)
let differentiate ((a, b): t): t = (F.differentiate a, F.differentiate b)
```

これも複素構造ファンクタ（`Complexical.Make`）にこの 1 行を足すだけで、複素数（`Complex`）はもちろん、`Complexical.Make(Complex)` である四元数（`Quaternion`）、そのまた上の八元数（`Octonion`）・十六元数（`Hexadecanion`）まで、**ファンクタの合成を通じて自動的に**微分が使えるようになります。

6.1.4 Polynomial.Make: 積の微分法とべき乗則

多項式 $p(x) = \sum_i c_i x^i$ の各項 $c_i x^i$ は、係数 c_i と単項式 x^i の**積**です。係数 c_i 自身が x に依存しうるので、一般には積の微分法が必要です。

ただし、係数 c_i を微分するには、係数のモジュール（C）自身に微分が定義されている必要があります。そこで、多項式ファンクタ（Polynomial.Make）が受け取る係数の条件を、これまでの体（Field.S）から一段階強め、返す側の多項式にも微分を追加しておきます。

▲リスト 6.6　"polynomial.ml"

```
module type Coeff = sig
  include Field.S
  val differentiate: t -> t
end

module type S = sig
  type coeff
  type t = coeff list
  val zero: t
  val one: t
  val normalize: t -> t
  val equal: t -> t -> bool
  val degree: t -> int
  val mul: t -> t -> t
  val add: t -> t -> t
  val neg: t -> t
  val eval: t -> coeff -> coeff
  val div_rem: t -> t -> t * t
  val pow: int -> t -> t
  val sub: t -> t -> t
  val gcd: t -> t -> t
  val ext_gcd: t -> t -> t * t * t
  val leading_coeff: t -> coeff
  val div_scalar: coeff -> t -> t
  val mul_scalar: coeff -> t -> t
  val differentiate: t -> t
end

module type FUNCTOR_TYPE = functor (C: Coeff) -> (S with type coeff = C.t)
```

係数の条件が Field.S から Coeff に変わったので、ファンクタ本体（Make）の引数の型注釈もあわせて書き換える必要があります。

▲リスト 6.7　"polynomial.ml"

```
module Make: FUNCTOR_TYPE = functor (C: Coeff) -> struct
```

次の微分規則を実装します。

$$(c_i x^i)' = c_i' x^i + c_i \cdot i x^{i-1} \tag{6.8}$$

▲リスト 6.8　"polynomial.ml"

```
(* 整数から係数への変換 *)
let rec int_to_coeff (n: int): C.t =
  if n <= 0 then C.zero else C.add C.one (int_to_coeff (n - 1))
```

```
(* 微分 *)
let differentiate (p: t): t =
  let coeff_deriv = List.map C.differentiate p in
  let power_deriv =
    match p with
    | [] -> []
    | _ :: rest -> List.mapi (fun i c -> C.mul c (int_to_coeff (i + 1))) rest
  in
  add coeff_deriv power_deriv
```

先ほどの式の前半（係数 c_i を微分して x^i に掛けた部分）と、後半の見慣れた**べき乗則**（*Power Rule*）の部分を、それぞれ計算してから足し合わせています。

係数の代数が有理数のように、微分すると常に 0 を返す代数であれば、前半の係数側の項は消えて、見慣れたべき乗則だけが残ります。実際に $1 + 2x + 3x^2$ を微分すると、

$$(1 + 2x + 3x^2)' = 2 + 6x \tag{6.9}$$

という、正しい結果が得られます。

6.1.5 Field_extension.Make: 微分を上書きする

最後は要注意です。拡大体ファンクタ（`Field_extension.Make`）は、もとになった多項式モジュールの演算をまるごと引き継ぐようにつくられていたので、何もしなければ、たった今つくった多項式の微分（べき乗則）をそのまま受け継いでしまいます。しかし、これは間違いです。

$K(\sqrt{2})$ のようなこの体の元は、内部的には多項式（$a + bx$ の形）として表現されていますが、意味としては $a + b\sqrt{2}$ という **1 つの値**です。ここでの x は $\sqrt{2}$ という特定の値を指す記号であって、動かせる変数ではありません。継承した微分のうち、べき乗則の部分（x を動かして微分する部分）をそのまま使うと、まるで $\sqrt{2}$ が変数であるかのように微分してしまい、意味のない値が返ってきます。

とはいえ、係数 a, b の微分まで捨ててしまってよいわけではありません。a, b がそれぞれ何らかの外部の変数に依存しているなら、

$$(a + b\sqrt{2})' = a' + \sqrt{2}\,b' \tag{6.10}$$

のように、$\sqrt{2}$ 自体は動かさずに、各係数だけを微分する必要があります。そのためには、既約多項式（`IrreduciblePoly`）の係数（`C`）も、多項式のときと同じく体（`Field.S`）から微分可能な係数（`Polynomial.Coeff`）へ格上げし、あわせて出力の条件にも微分を持たせておく必要があります。

▲リスト 6.9　"field_extension.ml"

```
module type IrreduciblePoly = sig
  module C : Polynomial.Coeff
  module P : Polynomial.S with type coeff = C.t
  val poly : P.t
end

module type FUNCTOR_TYPE =
  functor (I: IrreduciblePoly) ->
    (Polynomial.Coeff with type t = I.P.t)
```

▲リスト 6.10　"field_extension.ml"

```
(* 微分 *)
let differentiate (p: t): t =
  List.map I.C.differentiate p
```

`include` したあとに同じ名前の定義を書くと後勝ちで上書きされる、ルールがここでそのまま生きています。

6.1.6　テスト

ここまで追加してきた `differentiate` が、それぞれの代数で実際に正しく動くことを確認しておきましょう。

▲リスト 6.11　"differentiate_test.ml"

```
let () =
  let diff_int = Expr.postfix_unop "'" Integer.differentiate in
  Expr.put (Expr.unop_to_string (module Expr.Integer) diff_int 5);

  let diff_rat = Expr.postfix_unop "'" Rational.differentiate in
  let half : Rational.t = Rational.div (Rational.of_integer 1) (Rational.of_integer 2) in
  Expr.put (Expr.unop_to_string (module Expr.Rational) diff_rat half);

  let diff_complex = Expr.postfix_unop "'" Complex.differentiate in
  let c : Complex.t = (Rational.of_integer 3, Rational.of_integer 4) in
  Expr.put (Expr.unop_to_string (module Expr.Complex) diff_complex c);

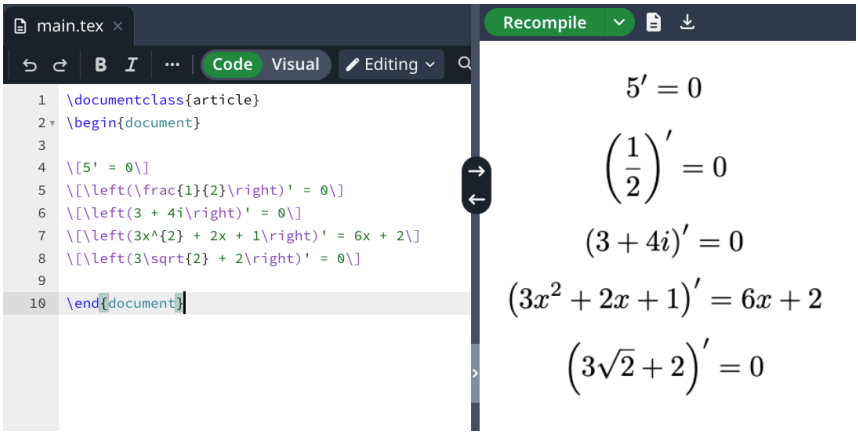
  let module Poly = Polynomial.Make (Rational) in
  let diff_poly = Expr.postfix_unop "'" Poly.differentiate in
  let p : Poly.t =
    [Rational.of_integer 1; Rational.of_integer 2; Rational.of_integer 3]
  in
  Expr.put (Expr.unop_to_string (module Expr.Q_poly) diff_poly p);

  let diff_k = Expr.postfix_unop "'" Root2_field.K.differentiate in
  let k : Root2_field.K.t =
    [Rational.of_integer 2; Rational.of_integer 3]
  in
  Expr.put (Expr.unop_to_string (module Expr.Root2_field) diff_k k)
```

最後の $K(\sqrt{2})$ の例に注目してください。$2+3\sqrt{2}$ は x に依存しない、ただの定数です。もし拡大体ファンクタ（`Field_extension.Make`）に先ほどの上書きがなく、

多項式のべき乗則をそのまま受け継いでいたら、内部表現の $2+3X$ を「X についての多項式」として微分してしまい、3 という誤った値が返ってきます。

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml complexical.ml complex.ml vector_space.ml polynomial.ml
field_extension.ml root2_field.ml to_tex.cmxa differentiate_test.ml && ./a.out
>>
\[5' = 0\]
\[\left(\frac{1}{2}\right)' = 0\]
\[\left(3 + 4i\right)' = 0\]
\[\left(3x^{2} + 2x + 1\right)' = 6x + 2\]
\[\left(3\sqrt{2} + 2\right)' = 0\]
```

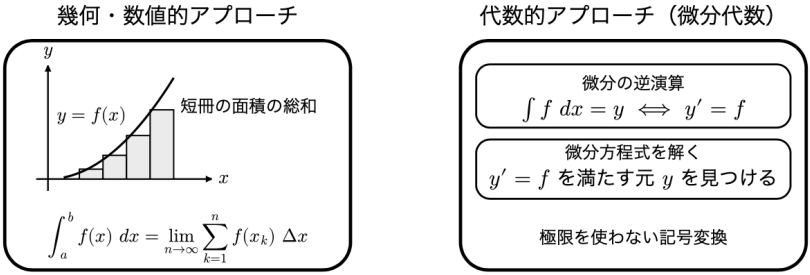


▲図 6.2　ブラウザ上でチェック

定数はすべて 0 に、多項式は見慣れたべき乗則どおり $6x+2$ に、そして $K(\sqrt{2})$ の定数はきちんと 0 になりました。上書きが効いていることが、これで確認できました。

6.2　積分を導入する

微分の逆演算である積分を実装してみましょう。ある関数 $f(x)$ に対して $F'(x) = f(x)$ を満たす $F(x)$ を求めるのが不定積分です。微分のときと同様、ここでも数値積分（区分求積法のように、面積を近似計算する方法）ではなく、不定積分を式のまま求めるシンボリック積分を実装します。



▲図 6.3　数値積分とシンボリック積分の違い

6.2.1　微分と積分の違い

ここまで見てきた微分には、共通する性質がありました。どの代数でも、微分した結果の型は、もとの型と**同じ**だったということです。多項式を微分しても多項式、有理数を微分しても有理数のままで、微分は代数の中で完全に閉じた演算だったのです。

積分はそうはいきません。試しに、有理関数 $1/x$ を積分してみましょう。答えは $\log x$ です。しかし $\log x$ は有理関数ではなく、**有理関数を積分すると、有理関数の外に出てしまう**のです。積分を実装するとは、まず積分した結果が住むことになる、もう少し広い代数を設計することから始まる、という点で、微分とは根本的に違う仕事になります。

6.2.2　積分のために設計する代数

具体的な積分の答えをいくつか見ながら、その答えが住むことになる世界を考えていきましょう。

有理関数 $1/x$ を積分すると、答えは有理関数の外に出て $\log x$ になるのでした。分母が複数の 1 次式に分解できる有理関数、たとえば $1/(x^2-1)$ なら、部分分数分解によって

$$\int \frac{1}{x^2-1}\,dx = -\frac{1}{2}\log(x+1) + \frac{1}{2}\log(x-1) \tag{6.11}$$

のように、log どうしの 1 次結合になります。log が 1 つでは済まず、いくつも並列に必要になるわけです。

多項式×指数関数の積分では、

$$\int xe^x\,dx = (x-1)e^x \tag{6.12}$$

のように、今度は e^x に多項式 $x-1$ を掛けた形が答えに登場します。同じように $x\log x$ を積分すると

$$\int x\log x\,dx = \frac{x^2}{2}\log x - \frac{x^2}{4} \tag{6.13}$$

となり、今度は $\log x$ に多項式 $x^2/2$ を掛けた項が現れます。
つまり、積分の答えとして登場しうる部品は、

- 有理関数
- 多項式を係数とする $\log(x-\alpha)$ の項
- 多項式を係数とする e^{ax} の項

の 3 種類に整理できそうです。
実際、有理関数の不定積分に限って言えば、**リウヴィルの定理**（*Liouville's Theorem*）により、答えは必ず

$$(\text{有理関数}) + c_1\log(x-\alpha_1) + c_2\log(x-\alpha_2) + \cdots \tag{6.14}$$

という形に書けることが保証されています。log の中身が一般の有理関数ではなく、$x-\alpha$ というただの 1 次式で済むのがポイントです。分母は、係数体を複素数の範囲まで十分広げてしまえば必ず 1 次式の積にまで分解しきれるので、log の中身もそれ以上複雑にはならないのです。
そこで 2 次方程式の節で素数の平方根 $\sqrt{p}$ 達を並列な基底として添加したのと同じ発想で、有理関数の体 $K(x)$ に $\log(x-\alpha_1)$, $\log(x-\alpha_2)$, ……を、加法とスカラー倍だけで定義されるベクトル空間の独立な基底として並列に添加します。ただし 2 次方程式の時のように、ベクトル空間をつくった後で乗法や除法まで考えるようなことはしません。つまり、$(\log(x-\alpha_1))^2$ のようなものは、この代数には存在しません。ここでの

$$K(x)(\log(x-\alpha_1), \log(x-\alpha_2), \ldots) \tag{6.15}$$

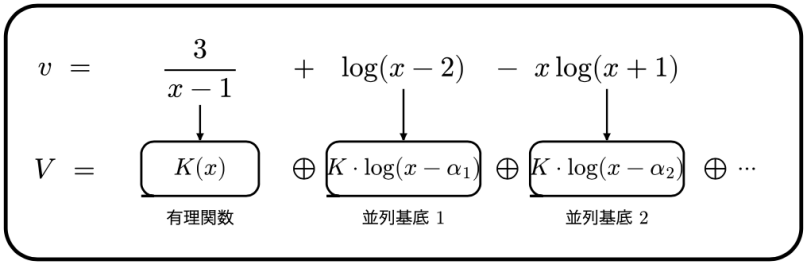
は、$K(x)$ を基点に、$\log(x-\alpha_1)$, $\log(x-\alpha_2)$, ……を独立な基底として並列に添加してできる、加法群とスカラー倍だけを備えたベクトル空間を表します。基底ごとの係数は、定数ではなく多項式 $K[x]$ にしておきます。理由は、この先 $x\log x$ のような多項式× log の積分も扱いたいからです。展開すると

$$V = K(x) \ \oplus \ K[x]\cdot\log(x-\alpha_1) \ \oplus \ K[x]\cdot\log(x-\alpha_2) \ \oplus \ \cdots \tag{6.16}$$

です。ここで使った直和 $\oplus$ という記号は、それぞれの基底が重複や過不足なく独立に効いていること、つまり V の元が「有理関数の部分」「$\log(x-\alpha_1)$ の係数」「$\log(x-\alpha_2)$ の係数」……という各部分に、**ただ 1 通りの方法でしか分解できない**ことを表しています。たとえば V の元は、

$$\frac{3}{x-1} + \log(x-2) - x\log(x+1) \tag{6.17}$$

のように、有理関数の部分（$K(x)$ の部分）に log の項がいくつか並列に足された形をしています。



▲図 6.4　対数拡大

ここで注意してほしいのは、V が積分について閉じているわけではない、という点です。たとえば $\log(x)/x$ を積分すると $(\log(x))^2/2$ となり、log どうしの積を持つため V の外に出てしまいます。しかし対象を V の中に限定すれば実際に積分できますし、V は**微分については完全に閉じています**。基底ごとの係数を $u(x)$ とすると、

$$(u(x)\log(x-\alpha))' = u'(x)\log(x-\alpha) + \frac{u(x)}{x-\alpha} \tag{6.18}$$

となり、右辺の第 1 項はまだ $\log(x-\alpha)$ の項（係数 $u'(x)$ も多項式のまま）、第 2 項は有理関数として、どちらも V の中に収まります。リウヴィルの定理は、この微分について閉じた V の中に積分の答えが住んでいることを保証してくれるので、「微分すると元の有理関数 f に戻るような G」、つまり $G' = f$ を満たす G を、V の中だけで探せばよいのです。

指数関数のケースも同様です。今度は $\log(x-\alpha_i)$ 達に加えて、$e^{a_1 x}$, $e^{a_2 x}$, …… も多項式（$K[x]$）倍の自由度を持つ基底として並列に添加した

$$K(x)(e^{a_1 x}, \log(x-\alpha_1), e^{a_2 x}, \log(x-\alpha_2), \ldots) \tag{6.19}$$

というベクトル空間

$$V' = K(x) \oplus K[x]\cdot e^{a_1 x} \oplus K[x]\cdot\log(x-\alpha_1) \oplus K[x]\cdot e^{a_2 x} \oplus K[x]\cdot\log(x-\alpha_2) \oplus \cdots \tag{6.20}$$

を考えます。基底ごとの係数を $u(x)$ とすると、

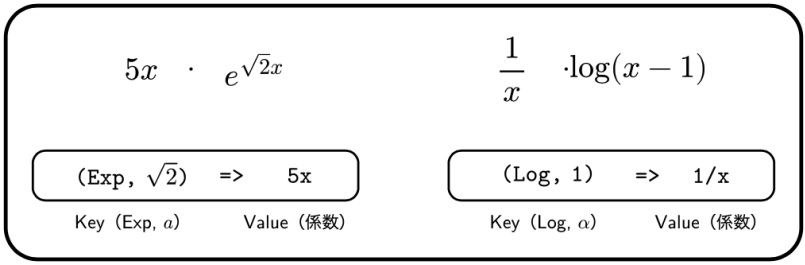
$$(u(x)e^{ax})' = (u'(x) + au(x))e^{ax} \tag{6.21}$$

となり、微分した結果も e^{ax} の項（係数 $u'(x) + au(x)$ も多項式のまま）に収まるので、こちらも微分について閉じています。V' の元は、

$$\frac{3}{x-1} + 2\log(x-2) + 5x\cdot e^{2x} \tag{6.22}$$

のように、log の項に加えて指数関数の項も並列に足された形をしています。

これをデータとして表すなら、「指数関数か対数関数か」と「添加した元 a」の組をキー、係数となる有理関数を値とする辞書型のデータ構造が使えそうです。2 次方程式の節で、素数の集合をキーにした辞書型のデータ構造を使ったのと、まったく同じ発想です。



▲図 6.5　辞書型データのイメージ

6.2.3　三角関数

初等関数のもう 1 つの代表格である三角関数 ($\sin x$, $\cos x$) は、V' の基底に加わっていません。三角関数も微分方程式で定義できる関数ですが、オイラーの公式

$$e^{ix} = \cos x + i \sin x \tag{6.23}$$

が示すとおり、複素数の範囲まで許せば、三角関数は e^x を使ってすでに表現できてしまうからです。ここで三角関数まで独立な基底として追加すると、$\cos x$ という同じ値が「三角関数の基底そのもの」と「e^{ix} と e^{-ix} の組み合わせ」という 2 通りの方法で作れてしまいます。これは、拡大体の章で確認した一意性の問題（$\sqrt{6}$ を独立な基底にすると、$\sqrt{2} \cdot \sqrt{3}$ という別の作り方と重複してしまう、という問題）とまったく同じ構造です。e^x だけを基底にしておけば、この重複は最初から起こりません。

6.2.4　代数拡大と超越拡大

$\log(x-a)$ や e^{ax} のように、$K(x)$ に対して超越的な元を添加する拡大のことを、**超越拡大**（*Transcendental Extension*）と呼びます。これまで見てきた $K(\sqrt{2})$ のように、多項式の根を添加する拡大は**代数拡大**（*Algebraic Extension*）と呼ばれ、超越拡大はこれと対照的な呼び方です。

この超越性には、もう 1 つ嬉しい帰結があります。異なる a に対する e^{ax} どうし、$\log(x-a)$ どうしも、互いに $K(x)$ 係数の 1 次関係式を持ちません。だからこそ、体の拡大の章で素数の集合を基底に $\sqrt{p}$ を並列拡大したときのように線形独立性を 1 つ 1 つ確認する必要はなく、「e^{ax}」または「$\log(x-a)$」を基底として、そのまま並列

拡大できるのです。

6.2.5 答えの型をつくる

V' の定義を、そのままコードにしてみましょう。まず、有理関数の体 $K(x)$ が必要です。K を二次方程式の解拡大モジュール（Quadratic）として、係数体 K 上の多項式環をつくります。Quadratic は複素構造ファンクタ（Complexical.Make）で二次方程式の解の拡大体（Q_rootp）を包んだものでした。整数や有理数と同じく、x に依存しない定数なので、二次方程式の解の拡大体（Q_rootp）にも微分は常に 0 として追加しておきます。

▲リスト 6.12　"q_rootp.ml"

```
(* 微分 *)
let differentiate (_: t): t = zero
```

▲リスト 6.13　"quadratic_poly.ml"

```
include Polynomial.Make(Quadratic)

(* 大小比較(Fraction.Makeに渡すための便宜的な全順序) *)
let rec compare (x1: t) (x2: t): int =
  match x1, x2 with
  | [], [] -> 0
  | [], _ -> -1
  | _, [] -> 1
  | c1 :: r1, c2 :: r2 ->
    let c = Quadratic.compare c1 c2 in
    if c <> 0 then c else compare r1 r2
```

この比較演算に、大小そのものの数学的な意味はありません。分数体ファンクタ（Fraction.Make）が分母の符号を正規化するために要求している性質を満たすだけの、便宜的な全順序です（2 次方程式の章で、二次方程式の解拡大モジュール（Quadratic）自身にも、実部・虚部それぞれの比較を使った同様の比較演算をすでに追加してありました）。

多項式環が手に入れば、有理関数体 $K(x)$ は、有理数を整数の分数体としてつくったのとまったく同じ要領で手に入ります。

▲リスト 6.14　"elementary.ml"

```
module QF = Fraction.Make(Quadratic_poly)
```

これで、$\log(x-a)$ や e^{ax} を基底とする Map の土台が用意できました。

▲リスト 6.15　"elementary.ml"

```
module Basis = struct
  type t = Exp of Quadratic.t | Log of Quadratic.t
  let compare (x1: t) (x2: t): int =
    match x1, x2 with
    | Exp a, Exp b -> Quadratic.compare a b
    | Log a, Log b -> Quadratic.compare a b
    | Exp _, Log _ -> -1
    | Log _, Exp _ -> 1
end
module BasisMap = Map.Make(Basis)

type t = QF.t BasisMap.t
```

$e^{0 \cdot x} = 1$ なので、指数が 0 の基底に載った値は、指数関数の飾りが取れた、ただの有理関数 $K(x)$ そのものを表します。V' の「有理関数の部分」と「log の部分」を別々のフィールドに分けて持つ必要はなく、この Map の 1 つの基底として自然に同居させられるわけです。

この型の上に、まず値を出し入れするための土台を用意しておきましょう。

▲リスト 6.16　"elementary.ml"

```
let zero: t = BasisMap.empty

(* 多項式pを分子、1を分母とする有理関数 *)
let of_poly (p: Quadratic_poly.t): QF.t = (p, Quadratic_poly.one)

(* Quadratic.t の定数を有理関数として埋め込む *)
let of_scalar (q: Quadratic.t): QF.t =
  if Quadratic.equal q Quadratic.zero then QF.zero
  else of_poly [q]

(* x - a を表す多項式 *)
let linear (a: Quadratic.t): Quadratic_poly.t =
  [Quadratic.neg a; Quadratic.one]

(* 基底への値の加算 *)
let add_term (key: Basis.t) (v: QF.t) (acc: t): t =
  let current =
    BasisMap.find_opt key acc |> Option.value ~default:QF.zero
  in
  BasisMap.add key (QF.add current v) acc
```

この体のゼロ元は空の Map です。

これを使えば、V' の要素を 1 項だけ持つ形で組み立てる関数と、加法・符号反転・定数倍がそのまま定義できます。

▲リスト 6.17　"elementary.ml"

```
(* 有理関数 u(x) の項をつくる *)
let of_rational (u: QF.t): t = add_term (Basis.Exp Quadratic.zero) u zero

(* u(x)*exp(ax) の項をつくる *)
let exp_term (u: QF.t) (a: Quadratic.t): t = add_term (Basis.Exp a) u zero

(* w(x)*log(x-a) の項をつくる *)
```

```
let log_term (w: QF.t) (a: Quadratic.t): t = add_term (Basis.Log a) w zero

(* 加法 *)
let add (v1: t) (v2: t): t =
  BasisMap.fold add_term v2 v1

(* 符号反転 *)
let neg (v: t): t = BasisMap.map QF.neg v

(* 定数倍 *)
let scale (k: Quadratic.t) (v: t): t =
  BasisMap.map (fun u -> QF.mul (of_scalar k) u) v
```

微分も、この Map の上でそのまま閉じます。

▲リスト 6.18　"elementary.ml"

```
(* 微分 *)
let differentiate (v: t): t =
  BasisMap.fold (fun key value acc ->
    match key with
    | Basis.Exp a ->
      (* (U*exp(ax))' = (U' + a*U)*exp(ax) *)
      let u' = QF.differentiate value in
      let au = QF.mul (of_scalar a) value in
      add_term (Basis.Exp a) (QF.add u' au) acc
    | Basis.Log a ->
      (* (U*log(x-a))' = U'*log(x-a) + U/(x-a) *)
      let w' = QF.differentiate value in
      let acc = add_term (Basis.Log a) w' acc in
      let pole = QF.div value (of_poly (linear a)) in
      add_term (Basis.Exp Quadratic.zero) pole acc
  ) v zero
```

どちらも、先ほど確認した微分について閉じているという性質を、そのままコードにしただけです。$\log(x-a)$ の項で分裂した有理関数部分 $U(x)/(x-a)$ が、`pole` として指数 0 の基底（有理関数部分）へ合流しています。

6.2.6　実装する 3 つのアルゴリズム

V' に住む答えを実際に求める方法は、部品の種類によって 3 通りに分かれます。

- 有理関数の積分: 部分分数分解が必要です。分母を 1 次式の積に因数分解し、根 α_i を求めなければなりませんが、本書がここまでに手に入れている解の公式は 2 次方程式までです。そこで、分母 $q(x)$ 自体の次数が 2 次までに収まる範囲に絞って実装します。3 次以上の既約な因子が残ってしまうと、根を求める手段がないので、その場合は積分をあきらめます。
- 多項式$\times\ e^{ax}$ の積分: 次数に関係なくいつでも解けます。
- 多項式$\times\ \log(x-a)$ の積分: これも次数に関係なくいつでも解けます。部分積分を 1 回使うだけで、残った積分が有理関数の積分（上の 1 つ目）にそのまま帰着するからです。

6.2.7 解ける例と解けない例

具体例で確認しておきましょう。「解けない」には、実は 2 種類あります。**そもそも誰にも解けない**（V' のどこにも答えが存在しない）場合と、**一般には解けるが、本書の実装は簡略化のためにあきらめている**場合です。

▼表 6.1　具体例

式	本書のアルゴリズムでの扱い
$1/((x-1)(x-2))$	解ける（有理関数、実根 2 つ）
$1/(x^2+1)$	解ける（有理関数、既約 2 次＝複素根 2 つ）
$1/(x-1)^2$	解ける（有理関数、多重度 2 の根。答えは $-1/(x-1)$ で log すら不要）
xe^x	解ける（多項式 $x \times e^x$）
$x^2\log(x-1)$	解ける（多項式 $x^2 \times \log(x-1)$、次数の制限なし）
e^{-x^2}	誰にも解けない（初等関数の範囲に答えが存在しない）
e^x/x	誰にも解けない（極を持つ有理関数× e^x、非初等的）
$1/(x^3+x+1)$	一般には解ける（3 次方程式を解けば 3 つの log 項になる）が、本実装は非対応

上から 5 つが実際に本書で解けるケースです。e^{-x^2} と e^x/x は、初等関数の範囲に答えが存在しない、正真正銘「誰にも解けない」例です（e^x/x の場合、**指数積分**（*Exponential Integral*）Ei(x) という非初等関数を新たに定義しない限り書き下せません）。e^{ax} の係数を多項式に限ったのは、まさにこの e^x/x のように極を持つ係数が混ざると初等的かどうかの判定自体が非自明になるケースを避けるためでした。

最後の $1/(x^3+x+1)$ は毛色が違います。3 次以上の既約因子も、理論的には何の問題もなく V の中に収まります。本書がここで対応しないのは答えが存在しないからではなく、3 次以上の因数分解に対応していないためです。

6.2.8 有理関数を積分する（a=0）

$p(x)/q(x)$ を積分することを考えます。まず、p を q で割った商と余りを求め、多項式部分と真分数部分に分けます（p の次数が q の次数以上なら、この多項式部分に意味のある項が出てきます）。多項式部分の積分は、べき乗則をそのまま逆に使うだけの簡単な計算です。

問題は、残った真分数 $r(x)/q(x)$（r の次数は q の次数未満）です。ここで使うのが部分分数分解です。分母 $q(x)$ の根 α が**単根**（多重度 1）であれば、

$$\frac{r(x)}{q(x)} = \sum \frac{c_i}{x - \alpha_i}, \qquad c_i = \frac{r(\alpha_i)}{q'(\alpha_i)} \tag{6.24}$$

という**留数公式**（*Residue Formula*）が成り立ちます。これさえわかれば、積分は $\sum c_i \log(x - \alpha_i)$ と、log の和として書き下せます。

根 α が多重度 2 の場合は、この公式のままでは使えません（$q'(\alpha) = 0$ になってしまいます）。そこで $q(x) = (x-\alpha)^2 h(x)$（$h(\alpha) \neq 0$）と分解すると、部分分数分解は

$$\frac{r(x)}{q(x)} = \frac{A}{x - \alpha} + \frac{B}{(x - \alpha)^2} + (h \text{ の他の根からの項}) \tag{6.25}$$

という形になります。両辺に $(x - \alpha)^2$ を掛けてから $x = \alpha$ を代入すると B が、もう一度微分してから $x = \alpha$ を代入すると A が求まります。

$$B = \frac{r(\alpha)}{h(\alpha)}, \qquad A = \frac{r'(\alpha)h(\alpha) - r(\alpha)h'(\alpha)}{h(\alpha)^2} \tag{6.26}$$

$B/(x - \alpha)^2$ の不定積分は $-B/(x - \alpha)$ という有理関数（対数にはなりません）なので、多重度 2 の根 α が積分全体に寄与する項は

$$A \log(x - \alpha) - \frac{B}{x - \alpha} \tag{6.27}$$

です。A は $\log(x - \alpha)$ の項に、$-B/(x - \alpha)$ は有理関数部分に、それぞれ合流させればよいわけです。

根 α_i 自体をどう求めるかを考えましょう。本書が対象にするのは分母 $q(x)$ の次数が高々 2 の場合だけなので、話は単純です。$q(x)$ が 1 次式であれば根はただ 1 つ、2 次式であれば 2 次方程式の解の公式でそのまま根が求まります（判別式が 0 なら 2 つの根が一致し、多重度 2 の根になります）。3 次以上の次数を持つ $q(x)$ は、そもそも本書の実装範囲外として例外を送出します。

実装してみましょう。まず、ここまで $p(x), q(x)$ と書いてきた有理数係数の多項式を表す型を用意しておきます。多項式の節でつくったファンクタ（`Polynomial.Make`）を有理数に適用するだけです。

▲リスト 6.19　"q_poly.ml"

```
include Polynomial.Make(Rational)
```

積分は、微分と違って Elementary.t を受け取りません。有理関数・多項式×指数関数・定数× log といった、特定の形の入力だけを受け取る、それぞれ独立した計算です。ですが、答えの型はいずれも Elementary.t なので、これらの積分関数は Elementary モジュール自身の中に置くことにします。こうしておく利点は、積分した結果をその場で微分し、もとの被積分関数に戻るかどうかを、モジュール自身の力で検算できることです。そのためにまず、Elementary に等価判定を用意しておきましょう。Q_rootp のときと同じように、係数が 0 の基底を取り除く正規化（normalize）を挟んでから比較します。

▲リスト 6.20　"elementary.ml"

```
(* 正規化: 係数が0の基底を取り除く *)
let normalize (v: t): t =
  BasisMap.filter (fun _ c -> not (QF.equal c QF.zero)) v

(* 等価判定 *)
let equal (x1: t) (x2: t): bool =
  BasisMap.equal QF.equal (normalize x1) (normalize x2)
```

これで、積分結果を微分した値と、もとの被積分関数とが一致するかどうかを比較できるようになりました。一致しなければ、それは積分の実装自体にバグがあるということなので、そこで処理を失敗させて知らせます。

▲リスト 6.21　"elementary.ml"

```
(* 積分結果を微分すると、もとの被積分関数(target)に戻ることを検算する。
   一致しなければ、積分の実装自体にバグがあることを意味するのでfailwithする *)
let check_derivative (name: string) (target: t) (result: t): unit =
  let diff = add (differentiate result) (neg target) in
  if not (equal diff zero) then
    failwith (name ^ ": differentiate(result)が元の被積分関数と一致しません")
```

これから作る積分関数は、どれも最後に、微分するとまた元の被積分関数に戻ることを検算してから結果を返します。

補助関数です。

▲リスト 6.22　"elementary.ml"

```
(* 係数の埋め込み *)
let embed (p: Q_poly.t): Quadratic_poly.t = List.map Quadratic.of_rational p

(* 多項式部分の積分 *)
let integrate_poly (s: Q_poly.t): Q_poly.t =
  Rational.zero ::
    List.mapi (fun i c -> Rational.div c (Rational.of_integer (i + 1))) s
```

1 次式 $x - a$ を表す多項式（linear）は、答えの型の節ですでに用意してあるので、ここで作り直す必要はありません。

先ほど導いた 2 つの式を、そのままコードにします。ここから先は Elementary

モジュールの中身なので、モジュール自身の関数（log_term・of_scalar など）をモジュール名なしでそのまま呼び出せます。

▲リスト 6.23　"elementary.ml"

```
(* 単根aの寄与: r(a)/q'(a) を係数とするlog項 *)
let simple_root_term
    (r: Quadratic_poly.t) (q': Quadratic_poly.t) (a: Quadratic.t)
  : t =
  let c =
    Quadratic.div (Quadratic_poly.eval r a) (Quadratic_poly.eval q' a)
  in
  log_term (of_scalar c) a

(* 二重根aの寄与: A*log(x-a) - B/(x-a) *)
let double_root_term
    (r: Quadratic_poly.t) (q: Quadratic_poly.t) (a: Quadratic.t)
  : t =
  let lin = linear a in
  let h =
    fst (Quadratic_poly.div_rem (fst (Quadratic_poly.div_rem q lin)) lin)
  in
  let h' = Quadratic_poly.differentiate h in
  let r' = Quadratic_poly.differentiate r in
  let r_a = Quadratic_poly.eval r a in
  let h_a = Quadratic_poly.eval h a in
  let r'_a = Quadratic_poly.eval r' a in
  let h'_a = Quadratic_poly.eval h' a in
  let b = Quadratic.div r_a h_a in
  let a_coeff =
    Quadratic.div
      (Quadratic.sub (Quadratic.mul r'_a h_a) (Quadratic.mul r_a h'_a))
      (Quadratic.mul h_a h_a)
  in
  let log_part = log_term (of_scalar a_coeff) a in
  let pole = QF.div (of_scalar (Quadratic.neg b)) (of_poly lin) in
  add log_part (of_rational pole)
```

h は $q(x)/(x-\alpha)^2$ を計算したものです。あとは先ほどの A・B の式をそのままコードにし、それぞれの基底に載せています。

根をすべて求める部分です。

▲リスト 6.24　"elementary.ml"

```
(* 分母qの次数から、直接すべての根を求める *)
let roots_of (q: Q_poly.t): (Quadratic.t * int) list =
  match Q_poly.degree q with
  | d when d <= 0 -> []
  | 1 ->
    let c0 = List.nth q 0 in
    let c1 = List.nth q 1 in
    [ (Quadratic.of_rational (Rational.div (Rational.neg c0) c1), 1) ]
  | 2 ->
    let c0 = List.nth q 0 in
    let c1 = List.nth q 1 in
    let c2 = List.nth q 2 in
    let (x1, x2) = Quadratic.solve c2 c1 c0 in
    if Quadratic.equal x1 x2 then [ (x1, 2) ] else [ (x1, 1); (x2, 1) ]
  | _ ->
    failwith "Elementary.integrate_rational: denominator has degree >= 3"
```

次数が 1 なら根はただ 1 つ、2 なら 2 次方程式の求解（`Quadratic.solve`）で 2 つの根を求め、その 2 つが一致していれば多重度 2 の根として 1 つにまとめます。3 次以上であれば、ここで例外を送出します。

本体の積分処理です。最後に、微分してもとの $p(x)/q(x)$ に戻ることを検算してから結果を返します。

▲リスト 6.25　"elementary.ml"

```
(* 有理関数 p/q の積分 *)
let integrate_rational (p: Q_poly.t) (q: Q_poly.t): t =
  let (s, r_rem) = Q_poly.div_rem p q in
  let all_roots = roots_of q in
  let q_c = embed q in
  let q'_c = embed (Q_poly.differentiate q) in
  let r_c = embed r_rem in
  let result =
    List.fold_left (fun acc (root, mult) ->
      let term =
        match mult with
        | 1 -> simple_root_term r_c q'_c root
        | 2 -> double_root_term r_c q_c root
        | _ ->
          failwith
            ("Elementary.integrate_rational: a root with multiplicity >= 3 " ^
             "is unsupported")
      in
      add acc term
    ) (of_rational (of_poly (embed (integrate_poly s))))
      all_roots
  in
  check_derivative "Elementary.integrate_rational" (of_rational (embed p, embed q)) result;
  result
```

求まったすべての根について、多重度に応じた項を計算し、多項式部分に足し合わせていきます。

6.2.9　テスト

いくつか試してみましょう。分母の次数が 3 以上になるケース（既約 3 次多項式）も、きちんと失敗することを確認するためにあえて含めます。

▲リスト 6.26　partial_fraction_test.ml

```
module Elementary_expr = Expr.Elementary.Make (Q_rootp) (Quadratic) (Quadratic_poly) (
Elementary)

let show p q =
  (try
     let r = Elementary.integrate_rational p q in
     let integrand = Expr.frac (Expr.Q_poly.to_expr p) (Expr.Q_poly.to_expr q) in
     Expr.put (Expr.integral_to_string "x" integrand
       (Expr.to_string_of (module Elementary_expr) r))
   with Failure msg -> print_endline ("failwith: " ^ msg))

let () =
```

```
(* 実根のみ: 1/((x-1)(x-2)) = 1/(x-2) - 1/(x-1) *)
show [Rational.of_integer 1] [(2,1); (-3,1); (1,1)];

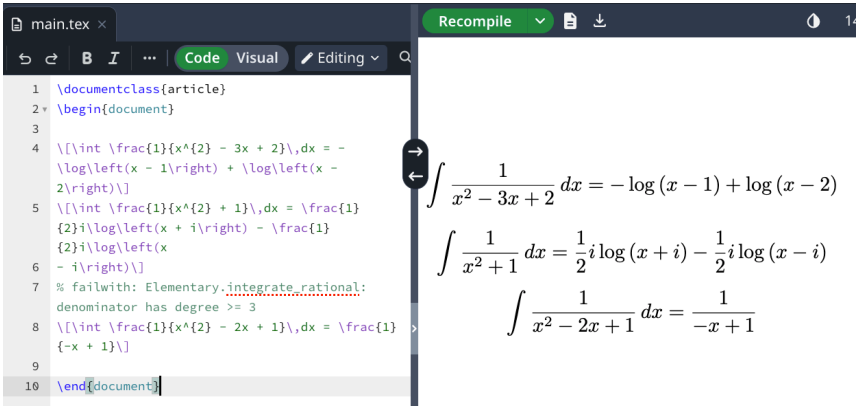
(* 既約2次(複素根): 1/(x^2+1) *)
show [Rational.of_integer 1] [(1,1); (0,1); (1,1)];

(* 3次以上が残る: 1/(x^3+x+1)（有理根なし、既約3次） *)
show [Rational.of_integer 1] [(1,1); (1,1); (0,1); (1,1)];

(* 重根: 1/(x-1)^2 = 1/(x^2 - 2x + 1) *)
show [Rational.of_integer 1] [(1,1); (-2,1); (1,1)]
```

`Elementary` も `Q_rootp` と同じく内部が `Map` の抽象型なので、`Expr.Quadratic` などと同じくファンクタになっています。ただし `Elementary` だけは、指数の項が有理関数部分かどうかの判定・分母がちょうど 1 かどうかの判定に `Q_rootp` の表示情報だけでは足りないため、`Quadratic`・`Quadratic_poly` の実体もあわせて渡します。

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml complexical.ml vector_space.ml polynomial.ml field_extension
.ml q_poly.ml q_rootp.ml quadratic.ml quadratic_poly.ml elementary.ml to_tex.cmxa
partial_fraction_test.ml && ./a.out
>>
\[\int \frac{1}{x^{2} - 3x + 2}\,dx = -\log\left(x - 1\right) + \log\left(x - 2\right)\]
\[\int \frac{1}{x^{2} + 1}\,dx = \frac{1}{2}i\log\left(x + i\right) - \frac{1}{2}i\log\left(x
 - i\right)\]
failwith: Elementary.integrate_rational: denominator has degree >= 3
\[\int \frac{1}{x^{2} - 2x + 1}\,dx = \frac{1}{-x + 1}\]
```



▲図 6.6　ブラウザ上でチェック

1 つ目は単根だけの素直なケースです。2 つ目がおもしろいところで、x^2+1 は実数の範囲では既約な多項式ですが、2 次方程式の求解に根の虚数・実数を区別させて

いないので、何の特別扱いもなく複素数の根 $\pm i$ が求まり、複素数係数の log の和として答えが出てきます。実は、この 2 つの複素対数の組み合わせは

$$-\frac{i}{2}\log(x-i)+\frac{i}{2}\log(x+i)=\arctan x+C \tag{6.28}$$

となり、arctan と一致します。通常の教科書では既約 2 次の場合を特別扱いして arctan を持ち出しますが、本書の実装では、複素数を扱える体をすでに持っているおかげで、単根の場合とまったく同じ「log の和」という扱いで処理できてしまうわけです。3 つ目は、分母 $q(x)$ の次数が 3 以上（既約な 3 次多項式 x^3+x+1）のケースで、`roots_of` がそのまま例外を送出して弾かれることを確認しています。4 つ目が多重度 2 の根 $x=1$ だけからなるケースで、表 6.1 で先取りしたとおり、対数の係数 A がちょうど 0 になるため、答えは有理関数部分の極 $1/(-x+1)$ だけになり、log が一切登場しません。

6.2.10　多項式×指数関数を積分する（a ≠ 0）

こちらは $a=0$ の場合よりずっと簡単です。ただし、$a=0$ のときは有理関数 $r(x)$ 全体（分母 $q(x)$ の次数が 2 までという制限付き）を扱ったのに対し、ここで扱う $P(x)$ は多項式に限ります。極を持つ有理関数 $P(x)/Q(x)$ に e^{ax} が掛かるケース（先ほどの表 6.1 にあった e^x/x がまさにその例）は、この先扱いません。$P(x)e^{ax}$ の不定積分は、必ず $R(x)e^{ax}$（R の次数は P の次数と等しい）という同じ形になることが知られています。両辺を微分して係数を比較すると、

$$R'(x)+aR(x)=P(x) \tag{6.29}$$

という関係が得られます。R の最高次の係数は R' に寄与しないので、最高次から順に決まっていきます。

▲リスト 6.27　"elementary.ml"

```
(* 多項式かける指数関数 p(x)*exp(ax) の積分(aは0でない) *)
let integrate_exp (a: Rational.t) (p: Q_poly.t): t =
  if Rational.equal a Rational.zero then
    failwith
      ("Elementary.integrate_exp: a must be non-zero " ^
       "(use integrate_rational for a = 0)");
  let n = Q_poly.degree p in
  let result =
    if n < 0 then zero
    else
      let r = Array.make (n + 1) Rational.zero in
      for i = n downto 0 do
```

```
      let p_i = List.nth p i in
      let higher =
        if i = n then Rational.zero
        else Rational.mul (Rational.of_integer (i + 1)) r.(i + 1)
      in
      r.(i) <- Rational.div (Rational.sub p_i higher) a
    done;
    let r_poly : Quadratic_poly.t =
      Array.to_list r |> List.map Quadratic.of_rational
    in
    exp_term (of_poly r_poly) (Quadratic.of_rational a)
in
let target = exp_term (of_poly (embed p)) (Quadratic.of_rational a) in
check_derivative "Elementary.integrate_exp" target result;
result
```

i 次の係数は、1 つ上の次数の係数（すでに求まっている）を使って

$$r_i = \frac{p_i - (i+1) r_{i+1}}{a} \tag{6.30}$$

と決まります。配列に高次から順に埋めていくだけの、単純な逆算です。

返り値の型は、部分分数分解のときと同じ `Elementary.t` です。ここでは指数関数の基底にだけ値を載せています。

6.2.11 テスト

実際に試してみましょう。積分するだけでなく、その結果を微分して元の $P(x)e^{ax}$ に戻るかどうかもあわせて検算します。

▲リスト 6.28　integrate_exp_test.ml

```
module Elementary_expr = Expr.Elementary.Make (Q_rootp) (Quadratic) (Quadratic_poly) (
Elementary)

let check a p =
  let r = Elementary.integrate_exp a p in
  let integrand = Expr.product [
    Expr.Q_poly.to_expr p;
    Expr.pow (Expr.atom "e") (Expr.product [Expr.Rational.to_expr a; Expr.atom "x"]);
  ] in
  Expr.put (Expr.integral_to_string "x" integrand
    (Expr.to_string_of (module Elementary_expr) r));
  (* 検算: 積分結果を微分すると、元の被積分関数に戻るはず *)
  let diff = Expr.postfix_unop "'" Elementary.differentiate in
  Expr.put (Expr.unop_to_string (module Elementary_expr) diff r)

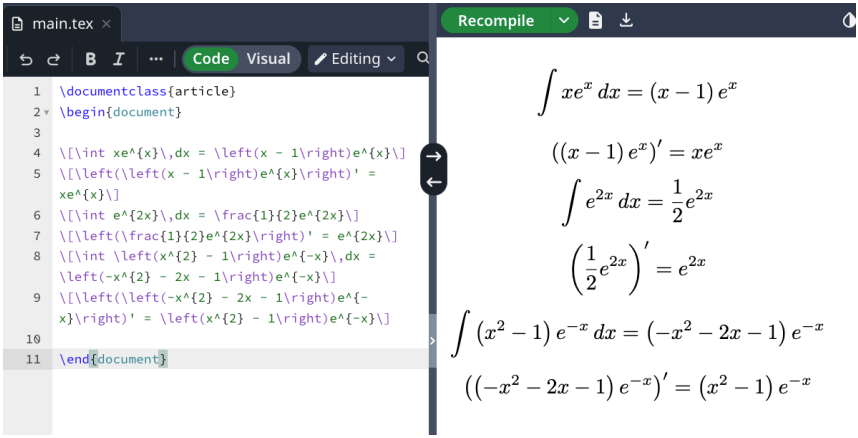
let () =
  (* ∫ x e^x dx = (x-1) e^x *)
  check (1, 1) [Rational.of_integer 0; Rational.of_integer 1];

  (* ∫ e^{2x} dx = (1/2) e^{2x} *)
  check (2, 1) [Rational.of_integer 1];

  (* ∫ (x^2 - 1) e^{-x} dx *)
```

```
  check (-1, 1)
    [Rational.of_integer (-1); Rational.of_integer 0; Rational.of_integer 1]
```

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml complexical.ml vector_space.ml polynomial.ml field_extension
.ml q_poly.ml q_rootp.ml quadratic.ml quadratic_poly.ml elementary.ml to_tex.cmxa
integrate_exp_test.ml && ./a.out
>>
\[\int xe^{x}\,dx = \left(x - 1\right)e^{x}\]
\[\left(\left(x - 1\right)e^{x}\right)' = xe^{x}\]
\[\int e^{2x}\,dx = \frac{1}{2}e^{2x}\]
\[\left(\frac{1}{2}e^{2x}\right)' = e^{2x}\]
\[\int \left(x^{2} - 1\right)e^{-x}\,dx = \left(-x^{2} - 2x - 1\right)e^{-x}\]
\[\left(\left(-x^{2} - 2x - 1\right)e^{-x}\right)' = \left(x^{2} - 1\right)e^{-x}\]
```



▲図 6.7　ブラウザ上でチェック

$$\int xe^x\,dx = (x-1)e^x \qquad \int e^{2x}\,dx = e^{2x}/2 \tag{6.31}$$

と、手計算どおりの結果になっており、3 つとも微分すると元の被積分関数に戻ることも確認できました。

6.2.12　多項式× $\log(x-a)$ を積分する

最後に残った、多項式× $\log(x-a)$ の積分です。$P(x)\log(x-a)$ の不定積分は、部分積分で求まります。$S(x) = \int P(x)\,dx$ とすると、

$$\int P(x)\log(x-a)\,dx = S(x)\log(x-a) - \int \frac{S(x)}{x-a}\,dx \tag{6.32}$$

という関係が成り立ちます。右辺に残った積分 $\int S(x)/(x-a)\,dx$ は、分母が $x-a$ という 1 次式だけの、ただの有理関数の積分です。つまり、さきほど作った `integrate_rational` にそのまま渡せます。新しいアルゴリズムを 1 つも書かずに、既存の道具の組み合わせだけで解けてしまうわけです。しかも分母が常に 1 次式なので、$P(x)$ がどんなに高い次数でも `roots_of` が失敗することはありません。有理関数の積分にあった次数の制限が、log の積分には要らない理由です。

▲リスト 6.29　"elementary.ml"

```
(* 多項式かけるlog(x-a) の積分。部分積分を使う。S(x) = ∫p(x)dx とすると、
   P(x)log(x-a)の不定積分は S(x)*log(x-a) - ∫S(x)/(x-a)dx となる。
   残った積分はただの有理関数の積分なので、integrate_rationalにそのまま
   帰着できる *)
let integrate_log (p: Q_poly.t) (a: Rational.t): t =
  let s = integrate_poly p in
  let q_lin : Q_poly.t = [Rational.neg a; Rational.one] in
  let rest = integrate_rational s q_lin in
  let a_q = Quadratic.of_rational a in
  let log_part = log_term (of_poly (embed s)) a_q in
  let result = add log_part (neg rest) in
  let target = log_term (of_poly (embed p)) a_q in
  check_derivative "Elementary.integrate_log" target result;
  result
```

6.2.13　テスト

定数係数から多項式係数まで試してみましょう。

▲リスト 6.30　integrate_log_test.ml

```
module Elementary_expr = Expr.Elementary.Make (Q_rootp) (Quadratic) (Quadratic_poly) (
Elementary)

let check p a =
  let r = Elementary.integrate_log p a in
  let integrand = Expr.product [
    Expr.Q_poly.to_expr p;
    Expr.fn "log" [Expr.sum [Expr.atom "x"; Expr.neg (Expr.Rational.to_expr a)]];
  ] in
  Expr.put (Expr.integral_to_string "x" integrand
    (Expr.to_string_of (module Elementary_expr) r));
  (* 検算: 積分結果を微分すると、元の被積分関数に戻るはず *)
  let diff = Expr.postfix_unop "'" Elementary.differentiate in
  Expr.put (Expr.unop_to_string (module Elementary_expr) diff r)

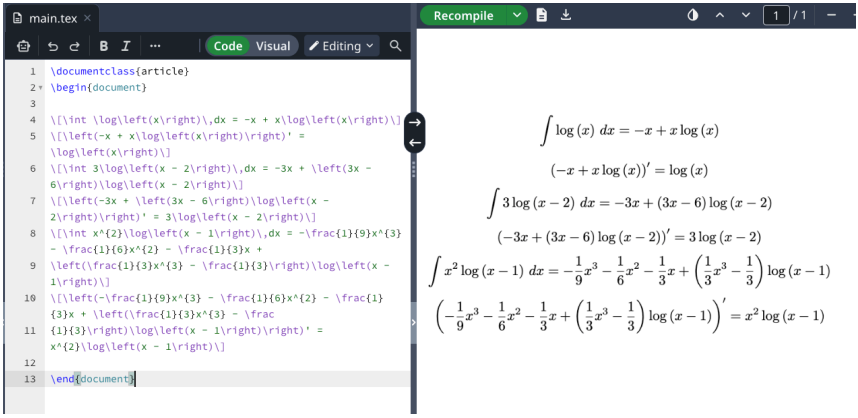
let () =
  (* log x (係数1) *)
  check [Rational.one] (0, 1);
```

```
  (* 3 log(x-2) (定数係数) *)
  check [Rational.of_integer 3] (2, 1);

  (* x^2 log(x-1) (多項式係数) *)
  check [Rational.zero; Rational.zero; Rational.one] (1, 1);

  (* (x+2) log(x+1) (多項式係数、1次) *)
  check [Rational.of_integer 2; Rational.one] (-1, 1)
```

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml complexical.ml vector_space.ml polynomial.ml field_extension
.ml q_poly.ml q_rootp.ml quadratic.ml quadratic_poly.ml elementary.ml to_tex.cmxa
integrate_log_test.ml && ./a.out
>>
\[\int \log\left(x\right)\,dx = -x + x\log\left(x\right)\]
\[\left(-x + x\log\left(x\right)\right)' = \log\left(x\right)\]
\[\int 3\log\left(x - 2\right)\,dx = -3x + \left(3x - 6\right)\log\left(x - 2\right)\]
\[\left(-3x + \left(3x - 6\right)\log\left(x - 2\right)\right)' = 3\log\left(x - 2\right)\]
\[\int x^{2}\log\left(x - 1\right)\,dx = -\frac{1}{9}x^{3} - \frac{1}{6}x^{2} - \frac{1}{3}x +
 \left(\frac{1}{3}x^{3} - \frac{1}{3}\right)\log\left(x - 1\right)\]
\[\left(-\frac{1}{9}x^{3} - \frac{1}{6}x^{2} - \frac{1}{3}x + \left(\frac{1}{3}x^{3} - \frac
{1}{3}\right)\log\left(x - 1\right)\right)' = x^{2}\log\left(x - 1\right)\]
\[\int \left(x + 2\right)\log\left(x + 1\right)\,dx = -\frac{1}{4}x^{2} - \frac{3}{2}x + \left
(\frac{1}{2}x^{2} + 2x + \frac{3}{2}\right)\log\left(x + 1\right)\]
\[\left(-\frac{1}{4}x^{2} - \frac{3}{2}x + \left(\frac{1}{2}x^{2} + 2x + \frac{3}{2}\right)\
log\left(x + 1\right)\right)' = \left(x + 2\right)\log\left(x + 1\right)\]
```



▲図 6.8　ブラウザ上でチェック

3 つ目・4 つ目の $x^2\log(x-1)$、$(x+2)\log(x+1)$ が、多項式係数まで扱えることを示す例です。答えの log の係数が、もはや定数ではなく多項式になっていることに注目してください。手計算（部分積分を 2 回展開）でもまったく同じ答えになることを確認済みです。

6.2.14　3 つの積分を 1 つにまとめる

有理関数・多項式× $\log(x-a)$・多項式× e^{ax} という 3 種類の被積分関数を、1 つの型にまとめておきましょう。呼び出す側が「今回はどのアルゴリズムを使うべきか」を毎回意識しなくて済むようにするためです。

▲リスト 6.31　"elementary.ml"

```
(* 被積分関数: 有理関数・多項式かけるlog(x-a)・多項式かけるexp(ax)の3種類しか
   受け取らない *)
type integrand =
  | Rational of Q_poly.t * Q_poly.t
  | Log of Q_poly.t * Rational.t
  | Exp of Rational.t * Q_poly.t

(* 積分。被積分関数の形に応じて、3つのアルゴリズムのどれを使うかを
   その場で振り分けるだけで、呼び出す側は分岐を意識しなくてよい *)
let integrate (i: integrand): t =
  match i with
  | Rational (p, q) -> integrate_rational p q
  | Log (p, a) -> integrate_log p a
  | Exp (a, p) -> integrate_exp a p
```

`integrand` という 3 択の型を作り、それぞれに対応するアルゴリズムへ振り分けるだけです。新しい計算は何もしていません。これで、`Elementary.integrate` を 1 回呼ぶだけで、有理関数でも log でも e^{ax} でも、内部で自動的に正しいアルゴリズムが選ばれて積分が実行されます。

▲リスト 6.32　integrate_all_test.ml

```
module Elementary_expr = Expr.Elementary.Make (Q_rootp) (Quadratic) (Quadratic_poly) (
Elementary)

let integrand_expr = function
  | Elementary.Rational (p, q) -> Expr.frac (Expr.Q_poly.to_expr p) (Expr.Q_poly.to_expr q)
  | Elementary.Log (p, a) ->
    Expr.product [
      Expr.Q_poly.to_expr p;
      Expr.fn "log" [Expr.sum [Expr.atom "x"; Expr.neg (Expr.Rational.to_expr a)]];
    ]
  | Elementary.Exp (a, p) ->
    Expr.product [
      Expr.Q_poly.to_expr p;
      Expr.pow (Expr.atom "e") (Expr.product [Expr.Rational.to_expr a; Expr.atom "x"]);
    ]

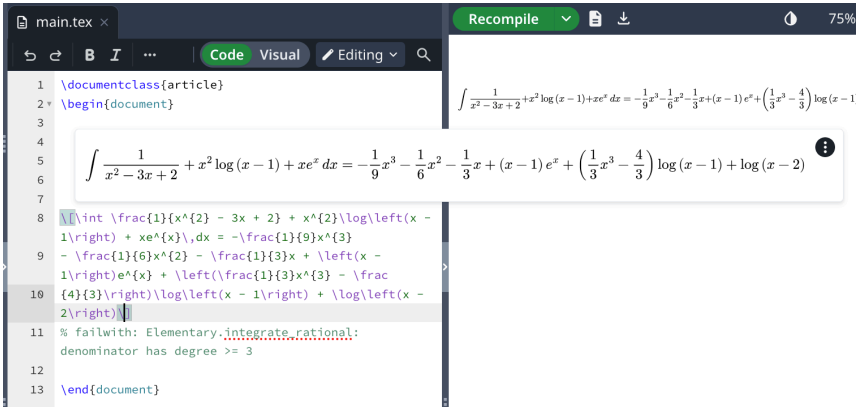
let show i =
  (try
     let r = Elementary.integrate i in
     Expr.put (Expr.integral_to_string "x" (integrand_expr i)
       (Expr.to_string_of (module Elementary_expr) r))
   with Failure msg -> print_endline ("failwith: " ^ msg))

(* 有理関数・多項式かけるlog・多項式かけるexpがすべて足し合わされた被積分関数を、
   項ごとにintegrateしてから足し合わせる(積分の線形性) *)
let show_sum items =
```

```
  let combined = List.fold_left
    (fun acc i -> Elementary.add acc (Elementary.integrate i))
    Elementary.zero items
  in
  let integrand = Expr.sum (List.map integrand_expr items) in
  Expr.put (Expr.integral_to_string "x" integrand
    (Expr.to_string_of (module Elementary_expr) combined))

let () =
  show_sum [
    Elementary.Rational ([Rational.one], [(2,1); (-3,1); (1,1)]);
    Elementary.Log ([Rational.zero; Rational.zero; Rational.one], (1, 1));
    Elementary.Exp ((1, 1), [Rational.zero; Rational.one]);
  ];
  (* 1/(x^3+x+1) (次数3以上、非対応) *)
  show (Elementary.Rational ([Rational.one], [(1,1); (1,1); (0,1); (1,1)]))
```

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml complexical.ml vector_space.ml polynomial.ml field_extension
.ml q_poly.ml q_rootp.ml quadratic.ml quadratic_poly.ml elementary.ml to_tex.cmxa
integrate_all_test.ml && ./a.out
>>
\[\int \frac{1}{x^{2} - 3x + 2} + x^{2}\log\left(x - 1\right) + xe^{x}\,dx = -\frac{1}{9}x^{3}
 - \frac{1}{6}x^{2} - \frac{1}{3}x + \left(x - 1\right)e^{x} + \left(\frac{1}{3}x^{3} - \frac
{4}{3}\right)\log\left(x - 1\right) + \log\left(x - 2\right)\]
failwith: Elementary.integrate_rational: denominator has degree >= 3
```



▲図 6.9　ブラウザ上でチェック

最初の例では、有理関数・log・e^{ax} の 3 種類すべてが足し合わされた被積分関数を扱っています。**integrate** はあくまで 1 種類ずつしか受け取れませんが、項ごとに振り分けて積分してから足し合わせるだけで、こうした混在した式も正しく積分できることが確認できます。2 つ目の例のように、対応できないケースは素直に失敗していることもあわせて確認できます。

6.3 2 階線形微分方程式を解く

積分で使った、答えの代数（Elementary）は、実は 2 階線形微分方程式のソルバーにもそのまま使えます。定数係数の同次方程式

$$y'' + by' + cy = 0 \quad (b, c \in \mathbb{Q}) \tag{6.33}$$

の解は、$y = e^{rx}$ の形を仮定して代入すると、

$$r^2 + br + c = 0 \tag{6.34}$$

という**特性方程式**（*Characteristic Equation*）を満たす r についての e^{rx} になることが知られています。この特性方程式は、ただの 2 次方程式です。2 次方程式の求解にそのまま投げてしまいましょう。

▲リスト 6.33　"linear_ode2.ml"

```
(* 求解 *)
let solve (b: Rational.t) (c: Rational.t): Elementary.t * Elementary.t =
  let (r1, r2) = Quadratic.solve Rational.one b c in
  (Elementary.exp_term (Elementary.of_scalar Quadratic.one) r1,
   Elementary.exp_term (Elementary.of_scalar Quadratic.one) r2)

(* 検算 *)
let check
    (b: Rational.t) (c: Rational.t) (y: Elementary.t): Elementary.t =
  let y' = Elementary.differentiate y in
  let y'' = Elementary.differentiate y' in
  Elementary.add y''
    (Elementary.add
       (Elementary.scale (Quadratic.of_rational b) y')
       (Elementary.scale (Quadratic.of_rational c) y))
```

検算は、y を 2 回微分して $y'' + by' + cy$ を計算するだけです。基本解であれば、これが恒等的に 0 になるはずです。

特性方程式の判別式が正であれば、r_1, r_2 は相異なる実数になり、$y_1 = e^{r_1 x}$, $y_2 = e^{r_2 x}$ という見慣れた指数関数の組が得られます。判別式が負であれば、2 次方程式の求解は複素数の根 r_1, r_2（互いに共役なペア）をそのまま返してきます。たとえば $y'' + y = 0$ なら、特性方程式は $r^2 + 1 = 0$ なので $r = \pm i$、つまり基本解は e^{ix} と e^{-ix} です。通常の教科書では、ここでオイラーの公式を持ち出して $\cos x$ と $\sin x$ という実数値の解に書き換えますが、本書の実装ではそうした書き換えを行わず、複素数のままの e^{ix} を 1 つの正当な解として扱います。

6.3.1 テスト

判別式が正の場合（異なる実根）と負の場合（複素共役根）の 2 パターンを試してみましょう。

▲リスト 6.34 linear_ode2_test.ml

```
module Elementary_expr = Expr.Elementary.Make (Q_rootp) (Quadratic) (Quadratic_poly) (
Elementary)

(* y''+by'+cy=0という式そのものをExpr.tで組み立てて表示する *)
let ode_str b c =
  let ode = Expr.sum [
    Expr.atom "y''";
    Expr.product [Expr.Rational.to_expr b; Expr.atom "y'"];
    Expr.product [Expr.Rational.to_expr c; Expr.atom "y"];
  ] in
  Printf.sprintf "%s = 0" (Expr.to_string ode)

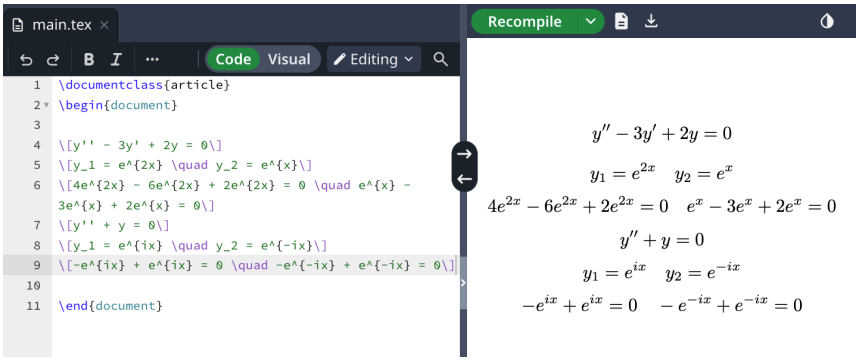
(* y''+by'+cyにyを代入した式を、項ごとに計算してからExpr.tで組み立てて表示する *)
let check_str b c y =
  let y' = Elementary.differentiate y in
  let y'' = Elementary.differentiate y' in
  let by' = Elementary.scale (Quadratic.of_rational b) y' in
  let cy = Elementary.scale (Quadratic.of_rational c) y in
  let lhs = Expr.sum [
    Elementary_expr.to_expr y'';
    Elementary_expr.to_expr by';
    Elementary_expr.to_expr cy;
  ] in
  Printf.sprintf "%s = %s" (Expr.to_string lhs)
    (Expr.to_string_of (module Elementary_expr) (Linear_ode2.check b c y))

let show b c =
  let (y1, y2) = Linear_ode2.solve b c in
  Expr.put (ode_str b c);
  Expr.put (Expr.join [
    Printf.sprintf "y_1 = %s" (Expr.to_string_of (module Elementary_expr) y1);
    Printf.sprintf "y_2 = %s" (Expr.to_string_of (module Elementary_expr) y2);
  ]);
  Expr.put (Expr.join [check_str b c y1; check_str b c y2])

let () =
  (* 判別式が正: y'' - 3y' + 2y = 0, r = 1, 2 *)
  show (-3, 1) (2, 1);

  (* 判別式が負: y'' + y = 0, r = i, -i *)
  show (0, 1) (1, 1)
```

```
ocamlopt add_monoid.ml mul_monoid.ml add_group.ml mul_group.ml euclidean.ml integer.ml ring.ml
 field.ml fraction.ml rational.ml complexical.ml vector_space.ml polynomial.ml field_extension
.ml q_poly.ml q_rootp.ml quadratic.ml quadratic_poly.ml elementary.ml linear_ode2.ml to_tex.
cmxa linear_ode2_test.ml && ./a.out
>>
\[y'' - 3y' + 2y = 0\]
\[y_1 = e^{2x} \quad y_2 = e^{x}\]
\[4e^{2x} - 6e^{2x} + 2e^{2x} = 0 \quad e^{x} - 3e^{x} + 2e^{x} = 0\]
\[y'' + y = 0\]
\[y_1 = e^{ix} \quad y_2 = e^{-ix}\]
\[-e^{ix} + e^{ix} = 0 \quad -e^{-ix} + e^{-ix} = 0\]
```



▲図 6.10　ブラウザ上でチェック

どちらのケースも、y に y_1 や y_2 を代入した式が、きちんと 0 になっていることが確認できます。

積分もこの 2 階線形微分方程式も、必要になる代数はここまで一貫して、二次方程式の解拡大モジュール（`Quadratic`）だけでした。多項式環（`Quadratic_poly`）も答えの代数（`Elementary`）も、二次方程式の解拡大モジュール（`Quadratic`）を土台に多項式ファンクタ（`Polynomial.Make`）・分数体ファンクタ（`Fraction.Make`）という既存のファンクタをもう一度適用しただけで、新しい代数を 1 から作ったわけではありません。

なお、ある初等関数の不定積分が初等関数で表せるかどうかを判定し、表せる場合には実際にその式を求めてしまう一般的な理論として、**リッシュのアルゴリズム**（*Risch Algorithm*）が知られています。本書が実装した部分分数分解や多項式×指数関数の積分は、この一般理論のうち扱いやすい特殊な場合（有理関数、および多項式と指数関数の積）に相当します。

■

あとがき

ここまで読み進めていただき、ありがとうございました。

前書きで書いたとおり、私は工学系の出身で、代数学を専門に学んだことはありません。だからこそ、群や体の定義を読んでも「それで結局何ができるのか」が長らくわかりませんでした。本書は、その疑問に決着をつけるために、群や体といった抽象的な代数の構造を実際に組み立て、動くところまで示した記録です。

数学的な構造を、プログラミング言語の型により、これほど素直に表現できることを、本書を通して実感していただけたなら幸いです。

本書のコードは GitHub(https://github.com/icchon/ocaml-algebraic-calculator-book) で公開しています。誤りがあれば、そちらを通じてご指摘いただければ幸いです。

参考文献

代数学（群・環・体）

- 原隆『手を動かしてまなぶ群論』
- 志賀浩二『群論への 30 講』
- 飯高茂『環論、これはおもしろい：素因数分解と循環小数への応用』
- 飯高茂『体論、これはおもしろい：方程式と体の理論』

超複素数

- I.L. Kantor, A.S. Solodovnikov『超複素数入門：多元環へのアプローチ』（笠原久弘訳）

方程式論

- 三宅克哉『方程式が織りなす代数学』

数式処理・記号積分

- Robert H. Risch, "The Problem of Integration in Finite Terms," Transactions of the American Mathematical Society, vol. 139, 1969, pp. 167-189.
- Manuel Bronstein, "Symbolic Integration I: Transcendental Functions," 2nd ed., Algorithms and Computation in Mathematics vol. 1, Springer, 2005.

OCaml・Re:VIEW

- The OCaml System, Documentation and User's Manual: `https://ocaml.org/manual/`
- Re:VIEW: `https://github.com/kmuto/review`

著者紹介

icchon
東京科学大学 工学院
ソフトウェアエンジニア

関数型言語 OCaml で作る代数計算機

2026 年 10 月 10 日　発行

著　者　icchon
編　集　icchon
印刷所　ねこのしっぽ

(C) 2026 icchon